



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

**ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ**

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

**Σύστημα Τεχνητής Νοημοσύνης για Αναγνώριση και
Περιγραφή Ρουχισμού σε Οπτικά Αναπήρους, με Χρήση
Ειδικά Ανεπτυγμένων Μοντέλων Βαθιάς Μάθησης**

Σωτήριος Δημητρακούλακος

**Επιβλέπων Καθηγητής:
Ηλίας Μαγκλογιάννης, Καθηγητής (Κοσμήτορας Σχολής)**

ΠΕΙΡΑΙΑΣ

ΙΟΥΛΙΟΣ 2025

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Σύστημα Τεχνητής Νοημοσύνης για Αναγνώριση και Περιγραφή Ρουχισμού σε Οπτικά
Αναπήρους, με Χρήση Ειδικά Ανεπτυγμένων Μοντέλων Βαθιάς Μάθησης

Σωτήριος Δημητρακουλάκος

A.M.: E20040

ΠΕΡΙΛΗΨΗ

Αντικείμενο της παρούσας πτυχιακής εργασίας είναι η ανάπτυξη ενός ολοκληρωμένου συστήματος τεχνητής νοημοσύνης για την αναγνώριση και περιγραφή ενδυμάτων, με σκοπό την προσβασιμότητα σε άτομα με οπτική αναπηρία. Το σύστημα συνδυάζει ειδικά ανεπτυγμένα μοντέλα βαθιάς μάθησης με μια κινητή εφαρμογή, επιτρέποντας τη λήψη ηχητικών περιγραφών χαρακτηριστικών ρουχισμού.

Η μεθοδολογία περιλαμβάνει την κατασκευή και εκπαίδευση μοντέλων ταξινόμησης εικόνας με βάση το σύνολο δεδομένων μόδας της Kaggle, με στόχο την πρόβλεψη των εξής χαρακτηριστικών: Γενική Κατηγορία, Υποκατηγορία, Ακριβής Κατηγορία (articleType), Βασικό Χρώμα, Φύλο, Εποχή, και Χρήση. Υλοποιήθηκαν τόσο πολυ-εξαγωγικά (multi-output), όσο και μονο-εξαγωγικά (single-output) μοντέλα, βασισμένα σε γνωστές αρχιτεκτονικές Συνελκτικών Νευρωνικών Δικτύων με χρήση Μεταφοράς Μάθησης, τεχνικών Ακριβούς Προσαρμογής, καθώς και κατάλληλης προ-επεξεργασίας, εκπαίδευσης και αξιολόγησης.

Η κινητή εφαρμογή του συστήματος υλοποιείται ως Android εφαρμογή, που επικοινωνεί με έναν Flask διακομιστή, μέσω HTTP API στο ίδιο δίκτυο (Wi-Fi). Ο χρήστης μπορεί να τραβήξει φωτογραφία ή να πλοηγηθεί στη συλλογή εικόνων της κινητής συσκευής του, να επιβεβαιώσει την εικόνα που επιθυμεί και να του ανακοινωθούν αποτελέσματα, όλα αποκλειστικά μέσω χειραφιών αφής και ηχητικής βοήθειας.

Κατά την αξιολόγηση του συστήματος, τα τελικό μονο-εξαγωγικό και πολύ-εξαγωγικό μοντέλο, απέδωσαν με υψηλή συνέπεια σε πολλαπλά χαρακτηριστικά. Η συνολική λειτουργικότητα της εφαρμογής κρίθηκε επιτυχής σε σενάρια πραγματικής χρήσης, με ομαλή εμπειρία για άτομα χωρίς οπτική πρόσβαση.

ΘΕΜΑΤΙΚΗ ΠΕΡΙΟΧΗ: Η θεματική περιοχή της παρούσας εργασίας είναι η ανάπτυξη ενός συστήματος τεχνητής νοημοσύνης και βαθιάς μάθησης, με σκοπό την αναγνώριση χαρακτηριστικών ρουχισμού, μέσω μιας κινητής εφαρμογής, που υποστηρίζει άτομα με οπτική αναπηρία στην καθημερινότητά τους.

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: τεχνητή νοημοσύνη, βαθιά μάθηση, ταξινόμηση ρούχων, προσβασιμότητα, κινητή εφαρμογή

ABSTRACT

The subject of this undergraduate thesis is the development of a complete artificial intelligence system for the recognition and description of clothing, aimed at improving accessibility for individuals with visual impairments. The system combines custom developed deep learning models with a mobile application, enabling the delivery of auditory descriptions of clothing attributes.

The methodology includes the construction and training of image classification models on Kaggle's fashion dataset, targeting the prediction of the following attributes: Main Category, Subcategory, Precise Category (articleType), Base Color, Gender, Season, and Usage. Both multi-output and single-output models were implemented, based on known Convolutional Neural Network (CNN) architectures, employing Transfer Learning, Fine-Tuning techniques, as well as appropriate preprocessing, training, and evaluation.

The mobile system is implemented as an Android application that communicates with a server via an HTTP API over the same Wi-Fi network. The user can capture a photo or browse the device's image gallery, confirm the desired image, and receive spoken results, all exclusively through touch gestures and auditory assistance.

During system evaluation, the final single-output and multi-output models demonstrated high consistency across multiple attributes. The overall application functionality was deemed successful in real-use scenarios, providing a smooth user experience for individuals without visual access.

SUBJECT AREA: The thematic area of this thesis is the development of an artificial intelligence and deep learning system for the recognition of clothing attributes, delivered through a mobile application designed to assist visually impaired individuals in their daily lives.

KEYWORDS: artificial intelligence, deep learning, clothing classification, accessibility, mobile application

Η παρούσα πτυχιακή εργασία είναι αφιερωμένη στην οικογένειά μου.

ΕΥΧΑΡΙΣΤΙΕΣ

Για τη διεκπεραίωση της παρούσας Πτυχιακής Εργασίας, θα ήθελα να ευχαριστήσω τον Καθηγητή κ. Ηλία Μαγκλογιάννη και τον Επίκουρο Καθηγητή κ. Ανδρέα Μενύχτα, για τη συνεργασία και την πολύτιμη συμβολή του στην ολοκλήρωση της.

ΠΕΡΙΕΧΟΜΕΝΑ

<u>ΠΡΟΛΟΓΟΣ</u>	15
<u>1. ΕΙΣΑΓΩΓΗ</u>	16
<u>2. ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ</u>	18
<u>2.1 Τεχνητή Νοημοσύνη και Μηχανική Μάθηση</u>	18
<u>2.2 Βαθιά Μάθηση, Αναγνώριση Προτύπων και Συνελκτικά Νευρωνικά Δίκτυα</u>	19
<u>2.3 Μεταφορά Μάθησης (Transfer Learning)</u>	20
<u>2.4 Πολυ-Ετικετική (Multi-Task) και Πολυ-Έξοδος (Multi-Output) Ταξινόμηση</u>	22
<u>2.5 Εφαρμογές Android και Διακομιστές Προγραμματιστικών Διεπαφών Εφαρμογών (API Servers)</u>	23
<u>2.6 Περίπτωση Χρήσης για την Δημιουργία Υπηρεσίας σε Οπτικά Ανάπηρους</u>	24
<u>3. ΑΝΑΠΤΥΞΗ ΜΟΝΤΕΛΩΝ ΒΑΘΙΑΣ ΜΑΘΗΣΗΣ</u>	25
<u>3.1 Περιγραφή του Συνόλου Δεδομένων (Dataset)</u>	25
<u>3.2 Προεπεξεργασία Δεδομένων</u>	27
<u>3.3 Αρχιτεκτονικές και Βάσεις Μοντέλων Ταξινόμησης</u>	30
<u>3.4 Αγωγός Εκπαίδευσης (Training Pipeline)</u>	34
<u>3.5 Πολυ-εξόδου Μοντέλα και Διαδικασία Εκπαίδευσης</u>	36
<u>3.5.1 Πρώτες Εκδόσεις</u>	39
<u>3.5.2 Δεύτερες Εκδόσεις</u>	40
<u>3.5.3 Τρίτες και Τελικές Εκδόσεις</u>	40
<u>3.6 Μονό-Εξόδου Μοντέλα (Single-Task) και Διαδικασία Εκπαίδευσης</u>	43
<u>3.7 Μέθοδοι αξιολόγησης και Μετρικά Απόδοσης</u>	45
<u>3.8 Μέθοδος Εκτέλεσης Αναγνώρισης</u>	47
<u>4. ΥΛΟΠΟΙΗΣΗ ΠΙΣΩ ΣΥΣΤΗΜΑΤΟΣ ΚΑΙ ΔΙΑΚΟΜΗΣΤΗ (SERVER)</u>	48

<u>4.1 Φόρτωση Δεδομένων και Μοντέλων</u>	48
<u>4.2 Διαχείριση Αιτημάτων Πρόβλεψης</u>	49
<u>5. ΑΝΑΠΤΥΞΗ ΚΙΝΗΤΗΣ ΕΦΑΡΜΟΓΗΣ</u>	50
<u>5.1 Γενική Περιγραφή Εφαρμογής</u>	50
<u>5.2 Αρχιτεκτονική Εφαρμογής και Ανάλυση Μερών</u>	50
<u>5.3 Διασύνδεση με τον Διακομιστή (Server), Διαχείριση Εικόνων και Αποτελεσμάτων</u>	53
<u>5.4 Χειρισμός από Άτομα με Οπτική Αναπηρία</u>	54
<u>5.5 Εγχειρίδιο Χρήστη και Περιγραφή Ροής Χρήστη</u>	56
<u>6. ΠΕΙΡΑΜΑΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΣΥΣΤΗΜΑΤΟΣ</u>	61
<u>6.1 Αναλυτική Αξιολόγηση και Επιλογή Τελικών Αλγορίθμων Ταξινόμηση</u>	61
<u>6.2 Δοκιμές Εφαρμογής σε Πραγματικές Συνθήκες</u>	74
<u>7. ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ ΚΑΙ ΒΕΛΤΙΩΣΕΙΣ</u>	86
<u>7.1 Υποστήριξη Ελληνικών και Άλλων Γλωσσών</u>	86
<u>7.2 Ενσωμάτωση Αναγνώρισης Επιπλέον Χαρακτηριστικών</u>	87
<u>7.3 Ενσωμάτωση Ανιχνευτή Αναγνώρισης Πλαισίου Έγκυρης Φωτογραφίας</u>	87
<u>7.4 Βελτίωση Ακρίβειας Μοντέλων (Επένδυση σε Δεδομένα και Υποδομές)</u>	87
<u>7.5 Ενσωμάτωση Μηχανικής Μάθησης για την Εκμάθηση Γούστου</u>	87
<u>7.6 Βελτίωση Εξωτερικής Εμφάνισης Εφαρμογής και Διαδικτυακή Διαθεσιμότητα</u>	88
<u>8. ΤΕΧΝΟΛΟΓΙΕΣ ΚΑΙ ΕΡΓΑΛΕΙΑ</u>	88
<u>8.1 Ανάπτυξη Μοντέλων Βαθιάς Μάθησης</u>	88
<u>8.2 Ανάπτυξη Διακομιστή και Κινητής Εφαρμογής</u>	88
<u>9. ΣΥΜΠΕΡΑΣΜΑΤΑ</u>	90

<u>ΠΙΝΑΚΑΣ ΟΡΟΛΟΓΙΑΣ</u>	92
<u>ΣΥΝΤΜΗΣΕΙΣ – ΑΡΚΤΙΚΟΛΕΞΑ – ΑΚΡΩΝΥΜΙΑ</u>	94
<u>ΠΑΡΑΡΤΗΜΑ Ι</u>	95
<u>ΠΑΡΑΡΤΗΜΑ ΙΙ</u>	98
<u>ΠΑΡΑΡΤΗΜΑ ΙΙΙ</u>	100
<u>ΠΑΡΑΡΤΗΜΑ ΙV</u>	102
<u>ΠΑΡΑΡΤΗΜΑ V</u>	104
<u>ΠΑΡΑΡΤΗΜΑ VI</u>	106
<u>ΠΑΡΑΡΤΗΜΑ VII</u>	114
<u>ΠΑΡΑΡΤΗΜΑ VIII</u>	118
<u>ΠΑΡΑΡΤΗΜΑ ΙΧ</u>	119
<u>ΠΑΡΑΡΤΗΜΑ Χ</u>	122
<u>ΠΑΡΑΡΤΗΜΑ ΧΙ</u>	126
<u>ΒΙΒΛΙΟΓΡΑΦΙΚΕΣ ΝΑΦΟΡΕΣ</u>	129

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

Σχήμα 1: Διαφορά Εποπτευόμενης Μάθησης (Supervised Learning) και η Μη Εποπτευόμενης Μάθησης (Unsupervised Learning)	18
Σχήμα 2: Σχηματική αναπαράσταση Νευρωνικού Δικτύου (Neural Network)	19
Σχήμα 3: Σχηματική Αναπαράσταση Μεταφοράς Μάθησης (Transfer Learning).....	21
Σχήμα 4: Σχηματική Αναπαράσταση Πολύ-Ετικετικής Μάθησης (Multi-Task Learning).	22
Σχήμα 5: Διαμοιρασμός δειγμάτων στις κλάσεις του articleType, πριν την εξισορρόπηση	27
Σχήμα 6: Σχηματική Αναπαράσταση Αρχιτεκτονικής ενός CNN (Convolutional Neural Network)	31
Σχήμα 7: Σχήμα Περιγραφής Σχέσης μεταξύ Χαρακτηριστικών.....	32
Σχήμα 7: Σχηματική Αναπαράσταση Αλληλεπίδρασης της Android Εφαρμογής στην Κινητή Συσκευή με τον Διακομιστή (Server)	49
Σχήμα 9: Σχηματική Αναπαράσταση Αρχιτεκτονικής της Android Εφαρμογής	51
Σχήμα 10: Πίνακας Σύγχυσης (Confusion Matrix) mainCategory	65
Σχήμα 11: Πίνακας Σύγχυσης (Confusion Matrix) subCategory	66
Σχήμα 12: Πίνακας Σύγχυσης (Confusion Matrix) baseColour	67
Σχήμα 12: Πίνακας Σύγχυσης (Confusion Matrix) gender	68
Σχήμα 14: Πίνακας Σύγχυσης (Confusion Matrix) season	69
Σχήμα 15: Πίνακας Σύγχυσης (Confusion Matrix) usage.....	70
Σχήμα 16: Πίνακας Σύγχυσης (Confusion Matrix) articleType	73

ΚΑΤΑΛΟΓΟΣ ΕΙΚΟΝΩΝ

Εικόνα 1: Κελί κώδικα εξισορρόπησης και διαχωρισμού του συνόλου δεδομένων, από το Python Notebook	27
Εικόνα 2: Κελί κώδικα ελέγχου ελλειπών ετικετών από το Python Notebook II	28
Εικόνα 3: Κελί συνάρτησης φιλτραρίσματος από το Python Notebook	29
Εικόνα 4: Κελί επιπλέον φιλτραρίσματος από το Python Notebook	29
Εικόνα 5: Τελικό πλήθος δεδομένων από το Python Notebook	30
Εικόνα 6: Παράδειγμα Εκπαίδευσης 1ης Εποχής, Δεύτερης έκδοσης από το Python Notebook	33
Εικόνα 7: Τύπος Softmax Συνάρτησης Ενεργοποίησης (Activation Function)	38
Εικόνα 8: Κομμάτι κώδικα ανακοίνωσης αποτελεσμάτων από το ResultsFragment.kt ..	52
Εικόνα 9: Κομμάτι κώδικα έναρξης OkHttpClient από το NetworkUtils.kt	53
Εικόνα 10: Κομμάτι κώδικα συμπλήρωσης αδειών από το AndroidManifest.xml	53
Εικόνα 11: Κομμάτι κώδικα διαχείρισης συρσίματος (swipe-fling) από το CameraFragment.kt	54
Εικόνα 12: Κομμάτι κώδικα διαχείρισης αναπαραγωγής ήχου αλλαγής εικόνας από το ConfirmFragment.kt	55
Εικόνα 13: Κομμάτι κώδικα διαχείρισης αναπαραγωγής φωνητικής ομιλίας (Text-To-Speech / TTS) από το MainActivity.kt	55
Εικόνα 14: Οθόνη Κάμερας/Αρχική Οθόνη	56
Εικόνα 15: Οθόνη Επιβεβαίωσης	56
Εικόνα 16: Οθόνη Επιβεβαίωσης σε Εικόνες Συλλογής Συσκευής	57
Εικόνα 17: Οθόνη Αναμονής	58
Εικόνα 18: Οθόνη Αποτελεσμάτων σε Φωτογραφία	58
Εικόνα 19: Οθόνη Αποτελεσμάτων σε Εικόνα Συλλογής	59
Εικόνα 20: Δοκιμή Μοντέλων 1	74
Εικόνα 21: Δοκιμή Μοντέλων 2	75
Εικόνα 22: Δοκιμή Μοντέλων 3	75

<u>Εικόνα 23: Δοκιμή Μοντέλων 4</u>	76
<u>Εικόνα 24: Δοκιμή Μοντέλων 5</u>	76
<u>Εικόνα 25: Δοκιμή Μοντέλων 6</u>	77
<u>Εικόνα 26: Δοκιμή Μοντέλων 7</u>	77
<u>Εικόνα 27: Δοκιμή Μοντέλων 8</u>	78
<u>Εικόνα 28: Δοκιμή Μοντέλων 9</u>	78
<u>Εικόνα 29: Δοκιμή Μοντέλων 10</u>	79
<u>Εικόνα 30: Δοκιμή Μοντέλων 11</u>	79
<u>Εικόνα 31: Δοκιμή Μοντέλων 12</u>	80
<u>Εικόνα 32: Δοκιμή Μοντέλων 13</u>	80
<u>Εικόνα 33: Δοκιμή Μοντέλων 14</u>	81
<u>Εικόνα 34: Δοκιμή Μοντέλων 15</u>	81
<u>Εικόνα 35: Δοκιμή Μοντέλων 16</u>	82
<u>Εικόνα 36: Δοκιμή Μοντέλων 17</u>	82
<u>Εικόνα 37: Δοκιμή Μοντέλων 18</u>	83
<u>Εικόνα 38: Παράδειγμα Χρήσης Εφαρμογής 1</u>	83
<u>Εικόνα 39: Παράδειγμα Χρήσης Εφαρμογής 2</u>	84
<u>Εικόνα 40: Παράδειγμα Χρήσης Εφαρμογής 3</u>	84
<u>Εικόνα 41: Παράδειγμα Χρήσης Εφαρμογής 4</u>	85
<u>Εικόνα 42: Παράδειγμα Χρήσης Εφαρμογής 5</u>	85
<u>Εικόνα 43: Παράδειγμα Χρήσης Εφαρμογής 6</u>	86

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

<u>Πίνακας 1: Αποδόσεις EfficientNetB3 Πολύ-ετικετικών (Multi-Task) μοντέλων</u>	62
<u>Πίνακας 2: Αποδόσεις InceptionV3 Πολύ-ετικετικών (Multi-Task) μοντέλων</u>	63
<u>Πίνακας 3: Αποτελέσματα Τελικού Πολύ-ετικετικού (Multi-Task) μοντέλου EfficientNetB3 Fashion 3 (Final Weights)</u>	64
<u>Πίνακας 4: Αποδόσεις EfficientNetB0 artType Μονό-ετικετικού (Single-Task) μοντέλου</u>	71
<u>Πίνακας 5: Αποδόσεις EfficientNetB1 artType Μονό-ετικετικού (Single-Task) μοντέλου</u>	71
<u>Πίνακας 6: Αποδόσεις EfficientNetB3 artType Μονό-ετικετικού (Single-Task) μοντέλου</u>	72
<u>Πίνακας 7: Αποτελέσματα Τελικού Μονό-ετικετικού (Single-Task) μοντέλου EfficientNetB1 artType (Best Weights)</u>	72

ΠΡΟΛΟΓΟΣ

Η τεχνητή νοημοσύνη και τα συστήματα μηχανικής μάθησης έχουν σημειώσει αξιοσημείωτη πρόοδο, ειδικά την τελευταία δεκαετία, προσφέροντας δυνατότητες που μέχρι πρότινος θεωρούνταν αδύνατες. Η εξέλιξη των αλγορίθμων και η ραγδαία αύξηση των διαθέσιμων δεδομένων, καθώς και υποδομών, κατέστησαν εφικτή την ανάπτυξη εξειδικευμένων εφαρμογών και μοντέλων, ικανών για ακρίβειες και αποδόσεις εντυπωσιακών επιπέδων.

Πολλά τέτοια συστήματα χρησιμοποιούνται και στη βελτίωση της καθημερινότητας και την υποστήριξη ευάλωτων κοινωνικών ομάδων. Σε αυτό το πλαίσιο, ιδιαίτερο ενδιαφέρον παρουσιάζει η αξιοποίηση της τεχνητής νοημοσύνης για την υποβοήθηση ατόμων με οπτική αναπηρία στην πλοήγησή τους στον κόσμο και στην κατανόηση του περιβάλλοντός τους.

Η κατανόηση των ενδυματολογικών χαρακτηριστικών μέσα από την αυτόματη ανάλυση εικόνων αποτελεί ένα σύνθετο υπολογιστικό πρόβλημα, το οποίο απαιτεί την αναγνώριση πολλαπλών στοιχείων – όπως το είδος ρούχου, το χρώμα, η εποχικότητα ή ακόμα και το φύλο για το οποίο απευθύνεται το ένδυμα. Οι κλασικές μέθοδοι αναγνώρισης έχουν αποδειχθεί ανεπαρκείς και χρονοβόρες, για τη διαχείριση της ποικιλομορφίας και των διαφορών μεταξύ των ρούχων, καθιστώντας αναγκαία την υιοθέτηση πιο σύνθετων προσεγγίσεων βασισμένων στη Βαθιά Μάθηση (Deep Learning).

Η παρούσα πτυχιακή εργασία επικεντρώνεται στην ανάπτυξη ενός ολοκληρωμένου συστήματος αναγνώρισης και περιγραφής ρουχισμού, προσαρμοσμένου στις ανάγκες ατόμων με οπτική αναπηρία. Το σύστημα αυτό αποτελείται από ένα συνδυασμό ειδικά ανεπτυγμένων μοντέλων βαθιάς μάθησης, για την αναγνώριση χαρακτηριστικών από εικόνες ρούχων, καθώς και από μια εφαρμογή κινητού τηλεφώνου, η οποία επιτρέπει την λήψη και χρήση φωτογραφιών ρουχισμού προς αναγνώριση, μέσω ειδικά διαμορφωμένων ακουστικών διαδικασιών.

Η υλοποίηση του συνολικού συστήματος έχει σαν κύριο στόχο την όσο το δυνατόν καλύτερη διαχείριση των αναγκών των ατόμων με οπτική αναπηρία, τόσο κατά την ανάπτυξη των μοντέλων, όσο και κατά τη δημιουργία της αντίστοιχης κινητής εφαρμογής.

Η πτυχιακή εργασία αυτή αποσκοπεί στη μελέτη, σχεδίαση και υλοποίηση μιας λύσης που ενσωματώνει πρακτικά την τεχνητή νοημοσύνη σε καθημερινές ανάγκες προσβασιμότητας, θέτοντας τις βάσεις για μελλοντικές εξελίξεις και επεκτάσεις στον τομέα της υποστηρικτικής τεχνολογίας.

1. ΕΙΣΑΓΩΓΗ

Η παρούσα πτυχιακή εργασία εστιάζει στη δημιουργία ενός ολοκληρωμένου συστήματος τεχνητής νοημοσύνης, ειδικά σχεδιασμένου για την αναγνώριση και περιγραφή χαρακτηριστικών ενδυμάτων, υποδημάτων και αντικειμένων μόδας, με απώτερο στόχο την υποβοήθηση ατόμων με οπτική αναπηρία. Η έμπνευση αυτής της ιδέας προήλθε από έρευνα και παρατηρήσεις πάνω στα προβλήματα και τις επιθυμίες των οπτικά ανάπηρων, ειδικά στο κομμάτι του ρουχισμού, κινητοποιώντας με έτσι να επιδιώξω την δημιουργία κατάλληλης τεχνολογίας, για την κάλυψη αυτής της ανάγκης. Το σύστημα αξιοποιεί τεχνικές βαθιάς μάθησης, καθώς περιλαμβάνει και την ανάπτυξη μιας ειδικά σχεδιασμένης Android εφαρμογής που χρησιμοποιεί ο οπτικά ανάπηρος χρήστης, για την λήψη και αποστολή εικόνων ρουχισμού. Κάθε τμήμα του συστήματος, από την ανάπτυξη των μοντέλων βαθιάς μάθησης, έως την κινητή εφαρμογή, είναι υλοποιημένα από το την αρχή και εξ ολοκλήρου, μέσα από χρήση μόνο βασικών βιβλιοθηκών που αναφέρονται παρακάτω, μελέτη και εκτενή προγραμματισμό.

Η εκπαίδευση των μοντέλων βασίστηκε στο σύνολο δεδομένων Fashion Product Images dataset της Kaggle, το οποίο περιλαμβάνει συνολικά περίπου 44 χιλιάδες εικόνες ενδυμάτων. Για τη βελτίωση της ποιότητας των δεδομένων και των προβλέψεων, το σύνολο δεδομένων και τα μετά-δεδομένα του προσαρμόστηκαν με εξισορρόπηση (σε συνδυασμό και με άλλη επεξεργασία που περιγράφεται παρακάτω), με αποτέλεσμα τη μείωση του μεγέθους του σε περίπου 17 χιλιάδες εικόνες. Οι ετικέτες ταξινόμησης που χρησιμοποιήθηκαν είναι οι εξής: masterCategory, subCategory, articleType, baseColour, gender, season και usage.

Το σύστημα περιλαμβάνει εκτενείς και λεπτές διαδικασίες προ-επεξεργασίας δεδομένων, κατασκευής μοντέλων νευρωνικών δικτύων, καθώς και την εκπαίδευση τους. Υλοποιήθηκαν τρεις εκδόσεις πολύ-εξαγωγικών μοντέλων βασισμένων στις αρχιτεκτονικές EfficientNetB3 και InceptionV3, για την πρόβλεψη όλων των προαναφερόμενων ετικετών (στα τελικά αποτελέσματα δεν χρησιμοποιήθηκε το articleType). Επιπρόσθετα, έγινε και η υλοποίηση τριών μονό-εξαγωγικών μοντέλων, βασισμένων στις αρχιτεκτονικές EfficientNetB0, EfficientNetB1 και EfficientNetB3, για την όσο το δυνατόν πιο ακριβή πρόβλεψη αποκλειστικά της ετικέτας articleType. Η επιλογή του συνόλου δεδομένων, καθώς και των συγκεκριμένων αρχιτεκτονικών βασίστηκε στις παρούσες διαθέσιμες υποδομές και δεδομένα, στη φύση των μετά-δεδομένων, στους περιορισμούς σε υπολογιστικό υλικό και στην βέλτιστη ισορροπία μεταξύ απόδοσης και υπολογιστικού κόστους.

Η εκπαιδευτική διαδικασία για τα πολύ-εξαγωγικά μοντέλα ακολούθησε τις εξής διακριτές εκδόσεις για καθεμία από τις δύο προαναφερόμενες αρχιτεκτονικές: Η πρώτη έκδοση περιλάμβανε εκπαίδευση χωρίς γενικεύσεις στις κατηγορίες της ετικέτας baseColour διατηρώντας όλες τις αρχικές κλάσεις του συνόλου δεδομένων, βελτιστοποίηση πάνω στις υπέρ-παραμέτρους, πολλές εποχές εκπαίδευσης πάνω σε όλες τις ετικέτες πριν το fine-tuning και λιγότερες εποχές επανεκπαίδευσης κατά τη δεύτερη φάση, μετά το fine tuning. Στη δεύτερη έκδοση εφαρμόστηκε συγχώνευση παρόμοιων κλάσεων χρωμάτων σε πιο γενικές, με στόχο τη μείωση του θορύβου, καθώς και ακολουθήθηκε και η ίδια μεθοδολογία εκπαίδευσης με την πρώτη έκδοση. Η τρίτη έκδοση βασίστηκε σε μια ειδικευμένη στρατηγική δύο φάσεων, όπου το μοντέλο προ-εκπαιδεύτηκε αρχικά αποκλειστικά πάνω στην ετικέτα articleType για μερικές εποχές και στη συνέχεια συνεχίστηκε η εκπαίδευση του στη δεύτερη φάση, αλλά αυτή τη φορά πάνω σε όλες τις ετικέτες, συνεχίζοντας από τα αποθηκευμένα βάρη της πρώτης φάσης. Η συγκεκριμένη έκδοση συνοδεύτηκε από περαιτέρω βελτιώσεις στα επίπεδα και τις υπέρ-παραμέτρους,

όπως χρήση κανονικοποίησης σε επίπεδα του δικτύου, ρυθμιζόμενα βάρη σε δύσκολες ετικέτες όπως season και baseColour και πιο εντατική ενίσχυση δεδομένων για καλύτερη γενίκευση.

Αντίστοιχα για τα μονό-εξαγωγικά μοντέλα η εκπαίδευση ακολούθησε μονοφασική προσέγγιση όπου κάθε μοντέλο εξειδικευόταν σε μία μόνο ετικέτα articleType. Πραγματοποιήθηκε απευθείας fine-tuning και εκπαίδευση σε συγκεκριμένα επίπεδα του κάθε μοντέλου, καθώς και εφαρμογή ειδικά προσαρμοσμένων τεχνικών και υπέρ-παραμέτρων για το καθένα.

Η εφαρμογή Android αποτελείται από διακριτές οθόνες που οργανώνονται σε μορφή τμημάτων (fragments), καθεμία υπεύθυνη για ένα συγκεκριμένο στάδιο της διαδικασίας χρήσης. Τα τμήματα που υλοποιήθηκαν και εφαρμόζονται όλα πάνω στη κύρια οθόνη MainActivity είναι τα εξής: CameraFragment, που επιτρέπει τη λήψη φωτογραφίας μέσω της κάμερας, το PlaceholderFragment που αποτελεί μια βοηθητική οθόνη που χρησιμοποιείται μόνο κατά την πρώτη χρήση της εφαρμογής, μέχρι να γίνει η αποδοχή των κατάλληλων αδειών, το ConfirmFragment, στο οποίο ο χρήστης μπορεί να επιβεβαιώσει ή να αλλάξει την επιλεγμένη εικόνα, το οποίο χρησιμοποιείται και για την πλοήγηση του χρήστη στις εικόνες της συλλογής του, το LoadingFragment, που εμφανίζεται κατά την αποστολή της εικόνας και την αναμονή των αποτελεσμάτων από τον διακομιστή, και το ResultsFragment, στο οποίο γίνεται η φωνητική ανακοίνωση (με χρήση τεχνολογίας Text-To-Speech) και παρουσίαση των αποτελεσμάτων. Επίσης υπάρχουν τα: GalleryNavigator που περιέχει μεθόδους που χρησιμοποιούνται από το ConfirmFragment για την περιήγηση στις εικόνες της συλλογής και το NetworkUtils που εξασφαλίζει όλη την επικοινωνία της εφαρμογής με τον διακομιστή. Όλες οι ενδιάμεσες μεταβάσεις συνοδεύονται από ακουστικά εφέ που υποστηρίζουν την πλοήγηση του χρήστη σε κάθε στάδιο της εφαρμογής χωρίς οπτική βοήθεια. Η εφαρμογή αυτή επικοινωνεί με έναν διακομιστή που τρέχει με φορτωμένα τα μοντέλα βαθιάς μάθησης σε μια εξωτερική τοπική συσκευή, ο οποίος δέχεται εικόνες από την κινητή συσκευή στην οποία βρίσκεται η εφαρμογή, εκτελεί προβλέψεις πάνω στις εικόνες, αναπαράγει αποτελέσματα και τα στέλνει πίσω στην κινητή εφαρμογή για να ανακοινωθούν.

Τα επόμενα κεφάλαια αναλύουν λεπτομερώς την παραπάνω αρχιτεκτονική και διαδικασία κατασκευής του συνολικού συστήματος, από την υλοποίηση και εκπαίδευση των μοντέλων, τη σχεδίαση της κινητής εφαρμογής και του διακομιστή, καθώς και την αξιολόγηση των αποτελεσμάτων υπογραμμίζοντας τη συμβολή της εργασίας στον τομέα της προσβασιμότητας, μέσω τεχνητής νοημοσύνης. Χωρίζονται σε τμήματα που αφορούν το θεωρητικό υπόβαθρο πάνω στο οποίο στηρίχτηκε η εργασία, την ανάπτυξη των μοντέλων βαθιάς μάθησης, την υλοποίηση του διακομιστή, την ανάπτυξη της κινητής εφαρμογής, πειραματικά αποτελέσματα και αξιολόγηση, καθώς και μελλοντικές επεκτάσεις – βελτιώσεις.

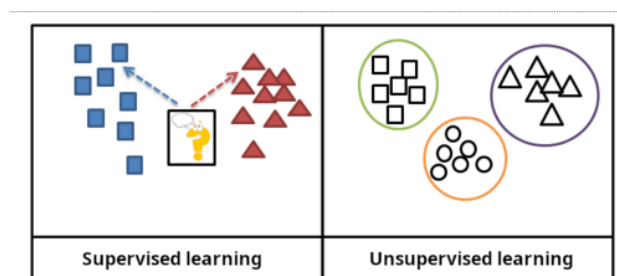
2. ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ ΚΑΙ ΕΡΕΥΝΑ

Σε αυτό το κεφάλαιο παρουσιάζεται το θεωρητικό υπόβαθρο πάνω στο οποίο βασίστηκε η ανάπτυξη του συστήματος. Εξετάζονται οι βασικές έννοιες της τεχνητής νοημοσύνης, της μηχανικής μάθησης και της βαθιάς μάθησης, καθώς και η αρχιτεκτονική των συνελκτικών νευρωνικών δικτύων. Επιπλέον, γίνεται αναφορά στις βασικές τεχνικές μεταφοράς μάθησης (transfer learning) και στην τεχνική της πολυ-ετικετικής ταξινόμησης (multi-label/multi-output classification), η οποία χρησιμοποιείται στην παρούσα εργασία. Τέλος, παρουσιάζονται γενικές αρχές σχεδίασης Android κινητών εφαρμογών, API διακομιστών (servers), καθώς και η μελέτη που έγινε στην περίπτωση χρήσης της δημιουργίας της υπηρεσίας αυτής, για χρήση από οπτικά αναπήρους

2.1 Τεχνητή Νοημοσύνη και Μηχανική Μάθηση

Η Τεχνητή Νοημοσύνη αποτελεί έναν από τους σημαντικότερους και ταχύτερα αναπτυσσόμενους κλάδους της επιστήμης των υπολογιστών, με στόχο την ανάπτυξη μηχανών που μπορούν να εκτελούν εργασίες οι οποίες απαιτούν ανθρώπινη νοημοσύνη. Σε αυτές περιλαμβάνονται η λήψη αποφάσεων, η αναγνώριση φωνής και εικόνας, η κατανόηση φυσικής γλώσσας και η ικανότητα μάθησης από εμπειρία. Οι πρώτες προσεγγίσεις βασίζονταν σε κανόνες (rule-based expert systems), ωστόσο αυτές ήταν περιορισμένες, καθώς δεν μπορούσαν να χειριστούν την πολυπλοκότητα και την αβεβαιότητα των πραγματικών δεδομένων. Η εξέλιξη της Τεχνητής Νοημοσύνης οδηγήθηκε μέσα από την ανάγκη για πιο ευέλικτες τεχνικές, ικανές να "μαθαίνουν" από παραδείγματα—κάτι που έφερε στο προσκήνιο τη Μηχανική Μάθηση.

Η Μηχανική Μάθηση (Machine Learning) είναι υπό-πεδίο της Τεχνητής Νοημοσύνης και εστιάζει στην ανάπτυξη αλγορίθμων που επιτρέπουν σε ένα σύστημα να ανακαλύπτει πρότυπα και να μαθαίνει από δεδομένα, χωρίς να έχει προγραμματιστεί ρητά για κάθε ενέργεια. Το βασικό συστατικό είναι η δημιουργία μοντέλων πρόβλεψης ή ταξινόμησης, τα οποία βελτιώνονται όσο αυξάνονται τα δεδομένα εισόδου. Υπάρχουν τρεις βασικές κατηγορίες Μηχανικής Μάθησης (Machine Learning): η Εποπτευόμενη Μάθηση (Supervised Learning), η Μη Εποπτευόμενη Μάθηση (Unsupervised Learning) και η Ενισχυτική Μάθηση (Reinforcement Learning). Η εποπτευόμενη μάθηση αφορά προβλήματα όπου έχουμε ετικέτες (labels), όπως η ταξινόμηση (classification) εικόνων. Η μη εποπτευόμενη στοχεύει στην ανακάλυψη υποκείμενων δομών (clusters) χωρίς ετικέτες, ενώ η ενισχυτική βασίζεται σε επιβράβευση/τιμωρία μετά από αλληλεπίδραση με το περιβάλλον. Η περίπτωση της παρούσας πτυχιακής κατηγοριοποιείται στον κλάδο της Εποπτευόμενης Μάθησης (Supervised Learning).



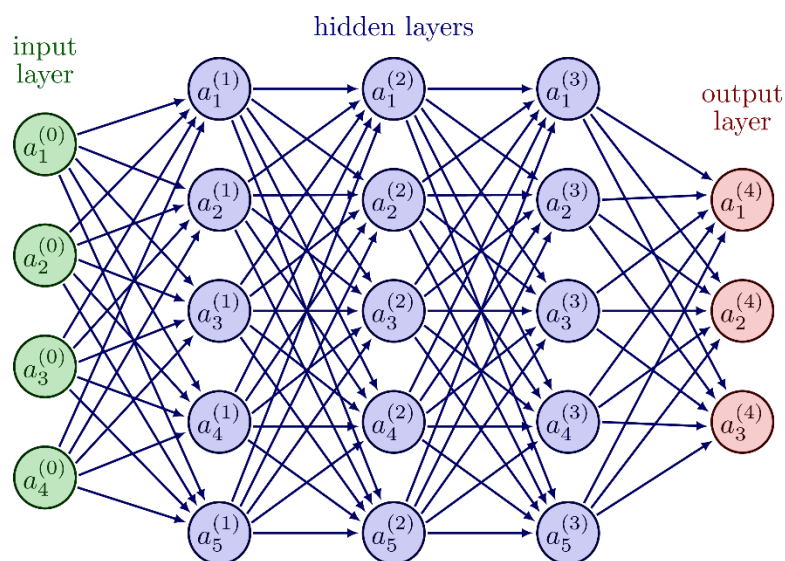
Σχήμα 1. Διαφορά Εποπτευόμενης Μάθησης (Supervised Learning) και η Μη Εποπτευόμενης Μάθησης (Unsupervised Learning)

Η εξέλιξη των υπολογιστικών πόρων, σε συνδυασμό με τη διαθεσιμότητα μεγάλων συνόλων δεδομένων (big data), έχει επιτρέψει στη μηχανική μάθηση να ξεπεράσει θεωρητικά εμπόδια και να βρει πρακτική εφαρμογή σε τομείς όπως η ιατρική διάγνωση, η αυτόνομη οδήγηση, οι χρηματοοικονομικές προβλέψεις, η κυβερνοασφάλεια και η ρομποτική. Η Τεχνητή Νοημοσύνη και η Μηχανική Μάθηση έχουν πλέον εισχωρήσει και στην καθημερινότητα των χρηστών, μέσω εργαλείων όπως ψηφιακοί βοηθοί, μηχανές σύστασης προϊόντων και υπηρεσίες μετάφρασης.

Η χρήση μηχανικής μάθησης σε εφαρμογές υποβοήθησης ατόμων με αναπηρίες είναι ιδιαίτερα σημαντική, καθώς δίνει τη δυνατότητα εξατομικευμένων, ευφυών λύσεων. Η παρούσα πτυχιακή εργασία αξιοποιεί ένα σημαντικό υποσύνολο της Μηχανικής Μάθησης, την Βαθιά Μάθηση (Deep Learning) για την αναγνώριση ενδυμάτων, χρησιμοποιώντας εικόνες από την κάμερα του κινητού και ειδικά ανεπτυγμ'όνα και εκπαιδευμένα μοντέλα για ταξινόμηση χαρακτηριστικών τους, προσφέροντας φωνητική ανατροφοδότηση σε άτομα με προβλήματα όρασης. Με τον τρόπο αυτό, η τεχνητή νοημοσύνη καθίσταται εργαλείο κοινωνικής ενσωμάτωσης και ενδυνάμωσης. [1], [3], [4], [9], [10]

2.2 Βαθιά Μάθηση, Αναγνώριση Προτύπων και Συνελικτικά Νευρωνικά Δίκτυα

Η Βαθιά Μάθηση (Deep Learning) αποτελεί ένα υποσύνολο της Μηχανικής Μάθησης και γενικότερα της Αναγνώρισης Προτύπων, που βασίζεται στη χρήση τεχνητών νευρωνικών δικτύων με πολλές ενδιάμεσες (κρυφές) στρώσεις-επίπεδα. Αυτά τα πολύ-επίπεδα δίκτυα μιμούνται τη δομή και λειτουργία του ανθρώπινου εγκεφάλου, επιτρέποντας τη σταδιακή εξαγωγή χαρακτηριστικών από τα δεδομένα, με αυξανόμενο επίπεδο αφαίρεσης. Για παράδειγμα, σε μια εικόνα ρούχου, οι πρώτες στρώσεις ενδέχεται να εντοπίσουν απλές ακμές και χρώματα, ενώ οι βαθύτερες στρώσεις αναγνωρίζουν σύνθετα σχήματα, κομμάτια ρουχισμού (π.χ. κουμπιά) ή ακόμα και ολόκληρα ρούχα. Η βαθιά μάθηση έχει αποδειχθεί ιδιαίτερα αποτελεσματική σε προβλήματα όπου οι κλασικές μέθοδοι της μηχανικής μάθησης αποτυγχάνουν, ειδικά όταν υπάρχουν μεγάλα και ποικιλόμορφα σύνολα δεδομένων.



Σχήμα 2. Σχηματική αναπαράσταση Νευρωνικού Δικτύου (Neural Network)

Ένα από τα σημαντικότερα είδη νευρωνικών δικτύων στη βαθιά μάθηση είναι τα Συνελικτικά Νευρωνικά Δίκτυα (Convolutional Neural Networks - CNNs). Τα CNNs είναι σχεδιασμένα ειδικά για την επεξεργασία εικόνων και εκμεταλλεύονται τοπικά πρότυπα και συσχετίσεις μεταξύ γειτονικών pixel. Χρησιμοποιούν ειδικές συνελικτικές στρώσεις (convolutional layers) που εφαρμόζουν φίλτρα (kernels) για να εξάγουν χαρακτηριστικά, όπως γωνίες, καμπύλες ή χρωματικές μεταβολές. Μετά τις συνελικτικές στρώσεις ακολουθούν συνήθως στρώσεις υπο-δειγματοληψίας (pooling layers), οι οποίες μειώνουν τη διαστατικότητα, διατηρώντας μόνο τα πιο σημαντικά χαρακτηριστικά. Αυτή η ιεραρχική εξαγωγή πληροφορίας καθιστά τα CNNs εξαιρετικά αποδοτικά για την αναγνώριση προτύπων εικόνας, με εφαρμογές στην ιατρική, την ασφάλεια, την αυτοκίνηση και φυσικά στη μόδα.

Η εκπαιδευτική διαδικασία των CNNs βασίζεται στη βελτιστοποίηση ενός αντικειμενικού συναρτήματος απώλειας μέσω οπισθοδιάδοσης (backpropagation) και βαθμίδωσης (gradient descent). Η προσαρμογή των βαρών του δικτύου επηρεάζεται από τεχνικές όπως η κανονικοποίηση παρτίδας (batch normalization), η εγκατάλειψη (dropout) και η ενίσχυση δεδομένων (data augmentation), ώστε να αποφεύγεται το φαινόμενο της υπερ-εκπαίδευσης (overfitting). Η υπερ-εκπαίδευση (overfitting) είναι ένα φαινόμενο που παρουσιάζεται και σε αρχικά στάδια της παρούσας πτυχιακής, κατά την οποία το κατά εκπαίδευση ή εκπαιδευμένο μοντέλο παίρνει αποφάσεις και κάνει προβλέψεις με μεγάλη αυτοπεποίθηση, επειδή έχει μάθει καλά πάνω στο σύνολο δεδομένων με το οποίο εκπαιδεύτηκε, χωρίς όμως να κάνει καλή αναγνώριση σε γενικότερες προβλέψεις και διαφορετικά δεδομένα, στα οποία δεν έχει την ίδια ακρίβεια. Ένα παράδειγμα περίπτωσης υπερ-εκπαίδευσης είναι όταν η ακρίβεια εκπαίδευσης (training accuracy) είναι πολύ ψηλά, όμως η ακρίβεια επιβεβαίωσης (validation accuracy) ή/και η ακρίβεια ελέγχου (testing accuracy) παραμένει χαμηλή.

Η αποτελεσματικότητα των CNNs ενισχύεται σημαντικά όταν εκπαιδεύονται σε μεγάλες βάσεις δεδομένων, όπως το ImageNet, και στη συνέχεια επαναχρησιμοποιούνται μέσω μεταφοράς μάθησης (transfer learning) για νέες εφαρμογές. Στην παρούσα εργασία αξιοποιούνται αρχιτεκτονικές CNN όπως τα EfficientNetB3 (συν B0 και B1) και InceptionV3, γνωστές για την υψηλή τους ακρίβεια και την αποδοτικότητα σε προβλήματα ταξινόμησης εικόνων.

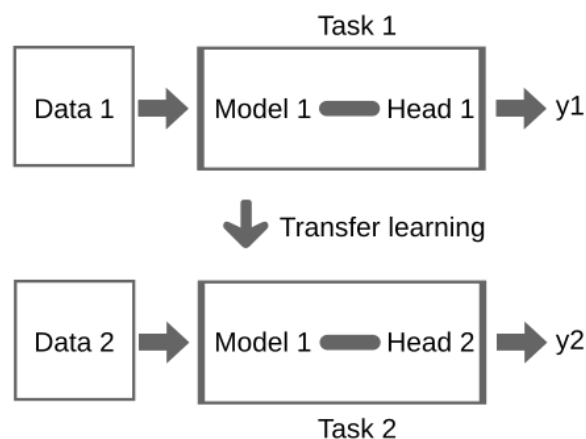
Η δύναμη των CNNs και της βαθιάς μάθησης γενικότερα έχει μεταμορφώσει την τεχνολογία, δίνοντας τη δυνατότητα σε συστήματα υπολογιστικής όρασης να «βλέπουν» και να κατανοούν το περιβάλλον τους. Σε ένα σύστημα για τυφλούς χρήστες, όπως αυτό της παρούσας εργασίας, η ικανότητα του CNN να αναγνωρίζει ρούχα και χαρακτηριστικά με ακρίβεια είναι κρίσιμη για τη χρησιμότητα της εφαρμογής. Με την κατάλληλη αρχιτεκτονική και προ-επεξεργασία εικόνων, τα δίκτυα βαθιάς μάθησης καθίστανται γέφυρα ανάμεσα στον φυσικό κόσμο και την αντιληπτική εμπειρία ατόμων με αναπηρία, προωθώντας την προσβασιμότητα και την ανεξαρτησία τους. [10], [11], [12]

2.3 Μεταφορά Μάθησης (Transfer Learning)

Η Μεταφορά Μάθησης (Transfer Learning) αποτελεί μια τεχνική στη μηχανική μάθηση και ειδικά στη βαθιά μάθηση, όπου ένα μοντέλο που έχει ήδη εκπαιδευτεί σε ένα μεγάλο σύνολο δεδομένων για ένα συγκεκριμένο πρόβλημα, επαναχρησιμοποιείται ως σημείο εκκίνησης ή βάση για την χτίση επιπλέον επιπέδων-στρώσεων πάνω του, την επεξεργασία του και την εκπαίδευση σε ένα διαφορετικό αλλά σχετικό πρόβλημα (και επομένως σύνολο δεδομένων). Αυτή η προσέγγιση αξιοποιεί τις γνώσεις που έχει ήδη αποκομίσει το μοντέλο από προηγούμενη εκπαίδευση, μειώνοντας σημαντικά τον απαιτούμενο χρόνο εκπαίδευσης, το κόστος υπολογιστικών πόρων, αλλά και την ανάγκη

για μεγάλη ποσότητα δεδομένων στο νέο πρόβλημα. Η μεταφορά μάθησης είναι ιδιαίτερα χρήσιμη σε περιπτώσεις όπου το νέο σύνολο δεδομένων είναι περιορισμένο ή δύσκολα συλλεκτό, επιτρέποντας τη δημιουργία ισχυρών μοντέλων με λιγότερη προσπάθεια. Στην περίπτωση της παρούσας πτυχιακής, χάρη της Μεταφοράς Μάθησης (Transfer Learning) επιτεύχθηκαν εντυπωσιακά αποτελέσματα, παρά τον περιορισμό που υπήρχε σε δεδομένα και υπολογιστικούς πόρους.

Η διαδικασία μεταφοράς μάθησης ξεκινά συνήθως από ένα προ-εκπαιδευμένο μοντέλο (pre-trained model), το οποίο έχει εκπαιδευτεί σε πολύ μεγάλο και γενικό σύνολο δεδομένων, όπως στην περίπτωση των μοντέλων που χρησιμοποιήσαμε, στο σύνολο ImageNet, που περιλαμβάνει εκατομμύρια εικόνες και χιλιάδες κατηγορίες. Τα πρώτα στρώματα αυτών των μοντέλων μαθαίνουν να εντοπίζουν βασικά χαρακτηριστικά, όπως άκρα, υφές και σχήματα, που είναι κοινά σε πολλές οπτικές αναπαραστάσεις. Στη συνέχεια, ανάλογα με το νέο πρόβλημα, οι τελευταίες στρώσεις του δικτύου είτε προσαρμόζονται (fine-tuning) με εκπαίδευση σε συγκεκριμένα δεδομένα είτε αντικαθίστανται πλήρως με νέες στρώσεις που εκπαιδεύονται από την αρχή. Η επιλογή μεταξύ πλήρους εκπαίδευσης, μερικού fine-tuning ή χρήσης του μοντέλου ως «σταθερού εξαγωγέα χαρακτηριστικών» εξαρτάται από το μέγεθος και τη φύση του νέου dataset. Το τι διαδικασία ακολουθήθηκε στην περίπτωση της παρούσας πτυχιακής, αναλύεται σε παρακάτω κεφάλαια.



Σχήμα 3. Σχηματική Αναπαράσταση Μεταφοράς Μάθησης (Transfer Learning)

Μια βασική θεωρητική υπόσταση της μεταφοράς μάθησης είναι η έννοια της γενίκευσης (generalization) και η απόδοση της γνώσης από μια περιοχή (domain) σε μια άλλη. Ενώ το προ-εκπαιδευμένο μοντέλο έχει μάθει να αναγνωρίζει γενικά μοτίβα από ένα μεγάλο σύνολο δεδομένων, είναι πιθανό τα μοτίβα αυτά να μην είναι απόλυτα εφαρμόσιμα στη νέα εργασία. Για το λόγο αυτό, το fine-tuning επιτρέπει την προσαρμογή του μοντέλου στα ειδικά χαρακτηριστικά της νέας περιοχής, χωρίς να χάσει τη βασική γνώση που έχει αποκτήσει. Με αυτόν τον τρόπο, η μεταφορά μάθησης επιτυγχάνει μια ισορροπία μεταξύ διατήρησης προηγούμενης γνώσης και απόκτησης νέας εξειδίκευσης.

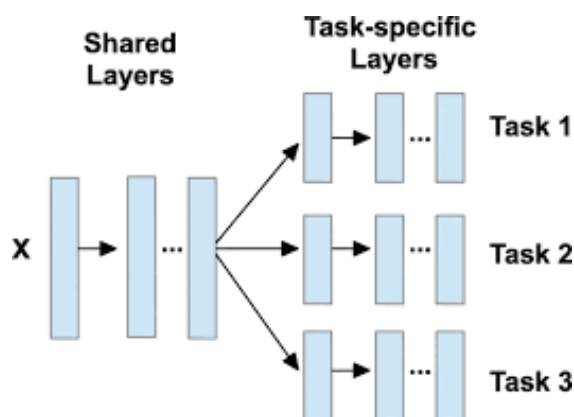
Η μεταφορά μάθησης έχει γίνει θεμελιώδης τεχνική σε πληθώρα εφαρμογών, όπως αναγνώριση εικόνας, επεξεργασία φυσικής γλώσσας, και ανάλυση βιοϊατρικών δεδομένων. Παράλληλα, έχει ενσωματωθεί σε πολλές δημοφιλείς αρχιτεκτονικές βαθιάς μάθησης, όπως τα transformers, VGG, ResNet, Inception, και EfficientNet. Επίσης, υπάρχουν διαφορετικές προσεγγίσεις μεταφοράς, όπως η μεταφορά χαρακτηριστικών (feature transfer), η μεταφορά παραμέτρων (parameter transfer) και η μεταφορά γνώσης (knowledge distillation). Η συνεχής έρευνα στο πεδίο βελτιώνει τις τεχνικές fine-tuning και

προσαρμογής μοντέλων, αυξάνοντας την αποδοτικότητα και την ακρίβεια σε νέα προβλήματα με περιορισμένα δεδομένα. [13], [14], [18], [19], [20], [21]

2.4 Πολυ-Ετικετική (Multi-Task) και Πολυ-Έξοδος (Multi-Output) Ταξινόμηση

Η πολυ-ετικετική (multi-task) και πολυ-έξοδος (multi-output) ταξινόμηση αποτελούν τεχνικές που επιτρέπουν σε ένα ενιαίο μοντέλο βαθιάς μάθησης να προβλέπει ταυτόχρονα πολλές κατηγορίες ή χαρακτηριστικά ενός δείγματος. Σε αντίθεση με την κλασική μονο-ετικετική προσέγγιση, όπου το μοντέλο εκπαιδεύεται να προβλέπει μία μόνο ετικέτα, οι multi-task και multi-output προσεγγίσεις συνδυάζουν πολλαπλά σχετικά καθήκοντα εκπαίδευσης σε μια ενιαία αρχιτεκτονική, βελτιώνοντας την αποδοτικότητα και τη γενίκευση. Αυτό είναι ιδιαίτερα χρήσιμο σε προβλήματα όπου οι ετικέτες ή χαρακτηριστικά έχουν κάποιο βαθμό συνάφειας, όπως στην ταξινόμηση ενδυμάτων με βάση πολλαπλά χαρακτηριστικά (τύπος, χρώμα, εποχή κτλ.).

Η πολυ-ετικετική μάθηση εκμεταλλεύεται την κοινή πληροφορία μεταξύ των διαφορετικών εργασιών, επιτρέποντας στο μοντέλο να μάθει πιο ισχυρά και γενικεύσιμα χαρακτηριστικά. Με τον τρόπο αυτό, κάθε εργασία λειτουργεί ως μια μορφή "κανονικοποίησης" για τις άλλες, μειώνοντας τον κίνδυνο υπερεκπαίδευσης (overfitting). Τα μοντέλα με πολλαπλές εξόδους, δηλαδή που παράγουν περισσότερες από μία προβλέψεις σε κάθε πέρασμα, χρησιμοποιούνται συχνά για να μάθουν ταυτόχρονα διαφορετικές ετικέτες, με ξεχωριστές "κεφαλές" εξόδου (output heads) που συνδέονται με μια κοινή βάση (shared backbone). Αυτό καθιστά τη διαδικασία πιο αποδοτική από το να εκπαιδεύονται πολλαπλά ανεξάρτητα μοντέλα για κάθε ετικέτα ξεχωριστά.



Σχήμα 4. Σχηματική Αναπαράσταση Πολύ-Ετικετικής Μάθησης (Multi-Task Learning)

Η εκπαίδευση πολυ-ετικετικών μοντέλων περιλαμβάνει την ορθή διαχείριση των πολλαπλών απωλειών (losses) που προκύπτουν από κάθε έξοδο. Η κοινή πρακτική είναι ο υπολογισμός μιας συνολικής απώλειας ως συνδυασμού των επιμέρους απωλειών, με κατάλληλη βαρύτητα (weighting) σε κάθε επιμέρους στόχο. Η επιλογή των βαρών αυτών μπορεί να γίνει είτε με σταθερές τιμές είτε δυναμικά κατά τη διάρκεια της εκπαίδευσης, ώστε να επιτευχθεί ισορροπία μεταξύ των στόχων. Με την κατάλληλη ρύθμιση, τα πολυ-ετικετικά μοντέλα μπορούν να βελτιώσουν την ακρίβεια σε όλες τις εργασίες σε σχέση με ανεξάρτητα μοντέλα, ειδικά όταν οι εργασίες έχουν κάποια σχέση μεταξύ τους.

Πέρα από τη βελτίωση της απόδοσης, τα πολυ-ετικετικά μοντέλα προσφέρουν σημαντικά πλεονεκτήματα όσον αφορά την αποδοτικότητα της εκπαίδευσης και την υπολογιστική

χρήση, αφού απαιτούν έναν μόνο κύκλο εκπαίδευσης και λιγότερη αποθήκευση παραμέτρων από το να εκπαιδεύονται ξεχωριστά μοντέλα. Επιπλέον, τέτοια μοντέλα διευκολύνουν την ολική ενσωμάτωση συστημάτων όπου η πρόβλεψη πολλών χαρακτηριστικών ή ετικετών είναι απαραίτητη, όπως συστήματα ανάλυσης κειμένου ή πολλαπλών σημάτων βιοϊατρικών εφαρμογών και στην περίπτωση της παρούσας εργασίας, συστήματα αναγνώρισης ρούχων. Η πολυ-ετικετική και πολυ-έξοδος μάθηση συνεχίζει να αποτελεί ενεργό πεδίο έρευνας με στόχο την αύξηση της αποδοτικότητας και της ικανότητας γενίκευσης σε πολύπλοκα προβλήματα. [15], [16], [17], [23], [24], [25]

2.5 Εφαρμογές Android και Διακομιστές Προγραμματιστικών Διεπαφών Εφαρμογών (API Servers)

Οι εφαρμογές Android αποτελούν ένα από τα πιο διαδεδομένα είδη λογισμικού για κινητές συσκευές, με τη λειτουργικότητα τους να καλύπτει μια πληθώρα πεδίων, από κοινωνικά δίκτυα και ψυχαγωγία μέχρι εξειδικευμένες επιχειρησιακές εφαρμογές και υποστήριξη ατόμων με αναπηρίες. Η ανάπτυξη αυτών των εφαρμογών γίνεται κυρίως μέσω της επίσημης πλατφόρμας Android Studio, που χρησιμοποιεί ως κύρια γλώσσα προγραμματισμού τη Java ή πιο πρόσφατα τη Kotlin.

Η δομή της διεπαφής χρήστη σε εφαρμογές Android βασίζεται συχνά στη χρήση των τμημάτων (fragments), τα οποία αποτελούν ανεξάρτητα, επαναχρησιμοποιήσιμα τμήματα UI που ενσωματώνονται σε μια Activity. Ένα Fragment διαθέτει τον δικό του κύκλο ζωής (onCreateView(), onCreateView(), onDestroyView() κ.λπ.) και μπορεί να αντικατασταθεί ή να προστεθεί δυναμικά μέσω του FragmentManager και FragmentTransaction, π.χ. με `supportFragmentManager.beginTransaction().replace(R.id.container, MyFragment()).commit()`. Αυτή η αρχιτεκτονική διευκολύνει την αποσύνδεση της λογικής παρουσίασης από τη βασική Activity και ενισχύει την επεκτασιμότητα και επαναχρησιμοποίηση του κώδικα.

Οι εφαρμογές αυτές έχουν τη δυνατότητα να αξιοποιούν πολλαπλούς πόρους της κινητής συσκευής, όπως την κάμερα, τον αισθητήρα κίνησης, το GPS και το μικρόφωνο, προσφέροντας μια διαδραστική εμπειρία στους χρήστες. Επιπλέον, για τη βελτίωση της προσβασιμότητας, ενσωματώνονται λειτουργίες φωνητικής υποστήριξης, χειρονομιών αφής και άλλα εργαλεία που διευκολύνουν τη χρήση από άτομα με ειδικές ανάγκες.

Η επικοινωνία των Android εφαρμογών με εξωτερικούς διακομιστές επιτυγχάνεται συνήθως μέσω προγραμματιστικών διεπαφών εφαρμογών (API), οι οποίες προσφέρουν μεθόδους για την αποστολή και λήψη δεδομένων. Η χρήση RESTful APIs είναι η πιο διαδεδομένη πρακτική, όπου ο client (εφαρμογή Android) κάνει αιτήματα HTTP (GET, POST, PUT, DELETE) προς τον server, ο οποίος επεξεργάζεται το αίτημα και επιστρέφει τις απαντήσεις συνήθως σε μορφή JSON. Η αρχιτεκτονική αυτή επιτρέπει τη διασύνδεση της κινητής εφαρμογής με συστήματα πίσω μέρους (backend), όπως βάσεις δεδομένων, συστήματα μηχανικής μάθησης και άλλες υπηρεσίες, εξασφαλίζοντας την αποδοτική διαχείριση πόρων και την επεκτασιμότητα των συστημάτων.

Οι διακομιστές προγραμματιστικών διεπαφών εφαρμογών (API servers) όπως αυτοί που υλοποιούνται με frameworks όπως το Flask στην Python, αποτελούν το πίσω μέρος (backend), μέρος που διαχειρίζεται αιτήματα από πολλές συσκευές ταυτόχρονα. Ο Flask διακομιστής (server) είναι ελαφρύς, εύκολος στη χρήση και εξαιρετικά επεκτάσιμος, επιτρέποντας την ταχεία ανάπτυξη RESTful APIs που συνδέουν μοντέλα τεχνητής νοημοσύνης, βάσεις δεδομένων και εφαρμογές. Στην περίπτωση εφαρμογών που απαιτούν real-time ή γρήγορη απόκριση, όπως η αποστολή εικόνων προς ανάλυση, η

χρήση API servers στο ίδιο τοπικό δίκτυο (Wi-Fi) εξασφαλίζει χαμηλή καθυστέρηση και σταθερή σύνδεση, αναβαθμίζοντας σημαντικά την εμπειρία χρήστη. [5], [6], [7]

2.6 Περίπτωση Χρήσης για την Δημιουργία Υπηρεσίας σε Οπτικά Ανάπηρους

Η έμπνευση για την ανάπτυξη μιας υπηρεσίας που στοχεύει στην υποστήριξη οπτικά ανάπηρων ατόμων προήλθε από ενδελεχή έρευνα και παρατηρήσεις γύρω από τις καθημερινές δυσκολίες που αντιμετωπίζουν, ειδικά στον τομέα της ένδυσης. Σύμφωνα με τον Παγκόσμιο Οργανισμό Υγείας (World Health Organization), υπάρχουν παραπάνω από 45 εκατομμύρια άτομα με οπτική αναπηρία παγκοσμίως. Η διαδικασία επιλογής και συνδυασμού ρούχων αποτελεί για πολλούς ένα αυτονόητο και εύκολο έργο, ωστόσο για τα άτομα με οπτική αναπηρία συχνά μετατρέπεται σε σημαντικό εμπόδιο, επηρεάζοντας αρνητικά την αυτονομία και την ποιότητα ζωής τους. Αυτή η ανάγκη οδήγησε στην επιθυμία δημιουργίας μιας τεχνολογικής λύσης που θα μπορούσε να αναγνωρίζει και να περιγράφει τα ρούχα με ακρίβεια, υποβοηθώντας έτσι τον χρήστη στην καθημερινή του επιλογή. Βασικός καταλύτης σε αυτή την κατεύθυνση αποτέλεσαν διαδικτυακές προσωπικότητες οπτικά ανάπηρων ατόμων, όπως το προφίλ της Anna Balan, που δημόσια εξέφρασαν την επιθυμία τους για εργαλεία και υπηρεσίες που θα καλύπτουν τέτοιες ανάγκες σε καταστήματα ρούχων, ενισχύοντας την κινητοποίηση και την προσωπική εμπλοκή για την υλοποίηση αυτής της ιδέας. Επιπλέον, οι συζητήσεις με οπτικά ανάπηρα άτομα στον περιβάλλοντα κύκλο μου επιβεβαίωσαν και ενίσχυσαν την σημασία και αναγκαιότητα της συγκεκριμένης εφαρμογής.

Η μελέτη της περίπτωσης χρήσης επικεντρώνεται στη δημιουργία ενός προσαρμοσμένου οικοσυστήματος που συνδυάζει την έξυπνη αναγνώριση εικόνων μέσω τεχνητής νοημοσύνης με μια προσβάσιμη και φιλική προς τον χρήστη διεπαφή. Η τεχνολογία αυτή έχει στόχο να γεφυρώσει το κενό ανάμεσα στην ψηφιακή πληροφορία και την αντίληψη των χρηστών με οπτική αναπηρία, προσφέροντας φωνητική περιγραφή των ρούχων, των χρωμάτων και των βασικών χαρακτηριστικών τους. Έτσι, ο χρήστης αποκτά τη δυνατότητα να λαμβάνει άμεσες και αξιόπιστες πληροφορίες, χωρίς την ανάγκη οπτικής επιβεβαίωσης από κάποιο τρίτο πρόσωπο. Η έρευνα αναδεικνύει τη σημασία της προσαρμογής της τεχνολογίας στις πραγματικές ανάγκες και τις συνήθειες των χρηστών, καθώς και την ενσωμάτωση των αρχών προσβασιμότητας στην ανάπτυξη εφαρμογών, ώστε να διασφαλιστεί η ευρεία χρήση και αποδοχή τους.

Τέλος, η συνεχής αλληλεπίδραση με την κοινότητα των οπτικά ανάπηρων χρηστών και η συλλογή ανατροφοδότησης είναι καθοριστικής σημασίας για τη βελτίωση και εξέλιξη της υπηρεσίας. Μέσω επιπλέον μελλοντικής συμμετοχικής σχεδίασης (participatory design) και επαναληπτικών δοκιμών, η υπηρεσία μπορεί να προσαρμοστεί και να εξελιχθεί ώστε να καλύπτει τις ανάγκες με μεγαλύτερη ακρίβεια και να υποστηρίζει καλύτερα την καθημερινότητα των χρηστών. Αυτή η προσέγγιση αποτελεί μια ηθική και πρακτική βάση για την ανάπτυξη τεχνολογιών με κοινωνικό αντίκτυπο και έμφαση στην ισότητα και την ένταξη. [6]

3. ΑΝΑΠΤΥΞΗ ΜΟΝΤΕΛΩΝ ΒΑΘΙΑΣ ΜΑΘΗΣΗΣ

Ακολουθεί η λεπτομερής ανάλυση όλης της διαδικασίας υλοποίησης, εκπαίδευσης και αξιολόγησης των μοντέλων συνελκτικών νευρωνικών δικτύων βαθιάς μάθησης, τα οποία χρησιμοποιούνται για τις προβλέψεις του συστήματος πάνω σε εικόνες ρουχισμού. Όλες οι παρακάτω διαδικασίες τμηματοποιήθηκαν στα ξεχωριστά αρχεία Python, που παρατίθενται στα Παραρτήματα στο τέλος του εγγράφου, με τα εξής ονόματα: `data_loading.py` (ΠΑΡΑΡΤΗΜΑ I), `preprocessing.py` (ΠΑΡΑΡΤΗΜΑ II), `efficientnet_model.py` (ΠΑΡΑΡΤΗΜΑ III), `inceptionv3_model.py` (ΠΑΡΑΡΤΗΜΑ IV), `articleType_efnet_model.py` (ΠΑΡΑΡΤΗΜΑ V), `training.py` (ΠΑΡΑΡΤΗΜΑ VI) και `evaluation.py` (ΠΑΡΑΡΤΗΜΑ VII). Τα αρχεία αυτά εκτελέστηκαν σε ένα Python Notebook (`AI_inf_main.ipynb`), το οποίο έτρεχε σε online πλατφόρμα με μερικές διαθέσιμες υποδομές και υπολογιστικούς πόρους, για να καταφέρει να πραγματοποιηθεί η εκπαίδευση και γενικά οποιαδήποτε διαδικασία απαιτούσε υπολογιστικό φόρτο. Μερικά σημαντικά τμήματα του Python Notebook σε συνδυασμό με αποτελέσματα από αυτό, θα παραχωρούνται καθ' όλη την έκταση του κεφαλαίου.

3.1 Περιγραφή του Συνόλου Δεδομένων (Dataset)

Για την ανάπτυξη των μοντέλων τεχνητής νοημοσύνης, χρησιμοποιήθηκε το δημόσια διαθέσιμο σύνολο δεδομένων Fashion Product Images Dataset, το οποίο διατίθεται μέσω του Kaggle. Το σύνολο αυτό περιλαμβάνει εικόνες (καταλόγου) προϊόντων μόδας συνοδευόμενες από μεταδεδομένα που περιγράφουν διάφορα χαρακτηριστικά κάθε προϊόντος, εξυπηρετώντας ιδανικά τον σκοπό της πτυχιακής εργασίας, σε σχέση με όλα τα υπόλοιπα ελεύθερα διαθέσιμα σύνολα δεδομένων. [8]. Η διαδικασία προεπεξεργασίας υλοποιήθηκε στο Python αρχείο `preprocessing.py` (ΠΑΡΑΡΤΗΜΑ II).

Δομή και Περιεχόμενο:

Το αρχικό σύνολο περιλάμβανε πάνω από 44.000 εγγραφές, ωστόσο περιλάμβανε ακατάλληλες κατηγορίες-κλάσεις στις ετικέτες `masterCategory`, `subcategory` και `articleType` (όπως καλλυντικά, είδη περιποίησης, αξεσουάρ κινητών, κ.ά.). Κάθε εγγραφή συνοδεύεται από τα εξής πεδία:

- `id`: μοναδικό αναγνωριστικό της εικόνας
- `gender`: φύλο για το οποίο προορίζεται το προϊόν (π.χ., `men`, `women`, `boys`)
- `masterCategory`: βασική κατηγορία (π.χ., `apparel`, `footwear`)
- `subCategory`: υποκατηγορία (π.χ., `topwear`, `shoes`)
- `articleType`: τύπος προϊόντος (π.χ., `shirts`, `jeans`, `tshirts`)
- `baseColour`: βασικό χρώμα (π.χ., `black`, `navy blue`, `teal`, `charcoal`)
- `season`: εποχικότητα (π.χ., `summer`, `winter`, `spring`)
- `year`: έτος κυκλοφορίας
- `usage`: χρήση (π.χ., `casual`, `formal`)

- `productDisplayName`: εμφανιζόμενο όνομα του προϊόντος

Φιλτράρισμα και Κανονικοποίηση Δεδομένων:

Πραγματοποιήθηκε μια εκτενής αρχική επεξεργασία του πίνακα μετα-δεδομένων, μέσω της κλάσης `FashionDatasetLoader` του αρχείου `data_loading.py`, το οποίο παρατίθεται στο ΠΑΡΑΡΤΗΜΑ Ι:

- Αφαίρεση μη συναφών προϊόντων όπως: προϊόντα προσωπικής φροντίδας, αρώματα, οικιακά είδη, αθλητικά αντικείμενα (μπάλες), δώρα, κ.ά.
- Κανονικοποίηση κειμένου: όλα τα πεδία μετατράπηκαν σε πεζά και καθαρίστηκαν από περιττά κενά.
- Ομαδοποίηση εποχών: οι κατηγορίες `spring` και `summer` συγχωνεύθηκαν σε μία ενιαία κατηγορία `summer/spring` για λόγους ενίσχυσης των δεδομένων, δεδομένης της χαμηλής παρουσίας δειγμάτων της κλάσης `spring`, καθώς και της ομοιότητας των ρούχων που κατηγοριοποιούνται στις δύο αυτές εποχές. (Χρήση κατά τις δεύτερες και τρίτες εκδόσεις των μοντέλων)
- Γενίκευση βασικών χρωμάτων: πολλές εναλλακτικές περιγραφές (π.χ., `navy blue`, `steel`) ομαδοποιήθηκαν σε βασικές κατηγορίες (π.χ., `blue`, `grey`), με χρήση προκαθορισμένου λεξικού αντιστοίχισης. (Χρήση κατά τις δεύτερες και τρίτες εκδόσεις των μοντέλων)

Στατιστικά Κατανομής:

Μετά τον καθαρισμό και την κανονικοποίηση των δεδομένων, το καθαρό σύνολο περιλαμβάνει 40.394 εικόνες ρούχων και αξεσουάρ και οι προς χρήση κατηγορίες έχουν ως εξής:

- `masterCategory`: `apparel`, `accessories`, `footwear`, `sporting goods`
- `subCategory` (24 μοναδικές κατηγορίες): `topwear`, `shoes`, `bags`, `watches`, κ.ά.
- `articleType` (88 μοναδικές κατηγορίες): `shirts`, `jeans`, `tshirts`, `track pants` κ.ά.
- `baseColour` (15 γενικευμένες κατηγορίες): `blue`, `black`, `grey`, `white`, `red`, `pink` κ.ά.
- `season` (4 κατηγορίες): `summer/spring`, `fall`, `winter`, `nan`
- `gender`: `men`, `women`, `boys`, `girls`, `unisex`
- `usage` : `casual`, `formal`, `ethnic`, `sports`, `party`, `travel`, κ.ά.

Φόρτωση Εικόνων

Η φόρτωση εικόνων γίνεται μέσω της `load_image(image_id)`, η οποία ανοίγει κάθε αρχείο `.jpg` από τον κατάλληλο φάκελο και το μετατρέπει σε RGB μορφή. Η διαχείριση σφαλμάτων είναι ενσωματωμένη ώστε να παραλείπονται οι εικόνες που λείπουν ή είναι αλλοιωμένες.

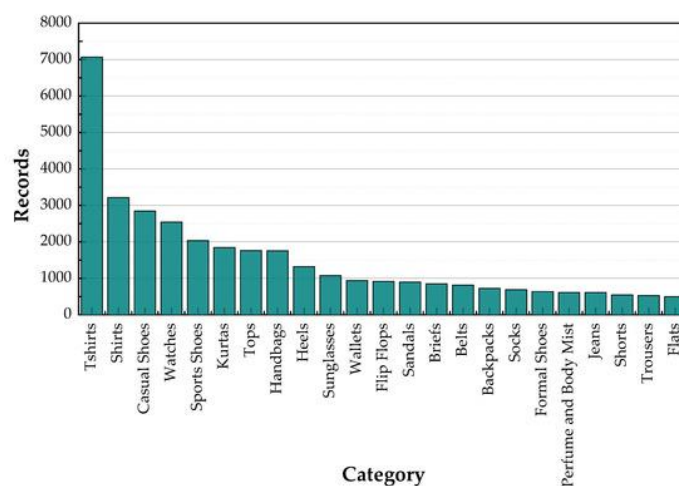
3.2 Προεπεξεργασία Δεδομένων

Η προεπεξεργασία των δεδομένων αποτελεί βασικό στάδιο σε κάθε έργο μηχανικής μάθησης, καθώς διασφαλίζει τη σωστή μορφή των εισόδων, τη συνεκτικότητα των ετικετών, καθώς και τη δημιουργία ισορροπημένων υποσυνόλων δεδομένων. Η διαδικασία αυτή είναι ιδιαίτερα σημαντική όταν χρησιμοποιούνται προ-εκπαιδευμένα μοντέλα μεταφοράς μάθησης, τα οποία απαιτούν συγκεκριμένες διαστάσεις και μορφές εικόνων. Η διαδικασία προεπεξεργασίας πραγματοποιήθηκε μέσω του Python αρχείου preprocessing.py (ΠΑΡΑΡΤΗΜΑ II), καθώς και μέσω μερικών κελιών κώδικα από το γενικότερο Python Notebook, τα οποία θα συμπεριληφθούν παρακάτω.

Κλάση DataPreprocessor

Δημιουργήθηκαν διαφορετικά τελικά σύνολα δεδομένων (datasets) για τα EfficientNet και το InceptionV3, λόγω των σημαντικών διαφορών μεταξύ των αρχιτεκτονικών τους. Η κλάση DataPreprocessor ορίστηκε για να συγκεντρώνει όλες τις απαραίτητες μεθόδους επεξεργασίας εικόνων και δημιουργίας κατάλληλων σύνολα δεδομένων (datasets), και για τις δύο αυτές αρχιτεκτονικές μοντέλων. Η κλάση περιλαμβάνει:

- Μετατροπή και κανονικοποίηση εικόνων: Κάθε εικόνα επαναδειγματοληπτείται στο κατάλληλο μέγεθος για το αντίστοιχο μοντέλο (300x300 για το EfficientNetB3 και 299x299 για InceptionV3) και μετατρέπεται σε πίνακα NumPy με τη χρήση της συνάρτησης `img_to_array()` της TensorFlow.
- Ισορροπημένη δειγματοληψία ανά κλάση για ένα συγκεκριμένο χαρακτηριστικό (articleType): Η μέθοδος `create_balanced_dataset()` εφαρμόζει δειγματοληψία μέχρι ένα μέγιστο πλήθος δειγμάτων ανά κατηγορία, διατηρώντας ισορροπία στις κατηγορίες του επιλεγμένου γνωρίσματος. Χρησιμοποιείται μόνο στο γνώρισμα `articleType` εκτελώντας έτσι εξισορρόπηση μέσω της υπο-δειγματοληψίας (undersampling) με 500 δείγματα ανά κλάση του γνωρίσματος αυτού, για να διασφαλιστεί δίκαιη σύγκριση και ύστερα εκπαίδευση μεταξύ όλων των κλάσεων του χαρακτηριστικού, το οποίο είναι και το πιο σημαντικό σε σχέση με όλα τα υπόλοιπα. Μετά την διαδικασία αυτή έχουμε ένα τελικό σύνολο 17724 εικόνων και για αυτό τον λόγο δεν εφαρμόζεται αυτή η εξισορρόπηση και στα υπόλοιπα χαρακτηριστικά, διότι μετά το σύνολο δεδομένων θα γινόταν πολύ μικρό. Ύστερα έγινε διαχωρισμός του συνολικού dataset σε σετ εκπαίδευσης (training set), σετ επικύρωσης (validation set) και σετ δοκιμής (test set), με την αναλογία 60%, 20% και 20% αντίστοιχα, από το συνολικό σύνολο δεδομένων. ([8])



Σχήμα 5. Διαμοιρασμός δειγμάτων στις κλάσεις του `articleType`, πριν την εξισορρόπηση [8]

```

from sklearn.model_selection import train_test_split

balanced_metadata = preprocessor.create_balanced_dataset(metadata, 'articleType', 500)

print("Full count: ", len(balanced_metadata))

# Split data
train_ids, test_ids = train_test_split(balanced_metadata['id'], test_size=0.2)
train_ids, val_ids = train_test_split(train_ids, test_size=0.25) # 60-20-20 split

Full count: 17724

```

Εικόνα 1. Κελί κώδικα εξισορρόπησης και διαχωρισμού του συνόλου δεδομένων, από το Python Notebook

- Κατασκευή dataset: Οι μέθοδοι `prepare_ef_dataset()` και `prepare_iv3_dataset()` δημιουργούν τα σύνολα εικόνων και ετικετών για κάθε αρχιτεκτονική (EfficientNet και InceptionV3), φορτώνοντας εικόνες μέσω ενός αντικειμένου loader και εξάγοντας τις αντίστοιχες ετικέτες από τα μεταδεδομένα (metadata).

Αυτή η προσέγγιση διασφαλίζει ότι οι εικόνες είναι συμβατές με τα αντίστοιχα προεκπαιδευμένα δίκτυα και ότι οι ετικέτες συλλέγονται με συνέπεια. Επίσης κάθε εικόνα δέχεται ειδική προεπεξεργασία από την αντίστοιχη συνάρτηση προεπεξεργασίας του κάθε μοντέλου, η οποία εκτελείται αργότερα, μέσα στην αρχιτεκτονική του κάθε μοντέλου, καθώς του εισάγονται εικόνες. Περισσότερα περί του θέματος εξηγούνται στο υποκεφάλαιο 3.4.

Έλεγχος ελλιπών ετικετών

Πριν την εκπαίδευση, έγινε έλεγχος για να διαπιστωθεί εάν κάποιες ετικέτες εμφανίζονται στο validation ή test set, αλλά απουσιάζουν από το training set. Αυτό είναι κρίσιμο, καθώς οι κατηγορίες που δεν έχουν εκπαιδευτεί ποτέ από το μοντέλο δεν μπορούν να προβλεφθούν σωστά.

Ο παρακάτω έλεγχος εφαρμόστηκε (παράδειγμα από την διαδικασία προεπεξεργασίας του EfficientNet):

```

for attr in efnet_training.y_train:
    train_labels = set(efnet_training.y_train[attr])
    val_labels = set(efnet_validation.y_val[attr])
    test_labels = set(efnet_testing.y_test[attr]) if efnet_testing.y_test else set()
    all_seen = train_labels | val_labels | test_labels

    missing = (val_labels | test_labels) - train_labels
    if missing:
        print(f"[{attr}] Missing in training: {missing}")

```

Εικόνα 2. Κελί κώδικα ελέγχου ελλιπών ετικετών από το Python Notebook

Η παραπάνω διαδικασία εντόπισε και εμφάνισε τις ετικέτες (labels) που λείπουν από τα δεδομένα εκπαίδευσης αλλά υπάρχουν στις υπόλοιπες διαιρέσεις, διευκολύνοντας έτσι την αποφυγή σφαλμάτων κατά την αξιολόγηση.

Φιλτράρισμα Συνόλων Επικύρωσης και Δοκιμής

Για να διασφαλιστεί η συνέπεια μεταξύ training και validation/test sets, εφαρμόστηκε φιλτράρισμα με βάση τις κατηγορίες που εμφανίζονται στο training set. Κάθε δείγμα

κρατήθηκε μόνο εάν όλες οι ετικέτες του (attributes) υπήρχαν ήδη στο training set. Η σχετική συνάρτηση filter_dataset() φαίνεται παρακάτω:

```
import numpy as np

def filter_dataset(X, y, y_train):
    """
    Filters dataset (X, y) keeping only samples where for ALL attributes
    the label exists in y_train[attr].
    Assumes y and y_train are dicts with the same keys.
    """
    num_samples = len(X)
    keep_mask = np.ones(num_samples, dtype=bool)

    for attr in y_train:
        train_labels = set(y_train[attr])
        attr_labels = np.array(y[attr])
        attr_mask = np.array([lbl in train_labels for lbl in attr_labels])
        keep_mask = keep_mask & attr_mask

    # Filter X and y by the mask
    X_filtered = np.array(X)[keep_mask]
    y_filtered = {attr: np.array(y[attr])[keep_mask] for attr in y}

    return X_filtered, y_filtered
```

Εικόνα 3. Κελί συνάρτησης φιλτραρίσματος από το Python Notebook

Η συνάρτηση εφαρμόστηκε τόσο στο validation όσο και στο test set, ώστε να μην περιλαμβάνουν ετικέτες μη εκπαιδευμένες.

Επιπλέον Φιλτράρισμα στο Ισορροπημένο Dataset

Τέλος, για συγκεκριμένα γνωρίσματα/ετικέτες για τα οποία παρατηρήθηκε ότι υπάρχουν απύσες κλάσεις από το y_train, όπως το articleType ή το baseColour, εφαρμόστηκε επιπλέον φιλτράρισμα στο ισορροπημένο metadata, ώστε να κρατηθούν μόνο οι κατηγορίες που υπήρχαν στο y_train. Αυτό ήταν κρίσιμο για την εκπαίδευση εξειδικευμένων μοντέλων ανά γνώρισμα:

```
# Get the set of labels present in y_train
y_train_labels_art = set(efnet_training.y_train['articleType'])>

# Filter balanced_metadata to keep only rows whose articleType is in y_train_labels_art
filtered_balanced_metadata_one = balanced_metadata[balanced_metadata['articleType'].isin(y_train_labels_art)].copy()
print(filtered_balanced_metadata_one['articleType'].unique(), len(filtered_balanced_metadata_one['articleType'].unique()))
```

Εικόνα 4. Κελί επιπλέον φιλτραρίσματος από το Python Notebook

Με αυτή τη στρατηγική εξασφαλίστηκε η συνοχή και η καθαρότητα των ετικετών, κάτι που έχει άμεση επίδραση στην ακρίβεια και αξιοπιστία των τελικών προβλέψεων των μοντέλων.

Το τελικό πλήθος εικόνων και για τα 3 υπο-σετ (training, validation, test sub-sets) μαζί είναι 17722 εικόνες, τα οποία χωρίστηκαν σε καθένα από τα 2 τελικά datasets ως εξής (7 ετικέτες το καθένα):

```
EfficientNet Training size: 10633 7  
EfficientNet Validation size: 3545 7  
EfficientNet Testing size: 3544 7  
InceptionV3 Training size: 10632 7  
InceptionV3 Validation size: 3545 7  
InceptionV3 Testing size: 3545 7
```

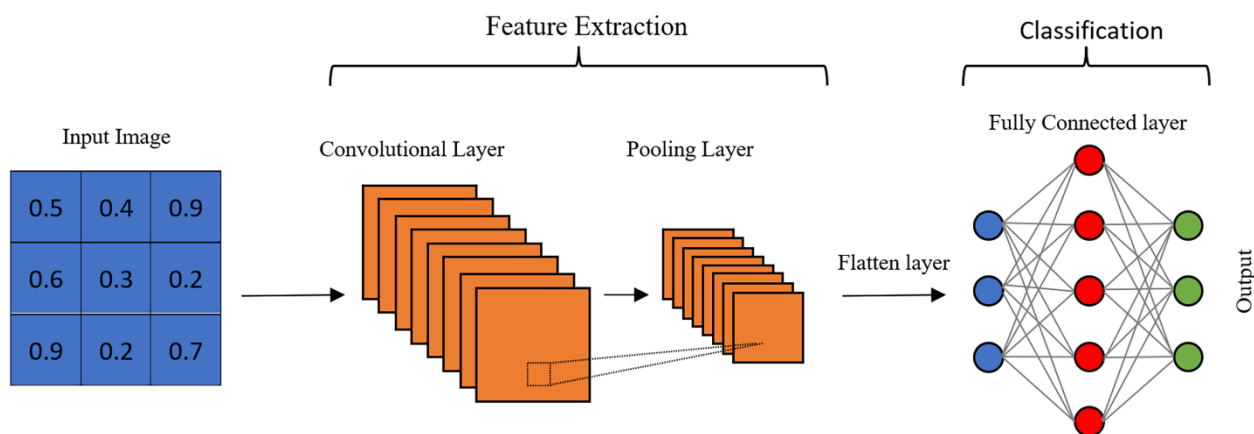
Εικόνα 5. Τελικό πλήθος δεδομένων από το Python Notebook

3.3 Αρχιτεκτονικές και Βάσεις Μοντέλων Ταξινόμησης

Κατά την ανάπτυξη του συστήματος ταξινόμησης ενδυμάτων, επιλέχθηκαν τρεις κύριες αρχιτεκτονικές βάσεις συνελκτικών νευρωνικών δικτύων προς υλοποίηση και δοκιμή, για τα μοντέλα βαθιάς μάθησης: Τα EfficientNetB3 και InceptionV3 για το πολυ-ετικετικό μοντέλο και τα EfficientNetB0/B1/B3 για το μονο-ετικετικό μοντέλο. Τα μοντέλα αυτά, όπως και τα περισσότερα γνωστά μοντέλα κατηγοριοποίησης εικόνων, είναι προ-εκπαιδευμένα στο σύνολο δεδομένων ImageNet, το οποίο περιέχει έναν πολύ μεγάλο όγκο εικόνων ποικίλων ειδών αντικειμένων, συμπεριλαμβανομένων και κάποιον ρούχων. Το χαρακτηριστικό τους αυτό τα καθιστά ήδη ικανά για βασική και στοιχειώδη κατηγοριοποίηση σχημάτων, και επομένως ιδανικά για να εκπαιδευτούν στη συνέχεια πάνω στο σύνολο δεδομένων ρουχισμού της εργασίας, με σκοπό μέγιστη απόδοση στην κατηγοριοποίηση εικόνων ρουχισμού. Η επιλογή αυτών των συγκεκριμένων βάσεων βέβαια δεν ήταν τυχαία, αλλά βασίστηκε τόσο σε τεχνικά χαρακτηριστικά όσο και σε σχετική βιβλιογραφία που τεκμηριώνει την απόδοσή τους σε προβλήματα οπτικής ταξινόμησης. [8], [12], [15], [16], [17], [20], [27], [28], [29], [30], [31]

Αρχικά, η επιλογή όσο αναφορά την χρήση αρχιτεκτονικών συνελκτικών νευρωνικών δικτύων (Convolutional Neural Networks) ως την κατηγορία μοντέλων βαθιάς μάθησης που θα αποτελούσαν την αρχική βάση για την εργασία, έγινε για τους εξής λόγους:

- Τα CNNs (Convolutional Neural Networks) ακμάζουν πάνω σε κατηγοριοποίηση δεδομένων εικόνων, λόγω της αρχιτεκτονικής τους και των συνελκτικών επιπέδων-στρώσεων που περιέχουν μέσα στα επίπεδά τους, όπως εξηγήθηκε και στο κεφάλαιο 2.
- Μία άλλη αρχιτεκτονική που παρατηρείται να αποδίδει καλά πάνω σε δεδομένα εικόνων τον τελευταίο καιρό, είναι αυτή των transformers, όπως η VGG. Παρ' όλα αυτά για να ξεπεράσουν σε απόδοση τα CNNs απαιτείται πολύ μεγαλύτερος όγκος δεδομένων για την διαδικασία εκπαίδευσης, καθώς και παραπάνω υπολογιστικοί πόροι. Τα παραπάνω όχι μόνο δεν είναι διαθέσιμα στην περίπτωση της παρούσας εργασίας, αλλά λόγω των λιγότερων υπολογιστικών απαιτήσεων τους, τα CNNs αποτελούν ιδανική επιλογή όσο αναφορά το σύνολο του συστήματος που υλοποιείται, λαμβάνοντας υπόψιν και την βελτιστοποίηση της εμπειρίας και της χρονικής αναμονής, του οπτικά ανάπηρου χρήστη. Τέλος, τα CNNs αποδίδουν εμφανώς καλύτερα σε σύγκριση με αρχιτεκτονικές transformers, όταν υπάρχουν περιορισμένες υπολογιστικές υποδομές και με μεσαίου-μικρού όγκου datasets. ([27], [28])



Σχήμα 6. Σχηματική Αναπαράσταση Αρχιτεκτονικής ενός CNN (Convolutional Neural Network)

Επιλογή Multi-Task Ταξινόμησης σε ένα μοντέλο για όλα τα χαρακτηριστικά, αντί για ένα Single-Task Μοντέλο για κάθε χαρακτηριστικό

Όπως αναφέρεται και στο υπο-κεφάλαιο 2.4:

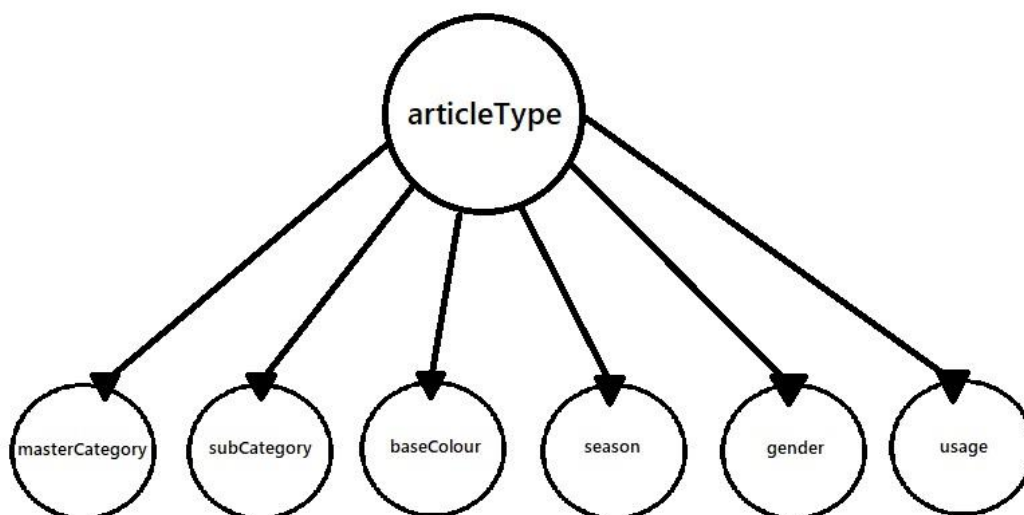
«Η πολυ-ετικετική μάθηση εκμεταλλεύεται την κοινή πληροφορία μεταξύ των διαφορετικών εργασιών, επιτρέποντας στο μοντέλο να μάθει πιο ισχυρά και γενικεύσιμα χαρακτηριστικά. Με τον τρόπο αυτό, κάθε εργασία λειτουργεί ως μια μορφή "κανονικοποίησης" για τις άλλες, μειώνοντας τον κίνδυνο υπερεκπαίδευσης (overfitting). Τα μοντέλα με πολλαπλές εξόδους, δηλαδή που παράγουν περισσότερες από μία προβλέψεις σε κάθε πέρασμα, χρησιμοποιούνται συχνά για να μάθουν ταυτόχρονα διαφορετικές ετικέτες, με ξεχωριστές "κεφαλές" εξόδου (output heads) που συνδέονται με μια κοινή βάση (shared backbone). Αυτό καθιστά τη διαδικασία πιο αποδοτική από το να εκπαιδεύονται πολλαπλά ανεξάρτητα μοντέλα για κάθε ετικέτα ξεχωριστά.»

Στην παρούσα περίπτωση, είναι αξιοσημείωτο το γεγονός ότι η πλειονότητα των χαρακτηριστικών που θέλουμε να προβλέπει το μοντέλο παρουσιάζουν ισχυρή συσχέτιση με το χαρακτηριστικό *articleType*. Για παράδειγμα, αν το μοντέλο εντοπίσει ότι το ρούχο είναι ένα απλό κοντομάνικο μπλουζάκι, μπορεί να συναγάγει σχετικά εύκολα ότι προορίζεται για καθημερινή χρήση (*usage*) ή ότι ανήκει πιθανότατα σε ανοιξιάτικη ή καλοκαιρινή περίοδο (*season*). Αντίστοιχα, η αναγνώριση μιας φούστας βοηθά έμμεσα στην πρόβλεψη του φύλου (*gender*), της κύριας κατηγορίας (*masterCategory*) και της υποκατηγορίας (*subCategory*).

Θεωρητικά, θα μπορούσε να αναπτυχθεί ξεχωριστό μοντέλο για κάθε χαρακτηριστικό, όπως το φύλο, η εποχή ή η χρήση, όμως αυτό δεν υλοποιήθηκε κυρίως λόγω του μεγάλου υπολογιστικού φόρτου και του χρόνου εκπαίδευσης που απαιτείται για έξι έως επτά ανεξάρτητα μοντέλα, κάτι που δεν ήταν εφικτό με τις διαθέσιμες υποδομές. Επιπλέον, από πειραματικές δοκιμές και εμπειρική παρατήρηση έχει διαπιστωθεί ότι η χρήση ξεχωριστών μοντέλων δεν προσφέρει σημαντική βελτίωση στην ακρίβεια για τα περισσότερα χαρακτηριστικά (στα περισσότερα χαρακτηριστικά η πολύ-ετικετική προσέγγιση είναι μάλιστα αυτή που προσφέρει μεγαλύτερες ακρίβειες, λόγω όσων προαναφέρθηκαν), καθιστώντας την πολύ-ετικετική (multi-task) προσέγγιση πιο αποδοτική και αποτελεσματική.

Έτσι, η πολύ-ετικετική αρχιτεκτονική όχι μόνο δεν επιβαρύνει την ακρίβεια των επιμέρους χαρακτηριστικών (εκτός του ίδιου του *articleType*), αλλά ενισχύει έμμεσα την απόδοσή τους, αξιοποιώντας τις ενδο-συσχετίσεις μεταξύ τους. Η από κοινού μάθηση των χαρακτηριστικών επιτρέπει στο μοντέλο να αναπτύξει μια πιο σφαιρική και εστιασμένη αναπαράσταση των ενδυμάτων, με το *articleType* να λειτουργεί ουσιαστικά ως κεντρικός

πυρήνας πληροφορίας, από το οποίο βοηθιούνται όλα τα υπόλοιπα χαρακτηριστικά. Παρατηρήθηκε κυρίως από τις δύο πρώτες εκδόσεις, ότι η εκμάθηση της περισσότερο ειδικής πληροφορίας πρώτα (articleType) βοηθά πιο πολύ στην εκμάθηση της πιο γενικής πληροφορίας μετά (υπόλοιπα χαρακτηριστικά πχ. season, mainCategory), απ' ότι η εκμάθηση όλων ταυτόχρονα κατευθείαν ή της γενικής πληροφορίας πρώτα (όλα τα υπόλοιπα χαρακτηριστικά και της πιο ειδικής πληροφορίας (articleType) ύστερα. Το πώς ακριβώς έγινε η διαχείριση της εκπαίδευσης του multi-task μοντέλου, στηρίζεται στην παραπάνω πληροφορία και αναλύεται περαιτέρω στο υπο-κεφάλαιο 3.5. [15], [16], [17], [20]



Σχήμα 7. Σχήμα Περιγραφής Σχέσης μεταξύ Χαρακτηριστικών

Επιλογή EfficientNetB3 ως Κύρια Βάση για Multi-Task Ταξινόμηση

Η αρχιτεκτονική EfficientNet προτάθηκε από την Google, με βασική καινοτομία την ομοιόμορφη κλιμάκωση των διαστάσεων πλάτους, βάθους και ανάλυσης εισόδου. Η παραλλαγή EfficientNetB3 προσφέρει έναν εξαιρετικό συμβιβασμό μεταξύ ακρίβειας και υπολογιστικού κόστους και διαθέσιμων υπολογιστικών υποδομών, γεγονός που την καθιστά ιδανική για multi-task ταξινόμηση, όπου ένα κοινό μοντέλο καλείται να προβλέψει πολλαπλά χαρακτηριστικά ταυτόχρονα (articleType, gender, season, κ.ά.). ([29], [30])

Επιλογή InceptionV3 ως Εναλλακτική Βάση για Multi-Task Ταξινόμηση

Η αρχιτεκτονική InceptionV3 παραμένει έως σήμερα μία από τις πιο αποδοτικές λύσεις για ταξινόμηση εικόνων, λόγω της έξυπνης διαχείρισης του βάθους μέσω διακλαδωμένων φίλτρων (inception modules) και για παρόμοιους επίσης λόγους με το EfficientNetB3 (υπολογιστικός φόρτος, κλπ.). Επιλέχθηκε ως εναλλακτική στο EfficientNetB3 για να εξεταστεί η συμπεριφορά διαφορετικών αρχιτεκτονικών στο multi-task learning, καθώς και για συγκριτικούς σκοπούς. ([31])

Η επιλογή του των EfficientNetB3 και InceptionV3 βασίστηκε σε συγκεκριμένες βιβλιογραφικές αναφορές, καθώς και ποικίλα άρθρα όπου επιδεικνύεται η ανωτερότητα απόδοσης των βάσεων των μοντέλων αυτών, κατά την κατηγοριοποίηση εικόνων. Επίσης, η επιλογή αυτών των δύο συγκεκριμένων αρχιτεκτονικών έγινε σύμφωνα και με τη μελέτη [8], όπου φαίνεται ότι οι πρόγονοι των μοντέλων αυτών EfficientNetB3 και GoogleLeNet (InceptionV3), έχουν την υψηλότερη απόδοση πάνω στο Fashion Product Images Dataset της Kaggle.

Δοκιμή Single-Task Ταξινόμησης, για το χαρακτηριστικό articleType (ακριβής κατηγορία)

Όπως αναφέρθηκε και πριν, η από κοινού μάθηση των χαρακτηριστικών με μια πολύ-ετικετική αρχιτεκτονική βοηθάει το μοντέλο στην αναγνώριση όλων των υπόλοιπων χαρακτηριστικών, με το articleType να λειτουργεί ουσιαστικά ως κεντρικός πυρήνας πληροφορίας. Ωστόσο, το ίδιο το articleType φαίνεται να αποτελεί μια ιδιαίτερη περίπτωση, διότι μπορεί να βοηθάει στην εκμάθηση τον υπόλοιπων χαρακτηριστικών, αλλά η εκμάθηση του ίδιου δεν βοηθιέται ιδιαίτερα στο multi-task μοντέλο. (Δεδομένου ότι τα υπόλοιπα χαρακτηριστικά συχνά εξαρτώνται έμμεσα από αυτό, τέθηκε αναγκαίο να εξεταστεί αν η δημιουργία ενός ανεξάρτητου, αποκλειστικά αφιερωμένου μοντέλου για την πρόβλεψή του μπορεί να προσφέρει περαιτέρω βελτιώσεις στην ακρίβεια και γενίκευση του χαρακτηριστικού articleType (ακριβής κατηγορία), όντας και το πιο σημαντικό χαρακτηριστικό-ετικέτα από όλα.

Επιλογή και δοκιμή βάσεων EfficientNetB0/B1/B3 για Single-Task Ταξινόμηση ακριβούς κατηγορίας (articleType)

Η πρόβλεψη μόνο του articleType απομονώθηκε σε single-task μοντέλα, με χρήση όχι μόνο του EfficientNetB3 ,αλλά και των βάσεων των EfficientNetB0 και EfficientNetB1. Ο λόγος για αυτή την επιλογή είναι διπλός:

- Περίπτωση αποφυγής Υπερ-προσαρμογής/Υπερ-εκπαίδευσης (Overfitting): Σε datasets όπου το πλήθος των παραδειγμάτων είναι σχετικά περιορισμένο, η χρήση μικρότερων μοντέλων αποδίδει συχνά καλύτερα από μεγαλύτερα μοντέλα, καθώς μειώνεται ο κίνδυνος υπερ-προσαρμογής.
- Υπολογιστική Ελαφρότητα: Τα EfficientNetB0 και B1 είναι εξαιρετικά ελαφριά μοντέλα που απαιτούν λιγότερους πόρους, γεγονός που τα καθιστά ιδανικά για μεσαίου μεγέθους σύνολα δεδομένων.

Το πώς ακριβώς αναπτύχθηκαν και εκπαιδεύτηκαν τα μοντέλα, καθώς και το ποιες εκδόσεις δημιουργήθηκαν για το καθένα, αναλύεται πιο συγκεκριμένα στο υπο-κεφάλαιο 3.5

```
Epoch 1/50
Batch 290: X shape (16, 300, 300, 3), y keys: ['masterCategory', 'subCategory', 'articleType', 'baseColour', 'gender', 'season', 'usage'], y[0]
Batch 236: X shape (16, 300, 300, 3), y keys: ['masterCategory', 'subCategory', 'articleType', 'baseColour', 'gender', 'season', 'usage'], y[0]
1/665 ----- 6:48:57 37s/step - articleType accuracy: 0.0000e+00 - articleType loss: 4.6049 - baseColour accuracy: 0.1250 - ba
2/665 ----- 1:06 100ms/step - articleType accuracy: 0.0000e+00 - articleType loss: 4.5950 - baseColour accuracy: 0.0938 - ba
3/665 ----- 1:05 98ms/step - articleType accuracy: 0.0069 - articleType loss: 4.5825 - baseColour accuracy: 0.0764 - baseColo
4/665 ----- 1:00 91ms/step - articleType accuracy: 0.0091 - articleType loss: 4.5719 - baseColour accuracy: 0.0651 - baseColo
5/665 ----- 56s 86ms/step - articleType accuracy: 0.0098 - articleType loss: 4.5584 - baseColour accuracy: 0.0596 - baseColo
6/665 ----- 54s 83ms/step - articleType accuracy: 0.0099 - articleType loss: 4.5542 - baseColour accuracy: 0.0566 - baseColo
7/665 ----- 52s 80ms/step - articleType accuracy: 0.0098 - articleType loss: 4.5496 - baseColour accuracy: 0.0536 - baseColo
8/665 ----- 51s 78ms/step - articleType accuracy: 0.0095 - articleType loss: 4.5441 - baseColour accuracy: 0.0508 - baseColo
9/665 ----- 50s 76ms/step - articleType accuracy: 0.0092 - articleType loss: 4.5386 - baseColour accuracy: 0.0483 - baseColo
```

Εικόνα 6. Παράδειγμα Εκπαίδευσης 1^{ης} Εποχής, Δεύτερης έκδοσης από το Python Notebook

3.4 Αγωγός Εκπαίδευσης (Training Pipeline)

Η εκπαίδευση των μοντέλων βαθιάς μάθησης για την ταξινόμηση ενδυμάτων αποτελεί κρίσιμο στάδιο της ανάπτυξης του συστήματος, καθώς η απόδοση του μοντέλου εξαρτάται άμεσα από την ποιότητα και αποτελεσματικότητα της διαδικασίας εκπαίδευσης. Στο πλαίσιο της παρούσας εργασίας, υλοποιήθηκε ένας πλήρως παραμετροποιήσιμος αγωγός (pipeline) εκπαίδευσης το οποίο υποστηρίζει τόσο πολύ-ετικετικά (multi-task) όσο και μονο-ετικετικά (single-task) μοντέλα, καθώς και δυνατότητα συνέχισης εκπαίδευσης, αποθήκευσης κατάστασης, και ευέλικτης διαχείρισης δεδομένων. Ο αγωγός εκπαίδευσης (training pipeline) υλοποιήθηκε στο Python αρχείο `training.py` (ΠΑΡΑΡΤΗΜΑ VI).

Παραγωγή Δεδομένων για Multi-task Εκπαίδευση

Για την αποδοτική τροφοδοσία του μοντέλου με δεδομένα, υλοποιήθηκε η κλάση `MultiOutputDataGenerator` που κληρονομεί από το `tf.keras.utils.Sequence`. Η κλάση αυτή είναι υπεύθυνη για τη δημιουργία `batches` δεδομένων κατάλληλων για μοντέλα με πολλαπλές εξόδους (multi-output / multi-task), όπως είναι οι διαφορετικές κατηγορίες χαρακτηριστικών ενδυμάτων (φύλο, χρώμα, εποχή, κλπ.). Διαχειρίζεται την ανάγνωση και μερική επεξεργασία των εικόνων, καθώς και την αντιστοίχιση των αντίστοιχων κωδικοποιημένων ετικετών σε κάθε batch, ενώ υποστηρίζει και την τυχαιοποίηση (shuffle) των δεδομένων σε κάθε εποχή για αποφυγή υπερεκπαίδευσης (overfitting).

Διαχείριση και Εκτέλεση της Εκπαίδευσης

Η κεντρική μονάδα διαχείρισης της εκπαίδευσης είναι η κλάση `ClothingClassifierTrainer`. Κατά την αρχικοποίησή της, δέχεται το μοντέλο που θα εκπαιδευτεί, το όνομα του μοντέλου, και τα μονοπάτια των φακέλων για την αποθήκευση των αποτελεσμάτων. Στη συνέχεια, ρυθμίζει τον αγωγό για μερική προεπεξεργασία δεδομένων και `Data Augmentation` (τυχαίες μετασχηματίσεις δεδομένων) για αύξηση της γενίκευσης του μοντέλου. Αναλόγως με το όνομα και είδος του μοντέλου που καταχωρείται ότι επρόκειτο να εκπαιδευτεί, εφαρμόζονται διαφορετικά `data augmentations`. Το σύνολο επικύρωσης (validation set) δεν υφίσταται `augmentation`, ώστε να προσφέρει μια πιο ρεαλιστική εκτίμηση της απόδοσης του μοντέλου σε μη γνωστά δεδομένα. Ο λόγος για την επιλογή αυτών των συγκεκριμένων `data augmentations` στο καθένα, αναλύεται παραπάνω στο υπο-κεφάλαιο 3.5. Ενώ κανονικά το `Data Augmentation` είναι μέρος του βήματος προεπεξεργασίας (3.2), εκτελείται μέσω του αγωγού εκπαίδευσης (training pipeline), κατά τη διαδικασία εκπαίδευσης, μετά την ανάγνωση κάθε εικόνας. Αυτό συμβαίνει, ώστε να διατηρηθεί η ακεραιότητα του αρχικού συνόλου δεδομένων εικόνων, καθώς εφαρμόζονται κάθε φορά διαφορετικοί μετασχηματισμοί δεδομένων (`data augmentations`), ανάλογα με το μοντέλο που πρόκειται να εκπαιδευτεί και την έκδοση του. Έτσι, η όλη διαδικασία γίνεται πιο εύκολη, καθώς δεν χρειάζεται να δημιουργούνται νέα τελικά dataset κάθε φορά και να επαναλαμβάνεται το βήμα της προεπεξεργασίας (3.2) με διαφορετικά `data augmentations`, αφού ο αγωγός εκπαίδευσης (training pipeline) έχει σχεδιαστεί να είναι πλήρως δυναμικός και παραμετροποιήσιμος.

Η διαδικασία της εκπαίδευσης περιλαμβάνει τα εξής στάδια:

- **Κωδικοποίηση Ετικετών:** Πριν από την εκπαίδευση, οι κατηγορικές ετικέτες των δεδομένων (όπως `articleType`, `baseColour`, `season`, κ.λπ.) πρέπει να μετατραπούν σε αριθμητική μορφή, ώστε να μπορούν να χρησιμοποιηθούν ως στόχοι από το νευρωνικό δίκτυο. Για τον σκοπό αυτό χρησιμοποιούνται μετατροπές τύπου `LabelEncoder` από τη βιβλιοθήκη `scikit-learn`, ένας για κάθε ξεχωριστή έξοδο του

μοντέλου. Οι κωδικοποιητές αυτοί προσαρμόζονται αποκλειστικά με βάση τις ετικέτες του εκπαιδευτικού συνόλου, εξασφαλίζοντας ότι οι αντιστοιχίσεις κατηγορίας-αριθμού είναι συνεπείς και σταθερές καθ' όλη τη διάρκεια της εκπαίδευσης και της αξιολόγησης. Οι κωδικοποιητές αποθηκεύονται, ώστε να επαναχρησιμοποιούνται και κατά την πρόβλεψη (inference) νέων δεδομένων.

- Δημιουργία Data Generators: Για την αποδοτική τροφοδοσία του μοντέλου με εικόνες και ετικέτες κατά τη διάρκεια της εκπαίδευσης, χρησιμοποιούνται data generators που υποστηρίζουν παραγωγή batches δεδομένων σε πραγματικό χρόνο, οι οποίοι δημιουργούνται με βάση την προσαρμοσμένη κλάση MultiOutputDataGenerator, που περιεγράφηκε παραπάνω

- Ορισμός Ελέγχων (Callbacks): Η διαδικασία εκπαίδευσης υποστηρίζεται από ένα σύνολο εργαλείων παρακολούθησης και ελέγχου (callbacks), τα οποία αυτοματοποιούν βασικές λειτουργίες και βοηθούν στην αποδοτική προσαρμογή του μοντέλου:

- ModelCheckpoint: Παρακολουθεί την επίδοση στο σύνολο επικύρωσης και αποθηκεύει τα καλύτερα βάρη του μοντέλου (best weights), κάθε φορά που σημειώνεται νέα βέλτιστη τιμή (η ελάχιστη) στην απώλεια επικύρωσης (validation loss). Το validation loss είναι μία μέτρηση που δείχνει πόσο καλά γενικεύει το μοντέλο σε δεδομένα που δεν έχει ξαναδεί κατά την εκπαίδευση. Είναι η τιμή της συνάρτησης απώλειας (loss function), υπολογισμένη πάνω στα δεδομένα επικύρωσης. Με αυτόν τον τρόπο διασφαλίζεται ότι διατηρείται πάντα η καλύτερη έκδοση του μοντέλου, δηλαδή των βαρών του.
- EarlyStopping: Διακόπτει πρόωρα την εκπαίδευση όταν δεν παρατηρείται βελτίωση στο validation loss για προκαθορισμένο αριθμό εποχών, σύμφωνα με την τιμή της υπομονής (patience). Η λειτουργία αυτή υπάρχει για να αποτραπεί τυχόν υπερεκπαίδευση/υπερπροσαρμογή.
- ReduceLROnPlateau: Όταν η απώλεια (loss) παραμένει στάσιμη για αρκετές εποχές, μειώνει αυτόματα τον ρυθμό εκμάθησης (learning rate), ώστε να διευκολυνθεί η σταδιακή σύγκλιση του μοντέλου σε ένα ελάχιστο.
- SaveTrainingStateCallback: Πρόκειται για ένα προσαρμοσμένο callback που υλοποιήθηκε ειδικά για την εφαρμογή αυτή, με σκοπό την αποθήκευση της πλήρους κατάστασης της εκπαίδευσης μετά από κάθε εποχή, επιτρέποντας την ασφαλή διακοπή και συνέχισή της. Η κλάση αυτού περιγράφεται παρακάτω.

- Έναρξη Εκπαίδευσης: Η εκπαίδευση του μοντέλου εκκινείται με την κλήση της μεθόδου fit(), στην οποία δίνονται ως είσοδος οι γεννήτριες (generators) του εκπαιδευτικού και επικυρωτικού συνόλου, καθώς και τα προκαθορισμένα callbacks. Κατά τη διάρκεια της εκπαίδευσης, το μοντέλο ενημερώνει σταδιακά τα βάρη του με βάση το σφάλμα (loss) σε κάθε έξοδο, ενώ οι μετρικές απόδοσης και οι τιμές του loss καταγράφονται ανά εποχή.

- Συνέχιση Εκπαίδευσης: Σε περιπτώσεις που η εκπαίδευση διακοπεί (π.χ. λόγω σφάλματος συστήματος, επανεκκίνησης ή αλλαγής ρυθμίσεων), υπάρχει η δυνατότητα επανεκκίνησης από το τελευταίο αποθηκευμένο σημείο. Αυτό γίνεται με φόρτωση των αποθηκευμένων βαρών, της προόδου της εκπαίδευσης (εποχή/epoch, ιστορικό απωλειών), καθώς και των αρχικών μετατροπών ετικετών. Η δυνατότητα αυτή καθιστά τη διαδικασία πιο ανθεκτική και ευέλικτη, εξασφαλίζοντας ότι η πρόοδος δεν θα χαθεί.

- Αποθήκευση Μοντέλου/Αποτελεσμάτων (Save Model): Μετά την ολοκλήρωση της διαδικασίας εκπαίδευσης, αποθηκεύονται σε αρχεία:

- Τα τελικά και τα καλύτερα βάρη του μοντέλου (.h5).
- Το ιστορικό εκπαίδευσης, με πληροφορίες για την εκπαίδευση και validation loss, καθώς και τις επιμέρους μετρικές απόδοσης για κάθε έξοδο.
- Οι μετατροπείς ετικετών (LabelEncoders) για κάθε προβλεπόμενο χαρακτηριστικό.

Αυτά τα δεδομένα καθιστούν δυνατή την μελλοντική αξιολόγηση του μοντέλου σε νέα δείγματα, την συνέχιση της εκπαίδευσης με νέα δεδομένα ή την επαναχρησιμοποίησή του σε πρακτικές εφαρμογές.

- Φόρτωση Μοντέλου (Load Model): Για τη χρήση ενός εκπαιδευμένου μοντέλου και των βαρών του (είτε τα τελικά, είτε τα βέλτιστα) σε νέα δεδομένα για πρόβλεψη/αξιολόγηση ή τη συνέχιση της εκπαίδευσης, υλοποιείται μέσα στον αγωγό εκπαίδευσης και η συνάρτηση φόρτωσης μοντέλου, που περιλαμβάνει:

- Την ανακατασκευή της αρχιτεκτονικής του μοντέλου από αρχείο .json.
- Την φόρτωση των βαρών (τελικών ή βέλτιστων) από αρχείο .weights.h5.
- Την επαναφόρτωση των LabelEncoders για κάθε προβλεπόμενο χαρακτηριστικό, ώστε να διατηρηθεί η σωστή αντιστοίχιση κατηγορικών τιμών

Αποθήκευση Κατάστασης Εκπαίδευσης Η κλάση SaveTrainingStateCallback αποτελεί έναν προσαρμοσμένο μηχανισμό ελέγχου που επεκτείνει τη λειτουργικότητα του Keras. Ενεργοποιείται αυτόματα στο τέλος κάθε εποχής και επιτελεί πολλαπλές κρίσιμες ενέργειες αποθήκευσης:

- Αποθηκεύει σε συγκεκριμένο αρχείο JSON τα στατιστικά/ιστορικό εκπαίδευσης (π.χ. απώλειες, ακρίβεια ανά εποχή για κάθε έξοδο).
- Διατηρεί την τρέχουσα κατάσταση εκπαίδευσης, δηλαδή σε ποια εποχή βρισκόταν το μοντέλο, καθώς και τις εσωτερικές τιμές του.
- Αποθηκεύει τους LabelEncoders, ώστε να διατηρείται η ίδια αναπαράσταση κατηγοριών στις προβλέψεις.
- Επιπλέον, καταγράφει τα βάρη του μοντέλου σε αρχείο τύπου .h5, εξασφαλίζοντας την πλήρη ανακατασκευή του σε επόμενη εκτέλεση.

3.5 Πολυ-εξόδου Μοντέλα και Διαδικασία Εκπαίδευσης

Στην παρούσα εργασία, σχεδιάστηκαν και εκπαιδεύτηκαν κάποια πολυ-εξόδου μοντέλα (multi-output models) βασισμένα στις αρχιτεκτονικές EfficientNetB3 και InceptionV3, για την ταυτόχρονη πρόβλεψη επτά διαφορετικών χαρακτηριστικών κάθε ένδυσης: articleType (δεν χρησιμοποιήθηκε), subCategory, baseColour, season, usage, masterCategory και gender. Η διαδικασία χτίσης και υλοποίησης των αρχιτεκτονικών των πολύ-ετικετικών μοντέλων υλοποιήθηκε στα Python αρχεία efficientnet_model.py (ΠΑΡΑΡΤΗΜΑ III) και inceptionv3_model.py (ΠΑΡΑΡΤΗΜΑ IV), για τα EfficientNetB3 και InceptionV3 αντίστοιχα.

Τα μοντέλα αυτά αποτελούνται από ένα κοινό κορμό (base model) και ξεχωριστές εξόδους (output branches), μία για κάθε χαρακτηριστικό-ετικέτα. Ο κοινός κορμός-βάση

είναι προεκπαιδευμένος σε μεγάλο όγκο δεδομένων (ImageNet), ενώ οι εξόδοι είναι πλήρως συνδεδεμένα στρώματα (Dense layers) με softmax ενεργοποίηση και διακριτό αριθμό εξόδων ανάλογα με τις κατηγορίες κάθε χαρακτηριστικού. Η εκπαίδευση ακολούθησε προσέγγιση δύο φάσεων σε όλες τις εκδόσεις (feature extraction + fine tuning), απλά με διαφορετικό τρόπο στην καθεμία.

Αρχιτεκτονική και Υλοποίηση

Η βασική υλοποίηση των πολυ-εξόδου μοντέλων με βάσεις τα EfficientNetB3 και InceptionV3 περιλαμβάνει:

- Βάση-κορμό του μοντέλου (base model) (παγωμένη για το πρώτο στάδιο εκπαίδευσης).
- Προεπεξεργασία με `ef_preprocess` και `iv3_preprocess` αντίστοιχα, το οποίο εφαρμόζει τις κατάλληλες μετασχηματίσεις στις εικόνες σύμφωνα με την καθορισμένη προεκπαίδευση που εφαρμόζεται στα EfficientNet και InceptionV3 αντίστοιχα, από τις αντίστοιχες βιβλιοθήκες τους. Ο λόγος που έγινε αυτή η προεπεξεργασία εδώ και όχι στο βήμα προεπεξεργασίας (3.2) είναι για να μην γίνεται η τελική μορφοροποίηση των εικόνων αμέσως πριν εισαχθούν στο μοντέλο κάθε φορά.
- Παγκοσμιοποιημένη μέση πισίνα - `GlobalAveragePooling2D` για μείωση της διάστασης των χαρακτηριστικών του βασικού μοντέλου (base model). Η συνάρτηση `GlobalAveragePooling2D` χρησιμοποιείται μετά από το προεκπαιδευμένο κορμό (backbone) των EfficientNet ή InceptionV3, και αντικαθιστά παραδοσιακά πλήρως συνδεδεμένα επίπεδα που ακολουθούσαν τον εξαγωγέα χαρακτηριστικών (feature extractor). Ο κύριος ρόλος της είναι να συμπυκνώνει τις χωρικές πληροφορίες της τελικής συνελκτικής εξόδου (convolutional output), υπολογίζοντας τον μέσο όρο σε κάθε κανάλι χαρακτηριστικών (feature map). Αυτό έχει δύο σημαντικά πλεονεκτήματα:
 - Μείωση της υπολογιστικής πολυπλοκότητας: Από ένα тенσορέα (tensor) 3 διαστάσεων (ύψος × πλάτος × κανάλια) προκύπτει ένα απλό διάνυσμα (1 × κανάλια).
 - Μείωση υπερπροσαρμογής (overfitting): Δεν εισάγονται παραπανίσιοι παράμετροι, όπως θα συνέβαινε με επιπέδωση (flatten) και πλήρως συνδεδεμένα επίπεδα (dense layers).
- Στρώση Απόρριψης - `Dropout(0.4)` layer για περαιτέρω αποφυγή υπερπροσαρμογής (overfitting). Η τεχνική Dropout εφαρμόζεται ως μέθοδος κανονικοποίησης, όπου τυχαία μηδενίζεται ένα ποσοστό των νευρώνων κατά τη διάρκεια της εκπαίδευσης. Στην περίπτωση αυτή, εφαρμόστηκε τιμή 0.4, δηλαδή το 40% των εξόδων των νευρώνων αγνοούνται τυχαία σε κάθε βήμα. Ο βασικός λόγος, όπως αναφέρθηκε, είναι η αποφυγή υπερπροσαρμογής του μοντέλου στα δεδομένα εκπαίδευσης. Ιδιαίτερα σε προβλήματα με μικρό σχετικά πλήθος δειγμάτων ή μεγάλη ποικιλομορφία (όπως η ταξινόμηση ρούχων και χαρακτηριστικών τους), το Dropout λειτουργεί ως μέτρο τακτικής αβεβαιότητας, που βοηθά το μοντέλο να μάθει πιο γενικευμένες αναπαραστάσεις. Η επιλογή της τιμής 0.4 προέκυψε έπειτα από πειραματισμό, ως ισορροπία μεταξύ διατήρησης πληροφορίας και επαρκούς κανονικοποίησης.
- Πολλαπλές εξοδοί. Δεδομένου ότι το μοντέλο εκπαιδεύεται για πολλαπλές και ταυτόχρονες ταξινομήσεις (όπως `articleType`, `baseColour`, `gender`, `season`, κ.λπ.), δημιουργούνται διαφορετικά `output branches` στην έξοδο του μοντέλου. Κάθε

branch περιλαμβάνει ένα πλήρως συνδεδεμένο επίπεδο/πυκνό (Dense layer) με τόσες εξόδους όσες και οι κλάσεις της αντίστοιχης κατηγορίας:

```
Dense(num_classes,activation='softmax', name=attr, kernel_regularizer=l2(0.01))
```

Τα βασικά χαρακτηριστικά του κάθε output branch είναι:

- Softmax συνάρτηση ενεργοποίησης (activation function), για να προκύπτουν πιθανότητες ταξινόμησης. Ο κύριος λόγος για την επιλογή της συγκεκριμένης συνάρτησης είναι ότι το εκάστοτε πρόβλημα ταξινόμησης που αντιμετωπίζουμε (όπως articleType, baseColour, gender, κ.λπ.) είναι πολυταξικό (multi-class classification). Δηλαδή, κάθε δείγμα ανήκει σε μία και μόνο μία κατηγορία από ένα σύνολο διακριτών επιλογών. Η softmax είναι ιδανική για τέτοια προβλήματα, διότι: 1. Μετατρέπει το διάνυσμα των εξόδων σε πιθανότητες (τιμές μεταξύ 0 και 1), οι οποίες αθροίζουν στο 1. 2. Το μοντέλο έτσι «μαθαίνει» να εκφράζει την εμπιστοσύνη του ως προς την πρόβλεψή του για κάθε κατηγορία. 3. Η κλάση με τη μεγαλύτερη πιθανότητα επιλέγεται ως τελική πρόβλεψη. Μαθηματικά, για ένα διάνυσμα εξόδων $z=[z_1, z_2, \dots, z_n]$, η softmax υπολογίζει:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} .$$

Εικόνα 7. Τύπος Softmax Συνάρτησης Ενεργοποίησης (Activation Function) [32]

- Όνομα (name=attr), για να διαχειρίζεται το μοντέλο ξεχωριστά τις απώλειες για κάθε task.
- L2 regularization (0.01): Παρόν μόνο στις Τρίτες και Τελικές Εκδόσεις των μοντέλων): Περαιτέρω ανάλυση περί αυτού υπάρχει στο υποκεφάλαιο «Τρίτες και Τελευταίες εκδόσεις (3.5.3).

Για την μεταγλώττιση (compile) του μοντέλου:

- Optimizer SGD (με momentum 0.9). Ο βελτιστοποιητής Stochastic Gradient Descent (SGD) είναι ένας από τους πιο κλασικούς και σταθερούς αλγόριθμους εκπαίδευσης, και σε συνδυασμό με την ορμή (momentum) αποκτά σημαντικά πλεονεκτήματα:
 - Το momentum αποθηκεύει μέρος της προηγούμενης ταχύτητας της κλίσης-παράγωγου σφάλματος (gradient), βοηθώντας την εκπαίδευση να διατηρεί σταθερή πορεία προς ένα ελάχιστο, αποφεύγοντας ταλαντώσεις.
 - Η τιμή 0.9 είναι μία ευρέως χρησιμοποιούμενη και αποδοτική τιμή για τα περισσότερα μοντέλα, δίνοντας αρκετή "ορμή" χωρίς να δημιουργεί αστάθεια.

Η επιλογή του SGD έναντι άλλων αλγορίθμων όπως Adam έγινε για λόγους σταθερότητας και καλύτερης γενίκευσης σε προβλήματα ταξινόμησης εικόνων, ιδιαίτερα όταν πρόκειται για fine-tuning προεκπαιδευμένων μοντέλων.

- Απώλειες: sparse categorical crossentropy για κάθε έξοδο. Η sparse_categorical_crossentropy είναι κατάλληλη όταν οι στόχοι (labels) είναι

ακέραιοι (π.χ. 0, 1, 2...). Χρησιμοποιείται για πολυταξική ταξινόμηση, που ταιριάζει με softmax εξόδους.

- Ακρίβειες: Δημιουργείται επίσης λεξικό, που αντιστοιχίζει την μετρική ακρίβειας (accuracy) σε κάθε έξοδο του μοντέλου. Έτσι, κατά την εκπαίδευση και αξιολόγηση, θα παρακολουθείται ξεχωριστά η ακρίβεια ανά κατηγορία εξόδου.

Κατά το fine-tuning, το μοντέλο ξεπαγώνει τα layers του base model από συγκεκριμένο σημείο και μετά, πριν ξαναεκπαιδευτεί για μερικές ακόμη εποχές. Συγκεκριμένα από το block5e_expand_conv για το EfficientNetB3 και από το mixed7 για το InceptionV3.

Η επιλογή του σημείου ξεπαγώματος έγινε σύμφωνα με τη βιβλιογραφία:

- Για EfficientNetB3: το block5e_expand_conv είναι ένα ενδιάμεσο σημείο όπου αρχίζουν να εξάγονται πιο αφηρημένα χαρακτηριστικά. Προτείνεται για fine-tuning σύμφωνα με την πηγή [32] και επίσης έγινε κατάληξη σε αυτό το επίπεδο, σύμφωνα και με μελέτη της αρχιτεκτονικής επιπέδων της βάσης του EfficientNetB3 και συγκεκριμένα των συνελικτικών επιπέδων πριν και μετά από αυτό, διαθέσιμη στο ΠΑΡΑΡΤΗΜΑ Χ.
- Για InceptionV3: το mixed7 είναι η τελευταία ενδιάμεση "mixed" μονάδα με καλή ισορροπία υπολογιστικού κόστους και απόδοσης, Προτείνεται για fine-tuning στην επίσημη τεκμηρίωση της Keras για το InceptionV3 [31] και επίσης έγινε κατάληξη σε αυτό το επίπεδο, σύμφωνα και με μελέτη της αρχιτεκτονικής επιπέδων της βάσης του στην επίσημη τεκμηρίωση της Keras για το InceptionV3 και συγκεκριμένα των συνελικτικών επιπέδων πριν και μετά από αυτό, διαθέσιμη στο ΠΑΡΑΡΤΗΜΑ ΧΙ.
- Στις Τρίτες και Τελικές εκδόσεις μόνο, στο καθένα αποφεύγεται το ξεπάγωμα των επιπέδων (layers) Κανονικοποίησης Παρτίδας (Batch Normalization) και εφαρμόζονται επιμέρους βάρη αποτυχίας (loss weights) στην κάθε ετικέτα, πριν τη μεταγλώττιση (compile), στη φάση fine-tuning. Περαιτέρω ανάλυση περί αυτού υπάρχει στο υποκεφάλαιο «Τρίτες και Τελευταίες εκδόσεις (3.5.3).

Ακολουθεί η αναλυτική παρουσίαση των τριών εκδόσεων εκπαίδευσης:

3.5.1 Πρώτες Εκδόσεις

Στην πρώτη έκδοση των πολυ-εξόδου μοντέλων:

- Δεν είχε γίνει καμία γενίκευση στα δεδομένα σχετικά με τις ετικέτες baseColour και season, δηλαδή χρησιμοποιούνταν όλοι οι αρχικοί τύποι χρωμάτων (π.χ. "navy blue", "metallic") και εποχών (π.χ. "spring", "fall", "winter", "summer"), χωρίς συγχώνευση των απαραίτητων (χωρίς εννοείται αυτό να επηρεάζει ιδιαίτερα την ποιότητα των τελικών προβλέψεων και την εμπειρία του χρήστη). Αυτό είχε ως αποτέλεσμα πολύ χαμηλές επιδόσεις στους εν λόγω άξονες, λόγω των μικρών πληθυσμιακών ομάδων.
- Κατά το fine-tuning, δεν διατηρούνταν παγωμένα τα Batch Normalization layers, κάτι που επηρέασε αρνητικά τη σταθερότητα των στατιστικών (moving mean και variance), οδηγώντας σε υποβάθμιση της απόδοσης.
- Δεν εφαρμόστηκαν loss weights, οπότε το μοντέλο αντιμετώπιζε όλα τα χαρακτηριστικά ισοδύναμα, χωρίς έμφαση σε πιο σημαντικά ή δύσκολα προς εκμάθηση χαρακτηριστικά.

- Δεν υπήρχε κανονικοποίηση στις εξόδους (δηλαδή δεν χρησιμοποιήθηκε L2 regularization (0.01)), κάτι που έκανε το μοντέλο επιρρεπές σε υπερπροσαρμογή (overfitting).
- Το data augmentation περιοριζόταν σε ένα μόνο οριζόντιο καθρέφτισμα (horizontal_flip=True), που βασίστηκε στην μεθοδολογία της έρευνας [8], όμως αποδείχθηκε ανεπαρκές για γενίκευση σε εικόνες από πραγματικά σενάρια (π.χ. λήψεις τυφλών χρηστών), προκαλώντας υπερπροσαρμογή (overfitting).
- Η εκπαίδευση έγινε σε δύο φάσεις:
 - Πρώτη φάση: 50 εποχές με παγωμένο το βασικό μοντέλο εκτελώντας διαδικασία εξαγωγή; χαρακτηριστικών (feature extraction).
 - Δεύτερη φάση: 20 επιπλέον εποχές, μετά την εφαρμογή fine-tuning.

Η επιλογή 50 και 20 εποχών έγινε με σκοπό να εξασφαλιστεί αρχικά σταθερή σύγκλιση στις εξόδους των χαρακτηριστικών και στη συνέχεια βελτιστοποίηση με fine-tuning.

3.5.2 Δεύτερες Εκδόσεις

Στη δεύτερη έκδοση πραγματοποιήθηκαν μικρές βελτιώσεις:

- Πραγματοποιήθηκε γενίκευση στα labels baseColour και season, συγχωνεύοντας παρόμοιες κατηγορίες (π.χ. "navy blue" και "blue" σε "blue", "spring" και "summer" σε "summer/spring"). Αυτή η επιλογή έγινε βάσει της μικρής παρουσίας ορισμένων κατηγοριών στο dataset και οδήγησε σε αισθητή βελτίωση της ακρίβειας στα εν λόγω χαρακτηριστικά.
- Ωστόσο, παρέμειναν τα ίδια προβλήματα με την πρώτη έκδοση:
 - Δεν διατηρήθηκαν παγωμένα τα Batch Normalization layers.
 - Δεν υπήρχαν loss weights.
 - Δεν υπήρχε kernel_regularizer στα output branches.
 - Υπήρχε μόνο horizontal flip ως augmentation.
- Η εκπαίδευση και εδώ ακολούθησε δύο φάσεις:
 - 50 εποχές με παγωμένο το βασικό μοντέλο.
 - 20 εποχές, μετά την εφαρμογή fine-tuning.

Η επιλογή των εποχών ήταν ίδια, προκειμένου να διευκολυνθεί η σύγκριση αποτελεσμάτων μεταξύ των εκδόσεων.

3.5.3 Τρίτες και τελικές Εκδόσεις

Με απόπειρα προς σημαντική βελτίωση στην ακρίβεια πολλαπλών ετικετών, η τρίτη και τελική έκδοση του πολυ-εξόδου μοντέλου ενσωμάτωσε πολλαπλές βελτιώσεις:

- Η γενίκευση των ετικετών baseColour και season παρέμεινε, καθώς είχε αποδειχθεί αποδοτική.
- Για πρώτη φορά, τα επίπεδα Κανονικοποίησης Παρτίδας (Batch Normalization layers) παρέμειναν παγωμένα και κατά τη φάση του fine-tuning. Η επιλογή αυτή έγινε ώστε να διατηρηθούν τα ήδη εκπαιδευμένα στατιστικά χαρακτηριστικά (mean, variance) του ImageNet, καθώς η εκ νέου εκπαίδευσή τους σε μικρότερο dataset ενδέχεται να επιφέρει αστάθεια. Επιπλέον, το ξεπάγωμα των στρωμάτων

αυτών μπορεί να οδηγήσει σε "προσαρμογή" του μοντέλου σε περιορισμένα batches κατά το fine-tuning, με αποτέλεσμα χαμηλότερη γενίκευση. [8], [17], [18]

- Εφαρμόστηκαν loss weights κατ'α τη φάση του fine-tuning, ώστε να δοθεί προτεραιότητα στα χαρακτηριστικά με χαμηλότερη απόδοση:
 - articleType: 1.2 – κύριο χαρακτηριστικό, πιο σημαντικό.
 - subCategory: 0.8 – εύκολα προβλέψιμο, μικρότερη ανάγκη για βάρος.
 - baseColour: 1.5 – πολύ χαμηλή ακρίβεια χωρίς ενίσχυση.
 - season: 1.5 – παρόμοιο πρόβλημα ακρίβειας.
 - usage: 1.0 – ισορροπημένο.
 - masterCategory: 0.7 – εύκολα προβλέψιμο.
 - gender: 1.0 – ισορροπημένο.
- Κάθε έξοδος ενισχύθηκε με L2 regularization (0.01), ώστε να περιοριστεί η υπερπροσαρμογή, ιδιαίτερα στα χαρακτηριστικά με πολλές κατηγορίες. Εφαρμόζεται L2 κανονικοποίηση στους βάρους του πυρήνα (kernel) του επιπέδου, με τιμή 0.01. Η L2 τιμωρία ενισχύει την γενίκευση του μοντέλου και δρα ως επιπλέον φίλτρο κανονικοποίησης (regularization), παράλληλα με το Dropout.
- Το data augmentation επεκτάθηκε σημαντικά με στόχο την προσομοίωση λήψεων από άτομα με οπτική αναπηρία:
 - horizontal_flip=True: αντιστροφή.
 - brightness_range=[0.8, 1.2]: αυξομείωση φωτεινότητας.
 - zoom_range=0.1: μικρές αλλαγές κλίμακας.
 - rotation_range=15: τυχαίες περιστροφές.
 - width_shift_range=0.05, height_shift_range=0.05: μετατοπίσεις.
 - shear_range=0.05: λοξότητα.

Όλα τα παραπάνω αυξάνουν τη γενίκευση του μοντέλου σε πραγματικά περιβάλλοντα λήψης φωτογραφιών (αντιστάθμιση κακής γωνίας, κίνησης, φωτισμού κλπ.) και αντιστοιχούν σε τυχόν λάθη που μπορεί να προκληθούν κατά την λήψη φωτογραφιών, από οπτικά ανάπηρους χρήστες, χωρίς τη γνώση τους (π.χ. στραβή φωτογραφία ή ελλειπής φωτισμός).

- Η εκπαίδευση των τρίτων και τελικών εκδόσεων των πολύ-ετικετικών μοντέλων βασίστηκε σε τροποποιημένη στρατηγική δύο φάσεων, με στόχο τη βελτίωση της γενικής απόδοσης του μοντέλου και την πιο σταδιακή προσαρμογή του στις απαιτήσεις του προβλήματος. Η επιλογή αυτής της μεθοδολογίας στηρίχθηκε στην ιδέα της σταδιακής εκμάθησης (curriculum learning), ξεκινώντας από το πιο βασικό και αναγνωρίσιμο χαρακτηριστικό, το articleType, και στη συνέχεια επεκτείνοντας την εκπαίδευση σε όλα τα υπόλοιπα χαρακτηριστικά:
 - 1η Φάση – Εκπαίδευση μόνο για articleType (25 εποχές): Στην πρώτη φάση, το base model παρέμεινε παγωμένο, ενώ εκπαιδεύτηκαν μόνο οι νευρωνικές συνδέσεις που οδηγούν στην έξοδο του χαρακτηριστικού articleType. Η διάρκεια της φάσης ορίστηκε στις 25 εποχές, αριθμός επαρκής ώστε να σταθεροποιηθεί η εκμάθηση χωρίς να προκύψει υπερπροσαρμογή (overfitting). Ο λόγος που επιλέχθηκε το articleType για την πρώτη φάση είναι διότι αποτελεί το πιο κεντρικό και αντιπροσωπευτικό χαρακτηριστικό ενός ενδύματος. Με άλλα λόγια, η σωστή κατηγοριοποίηση

του τύπου του ρούχου (π.χ. "μπλούζα", "φούστα", "παλτό") αποτελεί πυρήνα πληροφορίας πάνω στον οποίο μπορεί να στηριχτεί η εκμάθηση των υπόλοιπων χαρακτηριστικών. Έτσι, το μοντέλο μαθαίνει πρώτα να αναγνωρίζει τι είναι το ένδυμα καθαυτό, και όχι πληροφορίες γύρω από αυτό (όπως εποχή, χρήση ή φύλο).

- 2η φάση – Εκπαίδευση με Fine-tuning για όλα τα χαρακτηριστικά (30 εποχές): Αφού ολοκληρώθηκε η πρώτη φάση, ξεκίνησε η δεύτερη φάση εκπαίδευσης διάρκειας 30 εποχών, κατά την οποία: 1. Πραγματοποιήθηκε fine-tuning, δηλαδή ξεπάγωμα, του base model (από το σημείο block5e_expand_conv για EfficientNet ή mixed7 για InceptionV3). 2. Ενεργοποιήθηκαν όλες οι εξόδους του μοντέλου για τα υπόλοιπα χαρακτηριστικά (season, baseColour, gender, masterCategory, subCategory, usage). 3. Έμειναν παγωμένες οι στρώσεις-επίπεδα (layers) Batch Normalization, ώστε να διατηρηθούν οι στατιστικές του προεκπαιδευμένου μοντέλου. 4. Εφαρμόστηκαν συντελεστές βαρύτητας (loss weights) για να ισορροπηθεί η συμβολή κάθε χαρακτηριστικού στη συνολική απώλεια. 5. Χρησιμοποιήθηκε πλήρες augmentation για βελτίωση της γενίκευσης. Η επιμήκυνση της δεύτερης φάσης (30 εποχές) κρίθηκε απαραίτητη ώστε να δοθεί επαρκής χρόνος στο δίκτυο να προσαρμοστεί στην ταυτόχρονη εκμάθηση όλων των χαρακτηριστικών, ειδικά αφού έγινε πρώτα fine-tuning στο base model.

Η συγκεκριμένη στρατηγική εκπαίδευσης αποδείχθηκε ιδιαίτερα αποδοτική, κυρίως επειδή το χαρακτηριστικό articleType παρουσιάζει στενή συσχέτιση με τα περισσότερα από τα υπόλοιπα χαρακτηριστικά. Ο λόγος που η πρώτη φάση περιλαμβάνει την εκπαίδευση πάνω στο articleType και η δεύτερη φάση την εκπαίδευση πάνω σε όλα τα χαρακτηριστικά είναι, διότι παρατηρήθηκε κυρίως από τις δύο πρώτες εκδόσεις, ότι η εκμάθηση της περισσότερο ειδικής πληροφορίας πρώτα (articleType) βοηθά πιο πολύ στην εκμάθηση της πιο γενικής πληροφορίας μετά (υπόλοιπα χαρακτηριστικά πχ. season, mainCategory), απ' ότι η εκμάθηση όλων ταυτόχρονα κατευθείαν ή της γενικής πληροφορίας πρώτα (όλα τα υπόλοιπα χαρακτηριστικά και της πιο ειδικής πληροφορίας (articleType) ύστερα. Αυτό ισχύει διότι το χαρακτηριστικό articleType περιέχει πιο εξειδικευμένη και συγκεκριμένη πληροφορία που συχνά περικλείει ή υπονοεί τα υπόλοιπα γενικά χαρακτηριστικά, όπως το masterCategory ή το season. Εκπαιδεύοντας το μοντέλο πρώτα στο articleType, του δίνεται η δυνατότητα να μάθει λεπτομερείς και διακριτές αναπαραστάσεις, οι οποίες στη συνέχεια λειτουργούν ως πιο πλούσια και καθοριστικά εσωτερικά χαρακτηριστικά (feature representations) για την πρόβλεψη των υπόλοιπων πιο γενικών ετικετών. Έτσι, η εκπαίδευση γίνεται πιο στοχευμένη, και το μοντέλο αποδίδει καλύτερα συνολικά. Για παράδειγμα, αν το ρούχο είναι κοντομάνικο μπλουζάκι, είναι εύλογο να συνδέεται με θερινή εποχή, καθημερινή χρήση και συγκεκριμένο φύλο, ανάλογα με τον σχεδιασμό του. Αντίστοιχα, η αναγνώριση μιας φούστας παραπέμπει συχνά σε γυναικείο φύλο, καθώς και σε σαφώς ορισμένες κατηγορίες και ύφους. Επιπλέον, η προτεραιότητα στην εκμάθηση του articleType διευκολύνει το δίκτυο να επικεντρώνεται στο ίδιο το ρούχο κατά την επεξεργασία της εικόνας, βελτιώνοντας έτσι και την αναγνώριση του βασικού χρώματος (baseColour), χωρίς να παρασύρεται από στοιχεία του φόντου, στην αναγνώριση του γενικού βασικού χρώματος όλης της εικόνας ή δευτερεύοντα αντικείμενα. Η μείωση των εποχών της πρώτης φάσης έγινε επειδή η εκπαίδευση μόνο του articleType απαιτεί λιγότερη προσαρμογή. Αντιθέτως, η δεύτερη φάση επιμηκύνθηκε (30 εποχές) ώστε να αξιοποιήσει πλήρως το fine-tuning όλων των χαρακτηριστικών. [8], [15], [16], [17], [23], [24], [25]

3.6 Μονό-Εξόδου Μοντέλα (Single-Task) και Διαδικασία Εκπαίδευσης

Στο πλαίσιο της παρούσας εργασίας, εκτός από τα πολυ-εξόδου (multi-output) μοντέλα, αναπτύχθηκαν και τρία ξεχωριστά μοντέλα μονο-εξόδου (single-task) με αποκλειστικό σκοπό την ταξινόμηση του γνωρίσματος `articleType`. Το `articleType` ήταν το μόνο χαρακτηριστικό το οποίο φάνηκε να επωφελείται σημαντικά από την εκπαίδευση ως ξεχωριστό μοντέλο. Αυτό οφείλεται στη σχετική του αυτονομία και σημασία: δεν εξαρτάται έντονα από άλλα χαρακτηριστικά, αλλά τα επηρεάζει. Αντίθετα, τα υπόλοιπα χαρακτηριστικά, όταν εκπαιδεύονται όλα μαζί μέσα από την multi-task αρχιτεκτονική, εμφανίζουν συχνά βελτιωμένη απόδοση, καθώς επωφελούνται από την μοιραζόμενη πληροφορία και τα συσχετιζόμενα μοτίβα που μαθαίνει το κοινό backbone (κορμός) του μοντέλου. Τα παρακάτω μοντέλα βασίστηκαν στις αρχιτεκτονικές EfficientNetB0, EfficientNetB1 και EfficientNetB3. Ο λόγος που επιλέχθηκε να γίνει πειραματισμός με τις τρεις αυτές αρχιτεκτονικές, περιγράφεται πλήρως στο υποκεφάλαιο 3.3. Τα μοντέλα εκπαιδεύτηκαν ξεχωριστά, προκειμένου να επιτευχθεί η μέγιστη δυνατή ακρίβεια πρόβλεψης στο συγκεκριμένο χαρακτηριστικό (ακριβής κατηγορία ρούχου - `articleType`), όντας το πιο σημαντικό σε σχέση με τα υπόλοιπα, αποφεύγοντας την πιθανή μικρή "παρεμβολή" άλλων προβλεπόμενων χαρακτηριστικών, κατά εκμάθησή του. Η διαδικασία χτίσης και υλοποίησης των αρχιτεκτονικών των μονό-ετικετικών μοντέλων υλοποιήθηκε στο Python αρχείο `articleType_efnet_model.py` (ΠΑΡΑΡΤΗΜΑ V).

Αρχιτεκτονική και Fine-tuning

Η υλοποίηση των μοντέλων πραγματοποιήθηκε μέσω της κλάσης `EfficientNetArtTypeClothingClassifier`, η οποία φορτώνει την προεκπαιδευμένη έκδοση του αντίστοιχου EfficientNet μοντέλου (B0, B1 ή B3, αναλόγως με το όνομα που της δίνεται) με βάρη από το ImageNet. Στόχος ήταν η αξιοποίηση της μεταφοράς μάθησης (transfer learning), προσαρμόζοντας το δίκτυο στο νέο καθήκον ταξινόμησης.

Κατά την κατασκευή του τελικού μοντέλου μέσω της μεθόδου `build_fine_tuned_model`, εφαρμόστηκε το εξής σχήμα fine-tuning:

- Όλοι οι αδρανείς στρώσεις του βασικού μοντέλου αποψύχθηκαν (unfrozen) ώστε να είναι εκπαιδεύσιμες.
- Οι στρώσεις κανονικοποίησης παρτίδας (BatchNormalization) παρέμειναν παγωμένες/μη εκπαιδεύσιμες (`trainable=False`), προκειμένου να διατηρήσουν τη σταθερότητα κατά την εκπαίδευση και να μην διαταράξουν τα προεκπαιδευμένα στατιστικά τους.
- Πριν την είσοδο του δείγματος στο δίκτυο, εφαρμόστηκε επανακλιμάκωση (resizing) εικόνας στο κατάλληλο μέγεθος εισόδου (224×224 για B0, 240×240 για B1) και η ενδεδειγμένη κανονικοποίηση (`preprocess_input`) για τα EfficientNet μοντέλα. Ο λόγος που γίνεται εδώ η επανακλιμάκωση, είναι για να μπορούν να χρησιμοποιούν και τα 3 μοντέλα το ίδιο τελικό προεπεξεργασμένο σύνολο δεδομένων.

Το τελικό μέρος του μοντέλου περιλάμβανε:

- `GlobalAveragePooling2D` (περεταίρω περιγραφή στο υποκεφάλαιο 3.4).
- Dropout layer με τιμές 0.2, 0.3 και 0.4 για τα B0, B1 και B3 αντίστοιχα, ώστε να ελαχιστοποιηθεί ο υπερπροσαρμογή (overfitting). Οι τιμές διαφέρουν ανά μοντέλο, λόγω της διαφοράς στο μέγεθος και την πολυπλοκότητα μεταξύ των διαφορετικών αρχιτεκτονικών (περεταίρω περιγραφή στο υποκεφάλαιο 3.4).

- Πυκνή έξοδο (Dense) με συνάρτηση ενεργοποίησης softmax και 87 εξόδους (μία για κάθε κατηγορία articleType). Στην περίπτωση του EfficientNetB3 προστέθηκε και L2 regularization (0.01) για περαιτέρω προστασία από υπερεκπαίδευση, λόγω της μεγαλύτερης και πιο πολύπλοκης αρχιτεκτονικής που κατέχει, σε συνδυασμό με το μεσαίου μεγέθους σύνολο δεδομένων της εργασίας (περεταίρω περιγραφή στο υποκεφάλαιο 3.4).

Η βελτιστοποίηση πραγματοποιήθηκε με χρήση του αλγορίθμου SGD με momentum 0.9 και ρυθμός μάθησης (learning rate) ήταν 0.001 σε νέα εκπαίδευση.

Εκπαίδευση Μοντέλων

Κάθε μοντέλο εκπαιδεύτηκε από την αρχή, με όλους τους παγωμένους περιορισμούς να έχουν αρθεί (fine-tuned από την αρχή), πέρα των στρώσεων κανονικοποίησης παρτίδας (Batch Normalization Layers). Δεν εφαρμόστηκε αρχική φάση "πάγωμα + εκπαίδευση της κορυφής" όπως στα πολυ-εξόδου μοντέλα, αλλά τα μοντέλα προσαρμόστηκαν κατευθείαν στο νέο καθήκον.

Οι αριθμοί εποχών εκπαίδευσης για κάθε μοντέλο ήταν:

- EfficientNetB0: 12 εποχές (epochs)
- EfficientNetB1: 25 εποχές (epochs)
- EfficientNetB3: 30 εποχές (epochs)

Ο αριθμός εποχών καθορίστηκε λαμβάνοντας υπόψη την υπολογιστική πολυπλοκότητα κάθε μοντέλου και την ικανότητά του να μαθαίνει πιο σύνθετες αναπαραστάσεις.

Το EfficientNetB0, ως το απλούστερο και ταχύτερο μοντέλο, δεν απαιτούσε πολλές εποχές ώστε να συγκλίνει. Η χρήση βασικού augmentation (μόνο horizontal_flip) διατήρησε την ταχύτητα εκπαίδευσης υψηλή, αλλά με περιορισμένες δυνατότητες γενίκευσης. Η απόφαση αυτή έγινε επίσης λαμβάνοντας υπόψιν και την έρευνα του [8].

Το EfficientNetB1 είναι σημαντικά πιο ικανό από το B0, και ως εκ τούτου απαιτούσε περισσότερες εποχές (25) για να αξιοποιήσει το επιπλέον βάθος και τις παραμέτρους του. Το data augmentation που εφαρμόστηκε ήταν πιο ενισχυμένο, περιλαμβάνοντας περιστροφή, μετατοπίσεις, μεγέθυνση και οριζόντια αναστροφή. Αυτό ενίσχυσε τη γενίκευση, χωρίς να αυξήσει υπερβολικά τον κίνδυνο υπερεκπαίδευσης.

Το EfficientNetB3, ως το ισχυρότερο μοντέλο με σημαντικά μεγαλύτερη υπολογιστική πολυπλοκότητα, απαιτούσε μεγαλύτερο αριθμό εποχών (30) ώστε να αξιοποιήσει πλήρως τις δυνατότητές του. Επιπλέον, για την αύξηση της ανθεκτικότητας σε παραμορφώσεις, φωτισμό και γεωμετρικές αποκλίσεις, εφαρμόστηκε εμπλουτισμένο σχήμα data augmentation, το οποίο περιλάμβανε: Οριζόντια αναστροφή, Φωτεινότητα (brightness_range), Περιστροφή, μετατόπιση (σε πλάτος και ύψος), μεγέθυνση και διάτμηση (shear_range)

Το σύνολο αυτών των ρυθμίσεων στόχευε στην πλήρη αξιοποίηση της ικανότητας του κάθε μοντέλου να μάθει από πιο ποικιλόμορφα δεδομένα και να παράγει πιο αξιόπιστες προβλέψεις.

Δεν πραγματοποιήθηκε πειραματισμός με την ενδιάμεση έκδοση EfficientNetB2, καθώς η τελική απόδοσή του είναι παρόμοια με εκείνη του EfficientNetB1 και του EfficientNetB3. Δεδομένου ότι το EfficientNetB1 καλύπτει ικανοποιητικά το φάσμα απλότητας-ακρίβειας, δεν κρίθηκε σκόπιμη η προσθήκη του B2 στο πείραμα, καθώς δεν αναμενόταν να προσφέρει ουσιαστικές βελτιώσεις που να δικαιολογούν τον επιπλέον υπολογιστικό φόρτο και χρόνο. [8]

3.7 Μέθοδοι αξιολόγησης και Μετρικά Απόδοσης

Η αξιολόγηση μοντέλων βαθιάς μάθησης αποτελεί ένα από τα πιο κρίσιμα στάδια σε όλη τη διαδικασία ανάπτυξης ενός συστήματος τεχνητής νοημοσύνης και είναι απαραίτητη για να διαπιστωθεί κατά πόσο το μοντέλο που έχουμε εκπαιδεύσει όχι μόνο θυμάται τα δεδομένα εκπαίδευσης αλλά και μπορεί να γενικεύσει σε νέα και άγνωστα δεδομένα που δεν έχει ξαναδεί. Η ικανότητα γενίκευσης είναι ο ακρογωνιαίος λίθος για την πρακτική εφαρμογή των μοντέλων ειδικά σε προβλήματα πραγματικού κόσμου όπως η αναγνώριση και ταξινόμηση ρούχων όπου οι συνθήκες φωτισμού, η θέση του αντικειμένου και οι ποικίλες παραλλαγές στο στυλ μπορούν να διαφέρουν σημαντικά από τις εικόνες του εκπαιδευτικού συνόλου. Η διαδικασία αξιολόγησης των μοντέλων υλοποιήθηκε στο Python αρχείο `evaluation.py` (ΠΑΡΑΡΤΗΜΑ VII).

Διαδικασία Αξιολόγησης Μοντέλου

Η κλάση `ModelEvaluator` που έχει υλοποιηθεί αποτελεί ένα ολοκληρωμένο εργαλείο αξιολόγησης ενός εκπαιδευμένου μοντέλου ταξινόμησης ενδυμάτων, το οποίο μπορεί να λειτουργεί είτε με μονό-ετικετικά, είτε με πολύ-ετικετικά μοντέλα. Με άλλα λόγια, μπορεί να διαχειριστεί τόσο μοντέλα που προβλέπουν μία μόνο κατηγορία, όπως για παράδειγμα το άρθρο ρούχου, όσο και μοντέλα που προβλέπουν ταυτόχρονα πολλά χαρακτηριστικά, όπως τη χρήση, το χρώμα και την εποχή.

Αρχικά, στην κλάση γίνεται αποδοχή του μοντέλου και των κωδικοποιητών ετικετών (`label encoders`) που χρησιμοποιούνται για τη μετατροπή των κατηγοριών από κείμενο σε αριθμητικές ετικέτες και αντίστροφα. Αυτό είναι απαραίτητο γιατί το μοντέλο δουλεύει με αριθμημένες κλάσεις ενώ τα δεδομένα είναι σε μορφή κειμένου, οπότε χρειάζεται η σωστή αντιστοίχιση μεταξύ τους.

Κατά την κλήση της μεθόδου `evaluate`, ο κώδικας δέχεται ως είσοδο τα δεδομένα ελέγχου (`test`), δηλαδή τις εικόνες, τις αντίστοιχες ετικέτες που μπορεί να αφορούν ένα ή πολλαπλά χαρακτηριστικά, προαιρετικά το ιστορικό από την εκπαίδευση και στοιχεία για τον χρόνο και τον αριθμό των εποχών εκπαίδευσης. Με αυτόν τον τρόπο επιτυγχάνεται ολοκληρωμένη αποτίμηση της απόδοσης αλλά και του κόστους εκπαίδευσης.

Η μέθοδος `evaluate` ξεκινά εντοπίζοντας αν οι ετικέτες που δόθηκαν αφορούν μονό ή πολλαπλούς στόχους. Αν οι ετικέτες δεν είναι ήδη σε μορφή λεξικού, τις μετατρέπει ώστε να γίνει ένα λεξικό με ένα μόνο κλειδί, που είναι η συνηθισμένη περίπτωση όταν έχουμε μονο-ετικετικό μοντέλο (`single-task`). Έτσι διασφαλίζεται η ομοιομορφία στην επεξεργασία των δεδομένων.

Στη συνέχεια, χρησιμοποιούνται οι `label encoders` για να μετατραπούν οι κατηγορίες του σετ ελέγχου (`test set`) σε αριθμητική μορφή, ώστε να μπορούν να συγκριθούν με τις προβλέψεις του μοντέλου που επίσης παράγονται ως αριθμητικοί δείκτες. Αυτό επιτρέπει τη σύγκριση και την αξιολόγηση.

Έπειτα, μετριέται ο χρόνος που χρειάζεται το μοντέλο για να κάνει προβλέψεις σε όλο το `test set` και υπολογίζεται ο μέσος χρόνος ανά εικόνα. Αυτή η πληροφορία είναι πολύ χρήσιμη, γιατί δίνει μια εκτίμηση της ταχύτητας του μοντέλου σε πραγματικές συνθήκες χρήσης, κάτι που είναι σημαντικό ειδικά σε εφαρμογές σαν της παρούσας πτυχιακής που η απόκριση σε συνθήκες πραγματικού χρόνου έχει ιδιαίτερη σημασία για την εμπειρία του χρήστη.

Όταν το μοντέλο προβλέψει, ο κώδικας ελέγχει τη μορφή των αποτελεσμάτων. Τα αποτελέσματα μπορεί να είναι: λεξικό με προβλέψεις ανά χαρακτηριστικό, λίστα προβλέψεων για κάθε χαρακτηριστικό ή μονοδιάστατο numpy πίνακας (`array`) στην

περίπτωση μονο-ετικετικού μοντέλου (single-task). Αν η μορφή δεν είναι κάποια από αυτές, το σύστημα επιστρέφει σφάλμα που υποδεικνύει ασυμφωνία στα δεδομένα.

Για κάθε χαρακτηριστικό του test set, η μέθοδος προχωρά ως εξής: 1. Πρώτα μετατρέπει τις προβλέψεις σε διακριτές ετικέτες παίρνοντας το δείκτη της πιο πιθανής κατηγορίας (μέσω της συνάρτησης `argmax`) ανά δείγμα. Έπειτα ελέγχει αν όλες οι δυνατές κλάσεις υπάρχουν και στα test δεδομένα και προειδοποιεί αν υπάρχουν κλάσεις που απουσιάζουν, ώστε να γνωρίζουμε περιορισμούς στην εκπαίδευση ή στα δεδομένα.

Στη συνέχεια, υπολογίζεται το αναφορά κατηγοριοποίησης (classification report), που περιλαμβάνει μετρικές όπως ακρίβεια, ανάκληση, F1-score και άλλα μετρικά ανά χαρακτηριστικό, καθώς και συνολικοί πίνακες σύγχυσης (confusion matrices) για κάθε προβλεπόμενο χαρακτηριστικό. Η ανάλυση των μετρικών που υπολογίζονται, γίνεται παρακάτω.

Μετρικά Απόδοσης

Για κάθε χαρακτηριστικό υπολογίζονται οι εξής μετρικά:

- Accuracy (Ακρίβεια/Ορθότητα): Το ποσοστό των σωστών προβλέψεων στο σύνολο των δειγμάτων.

$$\text{Ακρίβεια} = \frac{\text{Σωστές Προβλέψεις}}{\text{Λάθος Προβλέψεις}}$$

Πολύ χρήσιμη όταν οι κλάσεις είναι ισορροπημένες, μπορεί όμως να παραπλανήσει αν έχουμε μεγάλη ασυμμετρία στις κλάσεις.

- Precision, Recall, F1-score (Ακρίβεια/Επικύρωση, Ανάκληση/Ευαισθησία, F1-σκορ). Υπολογίζονται ανά κλάση και συνολικά (micro και macro):

- Precision (Ακρίβεια): Ποσοστό σωστών θετικών προβλέψεων επί όλων των θετικών προβλέψεων.

$$\text{Precision} = \frac{TP}{FP + TP}, \text{ όπου}$$

TP = True Positives (σωστές θετικές προβλέψεις)

FP = False Positives (λανθασμένες θετικές προβλέψεις)

- Recall (Ανάκληση): Ποσοστό σωστών θετικών προβλέψεων επί όλων των πραγματικών θετικών

$$\text{Recall} = \frac{TP}{TP + FN}, \text{ όπου}$$

TP = True Positives (σωστές θετικές προβλέψεις)

FN = False Negatives (λανθασμένες αρνητικές προβλέψεις)

- F1-score: Ο αρμονικός μέσος Precision και Recall, δείκτης ισορροπημένης απόδοσης.

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Top-k Accuracy (Τοπ-κ Ακρίβεια): Το top-3 ή top-5 accuracy δείχνει αν η σωστή κλάση βρίσκεται μέσα στις τρεις ή τις πέντε πιο πιθανές προβλέψεις. Στην περίπτωση πολλών κατηγοριών, η ακριβής κορυφαία πρόβλεψη μπορεί να μην είναι πάντα η σωστή., αλλά είναι πολύ χρήσιμο όταν υπάρχει ασάφεια ή ομοιότητα ανάμεσα σε κατηγορίες (π.χ. παρόμοια χρώματα ή στυλ ρούχων).

- Confusion Matrix (Πίνακας Σύγχυσης): Απεικονίζει πόσες φορές κάθε πραγματική κλάση ταξινομήθηκε ως κάποια άλλη και έτσι επιτρέπει τον εντοπισμό συγκεκριμένων ζευγών κλάσεων που το μοντέλο συγχέει συχνά. Σημαντικό εργαλείο για τη στοχευμένη βελτίωση του μοντέλου.

Με την αποτύπωση όλων των παραπάνω μετρικών και την οπτικοποίηση των πινάκων σύγκρισης μέσω θερμοχάρτων (heatmaps) δημιουργείται μια σαφή εικόνα για το ποιες κατηγορίες προκαλούν προβλήματα στο μοντέλο και να στοχεύσουμε σε διορθωτικές ενέργειες. Τα παραπάνω βοήθησαν κάθε φορά που γινόταν αξιολόγηση μιας έκδοσης ενός μοντέλου, με αποτέλεσμα την καθοδήγηση σε περαιτέρω βελτιώσεις και την δημιουργία των επόμενων εκδόσεων κάθε φορά. Ακόμα βοήθησαν αρκετά στην επιλογή όσο αναφορά το ποια από τα τελικά μοντέλα θα χρησιμοποιηθούν στο τελικό συνολικό σύστημα.

3.8 Μέθοδος Εκτέλεσης Αναγνώρισης

Η μέθοδος εκτέλεσης αναγνώρισης αποτελεί το τελικό κομμάτι της διαδικασίας πρόβλεψης χαρακτηριστικών ρουχισμού από εικόνες, χρησιμοποιώντας τα εκπαιδευμένα μοντέλα βαθιάς μάθησης που αναπτύχθηκαν στην παρούσα εργασία. Η διαδικασία αναγνώρισης υλοποιήθηκε στο Python αρχείο `prediction.py` (ΠΑΡΑΡΤΗΜΑ VIII).

Για το σκοπό αυτό, υλοποιήθηκε η κλάση `ClothingAttributePredictor`, η οποία αναλαμβάνει να επεξεργαστεί την είσοδο εικόνας, να τροφοδοτήσει το μοντέλο και να επιστρέψει την τελική ταξινόμηση σε ανθρώπινα αναγνώσιμη μορφή.

Η λειτουργία της κλάσης ξεκινά με την αρχικοποίηση όπου τοποθετούνται ως ιδιότητες το εκπαιδευμένο μοντέλο (`model`), οι κωδικοποιητές ετικετών (`label_encoders`) που επιτρέπουν τη μετατροπή μεταξύ των αριθμητικών κωδικών που χρησιμοποιεί το μοντέλο και των κατηγοριών σε κείμενο, καθώς και το όνομα του μοντέλου (`model_name`). Το όνομα είναι σημαντικό ώστε να καθοριστεί η κατάλληλη μερική προεπεξεργασία των εικόνων ανάλογα με το μοντέλο που χρησιμοποιείται.

Η μερική προεπεξεργασία της εικόνας αποτελεί βασικό βήμα για να διασφαλιστεί ότι η είσοδος που δίνεται στο μοντέλο έχει το σωστό μέγεθος, μορφή και κανονικοποίηση. Συγκεκριμένα, η μέθοδος `preprocess_image_from_pil` δέχεται μια εικόνα τύπου PIL (Python Imaging Library) και την προσαρμόζει στο κατάλληλο μέγεθος εισόδου. Για παράδειγμα, εάν το μοντέλο είναι το "EffNet_artTypeB1", το μέγεθος αλλάζει στα 240x240 pixels αντί για το προεπιλεγμένο 300x300. Στη συνέχεια, η εικόνα μετατρέπεται σε πίνακα αριθμών (numpy array), εφαρμόζεται η προεπεξεργασία ειδική για το EfficientNet που περιλαμβάνει κλιμάκωση και μετατροπή τιμών pixel, και τέλος προστίθεται μια επιπλέον διάσταση για να σχηματιστεί ένα batch εισόδου με μέγεθος 1, ώστε να είναι συμβατή με το μοντέλο.

Μετά την μερική προεπεξεργασία, η μέθοδος `predict_attributes_from_pil` καλείται για να κάνει την πρόβλεψη. Αυτή τροφοδοτεί το προεπεξεργασμένο πίνακα εικόνας στο μοντέλο και λαμβάνει ως αποτέλεσμα τους πιθανότερους κωδικούς κατηγοριών. Ανάλογα με το αν το μοντέλο είναι πολυ-ετικετικό (multi-task) ή μονο-ετικετικό (single-task), η έξοδος μπορεί να είναι λεξικό με προβλέψεις για κάθε χαρακτηριστικό ή απλός πίνακας αποτελεσμάτων. Στην περίπτωση multi-task, για κάθε χαρακτηριστικό γίνεται επιλογή της κλάσης με τη μεγαλύτερη πιθανότητα (μέσω της συνάρτησης `argmax`) και η αντίστοιχη κωδική ετικέτα μετατρέπεται σε κείμενο χρησιμοποιώντας τον `label encoder` του χαρακτηριστικού. Στην περίπτωση single-task, γίνεται το ίδιο μόνο για το χαρακτηριστικό `articleType`.

Τελικά, η μέθοδος επιστρέφει ένα λεξικό όπου τα κλειδιά είναι τα χαρακτηριστικά του ρουχισμού (π.χ. `articleType`, `baseColour`) και οι τιμές οι αντίστοιχες προβλεπόμενες κατηγορίες σε μορφή κειμένου, έτοιμες για παρουσίαση ή περαιτέρω επεξεργασία από τον διακομιστή. Με αυτόν τον τρόπο, το σύστημα τεχνητής νοημοσύνης που αναπτύχθηκε

μπορεί να ενσωματωθεί απρόσκοπτα στην κινητή εφαρμογή, επιτυγχάνοντας αξιόπιστη και γρήγορη αναγνώριση χαρακτηριστικών ρουχισμού.

Τα ακριβή αποτελέσματα των αξιολογήσεων των μοντέλων, καθώς και η απόφαση για το ποια μοντέλα χρησιμοποιήθηκαν στο τελικό σύστημα, αναλύονται στο υποκεφάλαιο 6.1.

4. ΥΛΟΠΟΙΗΣΗ ΠΙΣΩ ΣΥΣΤΗΜΑΤΟΣ ΚΑΙ ΔΙΑΚΟΜΗΣΤΗ (SERVER)

Η υλοποίηση του πίσω συστήματος (backend) και διακομιστή στηρίζεται σε ένα Flask API που δέχεται αιτήματα από την Android εφαρμογή και επιστρέφει τις προβλέψεις των χαρακτηριστικών ρουχισμού. Η κύρια λειτουργία του backend είναι η φόρτωση των εκπαιδευμένων μοντέλων, η διαχείριση των HTTP αιτημάτων με αποστολή εικόνων, η εκτέλεση των προβλέψεων μέσω των μοντέλων, και η επιστροφή των αποτελεσμάτων σε κατάλληλη μορφή JSON. Η υλοποίηση επιτρέπει την εύκολη κλιμάκωση, παράλληλη εξυπηρέτηση πολλών αιτημάτων και διατηρεί το σύστημα ευέλικτο και επεκτάσιμο. Ο διακομιστής υλοποιήθηκε στο Python αρχείο `api_server.py` (ΠΑΡΑΡΤΗΜΑ ΙΧ).

4.1 Φόρτωση Δεδομένων και Μοντέλων

Κατά την εκκίνηση του Flask διακομιστή, πραγματοποιείται αρχικά η φόρτωση των απαραίτητων δεδομένων και μοντέλων. Αρχικά, φορτώνεται το ισορροπημένο σύνολο δεδομένων (balanced metadata) σε μορφή `parquet` μέσω της βιβλιοθήκης `pandas`. Αυτό το σύνολο περιλαμβάνει τις ετικέτες των χαρακτηριστικών (π.χ. `masterCategory`, `articleType`, `baseColour`) που χρησιμοποιήθηκαν κατά την εκπαίδευση των μοντέλων και είναι απαραίτητο για τη δημιουργία λεξικών που αποθηκεύουν τον αριθμό των κλάσεων ανά χαρακτηριστικό.

Με βάση τις πληροφορίες αυτές, ορίζεται ένα λεξικό `num_classes_dict` που περιέχει το πλήθος των μοναδικών κλάσεων ανά χαρακτηριστικό. Αυτό το λεξικό χρησιμοποιείται για την κατασκευή της αρχιτεκτονικής των μοντέλων.

Στη συνέχεια, φορτώνονται δύο προεκπαιδευμένα μοντέλα βαθιάς μάθησης:

- Ένα πολυ-ετικετικό (multi-task) μοντέλο (με χρήση των τελικών βαρών του `final weights`), το `EfficientNetB3` (`EfficientNetClothingClassifier`), το οποίο προβλέπει πολλαπλά χαρακτηριστικά ρούχων ταυτόχρονα. Η αρχιτεκτονική του χτίζεται βάσει του αριθμού κλάσεων, και τα αποθηκευμένα βάρη φορτώνονται από το δίσκο.
- Ένα μονο-ετικετικό (single-task) μοντέλο (με χρήση των βέλτιστων βαρών του `best weights`), το `EfficientNetB1` (`EfficientNetArtTypeClothingClassifier`) που προβλέπει αποκλειστικά το χαρακτηριστικό `articleType` (ακριβής κατηγορία). Αυτό έχει ξεχωριστή αρχιτεκτονική και μέγεθος εισόδου.

Κάθε μοντέλο συνοδεύεται από τους αντίστοιχους κωδικοποιητές ετικετών (`label encoders`) που μεταφράζουν τα αριθμητικά αποτελέσματα των προβλέψεων σε κατανοητές ετικέτες κειμένου. Τα μοντέλα και οι κωδικοποιητές φορτώνονται μία φορά κατά το ξεκίνημα του διακομιστή (`server`), ώστε να είναι έτοιμα να εξυπηρετήσουν γρήγορα τα αιτήματα.

4.2 Διαχείριση Αιτημάτων Πρόβλεψης

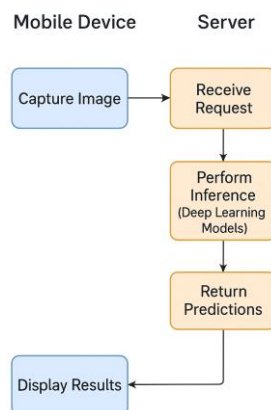
Ο Flask διακομιστής εκθέτει δύο βασικά endpoints: ένα απλό / για έλεγχο λειτουργίας, και το κύριο "/predict" που δέχεται αιτήματα POST με εικόνες.

Στο endpoint "/predict", ο διακομιστής περιμένει να λάβει ένα αρχείο εικόνας μέσα στο αίτημα. Εάν το αρχείο απουσιάζει, επιστρέφεται σφάλμα με κωδικό 400. Εφόσον το αρχείο είναι διαθέσιμο, διαβάζεται από τα bytes του και μετατρέπεται σε αντικείμενο εικόνας PIL, ενώ παράλληλα εφαρμόζεται μετατροπή σε RGB για ομοιομορφία.

Σε περίπτωση που το αρχείο δεν είναι έγκυρη εικόνα, το σύστημα αναγνωρίζει το σφάλμα και επιστρέφει κατάλληλο μήνυμα λάθους.

Μόλις η εικόνα είναι έτοιμη, γίνεται πρόβλεψη με δύο μοντέλα: το μοντέλο single-task προβλέπει το χαρακτηριστικό articleType, ενώ το multi-task μοντέλο παράγει προβλέψεις για τα όλα τα χαρακτηριστικά (π.χ. baseColour, season). Για να αποφευχθεί η διπλή εμφάνιση του articleType, αυτό αφαιρείται από τις προβλέψεις του multi-task μοντέλου. Τα αποτελέσματα συνδυάζονται σε ένα ενιαίο λεξικό με όλες τις προβλέψεις.

Τέλος, τα αποτελέσματα επιστρέφονται στον πελάτη (client), δηλαδή την κινητή εφαρμογή Android σε μορφή JSON, περιλαμβάνοντας κατάσταση επιτυχίας και τις κατηγορίες που προβλέφθηκαν. Σε περίπτωση απροσδόκητου σφάλματος, η υπηρεσία καταγράφει λεπτομέρειες στο αρχείο καταγραφής και επιστρέφει μήνυμα σφάλματος 500.



Σχήμα 8. Σχηματική Αναπαράσταση Αλληλεπίδρασης της Android Εφαρμογής στην Κινητή Συσκευή με τον Διακομιστή (Server)

Η παράλληλη εξυπηρέτηση πολλαπλών αιτημάτων εξασφαλίζεται με την ενεργοποίηση του Flask σε πολυνηματική λειτουργία (multi-threaded mode), επιτρέποντας την ταυτόχρονη επεξεργασία πολλαπλών εικόνων.

Με την παραπάνω υλοποίηση, ο διακομιστής προσφέρει ένα αξιόπιστο και αποδοτικό πίσω σύστημα (backend) για το σύστημα αναγνώρισης ρούχων, το οποίο μπορεί να ενσωματωθεί ομαλά με την Android εφαρμογή.

5. ΑΝΑΠΤΥΞΗ ΚΙΝΗΤΗΣ ΕΦΑΡΜΟΓΗΣ

5.1 Γενική Περιγραφή Εφαρμογής

Η κινητή Android εφαρμογή που υλοποιήθηκε αποτελεί το δεύτερο βασικό πυλώνα του συστήματος, πλάι στην υποδομή τεχνητής νοημοσύνης μοντέλων βαθιάς μάθησης που αποτελεί τον πρώτο βασικό πυλώνα, ενώ ο εξυπηρετητής Flask αποτελεί τον συνδετικό κρίκο μεταξύ των δύο. Αναπτύχθηκε με στόχο να προσφέρει στους χρήστες με οπτική αναπηρία τη δυνατότητα να αναγνωρίζουν και δέχονται έγκυρες περιγραφές σε ενδύματα και αντικείμενα μόδας, μέσω του κινητού τους τηλεφώνου. Η λειτουργικότητα επικεντρώνεται στην άμεση, απλή και προσβάσιμη χρήση, μέσω αφής, ηχητικών εφέ και φωνητικής ανάγνωσης (Text-To-Speech). Η εφαρμογή υποστηρίζει δύο βασικές εισόδους εικόνων: είτε μέσω λήψης φωτογραφίας με την κάμερα, είτε μέσω επιλογής εικόνας από τη συλλογή. Καθ' όλη τη χρήση, ο χρήστης καθοδηγείται με ήχους και φωνητικές ανακοινώσεις.

Η εφαρμογή αποτελείται από πέντε βασικά Τμήματα (Fragments):

- CameraFragment.kt
- ConfirmFragment.kt
- LoadingFragment.kt
- ResultsFragment.kt
- Και συμπληρωματικά το PlaceholderFragment

Υπάρχει επίσης η κεντρική δραστηριότητα (MainActivity.kt) η οποία διαχειρίζεται τη ροή πλοήγησης και στην οποία φορτώνονται κάθε φορά καθένα από τα παραπάνω Fragments.

Επίσης τα Fragments αυτά κάνουν χρήση των Kotlin αρχείων/κλάσεων:

- GalleryNavigator.kt (από τα CameraFragment και ConfirmFragment)
- NetworkUtils.kt (από το LoadingFragment).

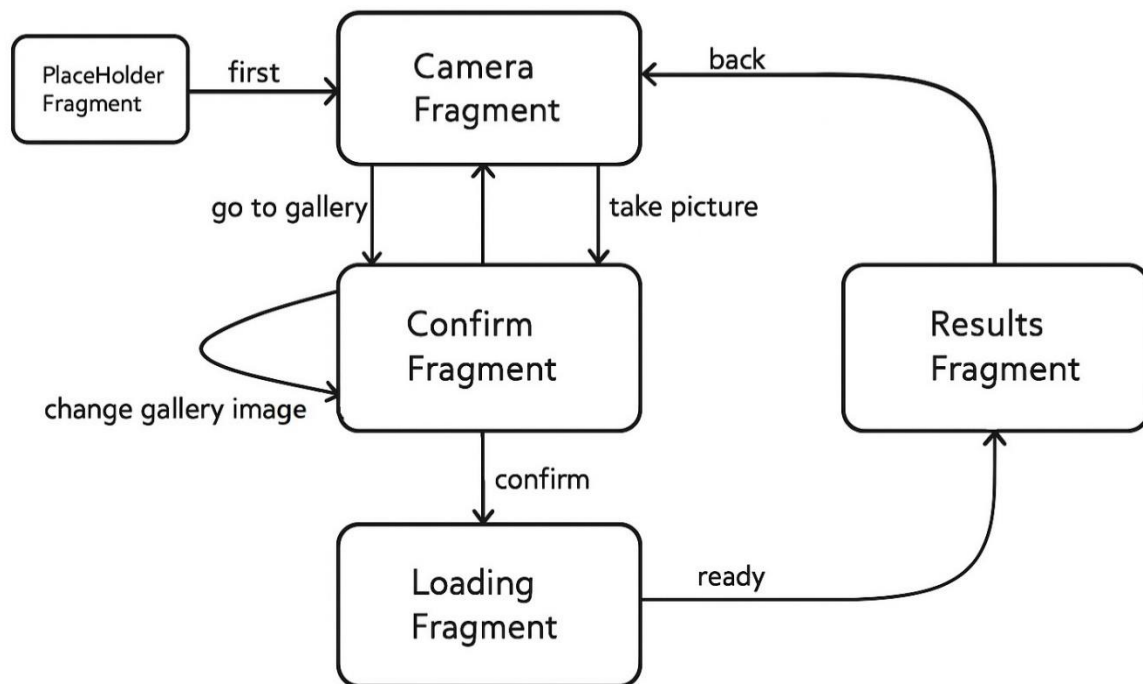
Ολόκληρη η εφαρμογή έχει σχεδιαστεί με γνώμονα την εργονομία και την ευχρηστία για άτομα χωρίς οπτική πρόσβαση. Κάθε οθόνη αντιστοιχεί σε κυρίως ένα βήμα της διαδικασίας: λήψη ή επιλογή εικόνας, επιβεβαίωση εικόνας και πλοήγηση στις εικόνες της συλλογής του κινητού (χρησιμοποιείται για δύο λειτουργίες), αποστολή στον διακομιστή, και παρουσίαση αποτελεσμάτων.

Οι αλληλεπιδράσεις γίνονται αποκλειστικά με απλές χειρονομίες αφής, που αναλύονται παρακάτω. Οι χρήστες δεν απαιτείται να διαβάζουν ή να βλέπουν την οθόνη. Με το άγγιγμα σε κάθε στοιχείο, ενεργοποιούνται ηχητικά εφέ και συνθετική ομιλία που ανακοινώνει τα απαραίτητα.

5.2 Αρχιτεκτονική Εφαρμογής και Ανάλυση Μερών

Η εφαρμογή βασίζεται σε modular τμηματική δομή με χρήση Android Τμήματα (Fragments), που χωρίζουν τις διαφορετικές λειτουργίες σε αυτόνομα, επαναχρησιμοποιήσιμα κομμάτια. Τα fragments επικοινωνούν και αλληλοεπιδρούν μέσω μιας κλειστής αρχιτεκτονικής, όπου δημιουργούνται κύκλοι που μπορεί να ακολουθήσει ένας χρήστης μεταξύ των fragments, με αποτέλεσμα την πλήρη αποφυγή σφαλμάτων και λανθασμένων κλήσεων. Η εφαρμογή είναι σχεδιασμένη με τέτοιο τρόπο έτσι ώστε όλα τα fragments και η μετάβαση μεταξύ τους, εν τέλει να οδηγεί στο αρχικό και κεντρικό

CameraFragment, φωτωμένο στο MainActivity. Κάθε fragment έχει συγκεκριμένο ρόλο και φροντίζει για τη διαχείριση συγκεκριμένης διεπαφής και λειτουργικότητας.



Σχήμα 9. Σχηματική Αναπαράσταση Αρχιτεκτονικής της Android Εφαρμογής

MainActivity:

Το MainActivity αποτελεί τον βασικό «σκληρό δίσκο» της εφαρμογής, τον κεντρικό κόμβο διαχείρισης και ελέγχου των διαφορετικών τμημάτων (fragments) και των βασικών ροών πλοήγησης. Μέσα στο MainActivity υλοποιείται η αρχικοποίηση των διάφορων fragments, η διαχείριση των μεταβάσεων μεταξύ τους, καθώς και η δέσμευση των χειρονομιών αφής σε συνδυασμό με τη συνάρτηση του Text-To-Speech (TTS), ώστε να παρέχεται ομαλή εμπειρία χρήσης. Επίσης, το MainActivity αναλαμβάνει την αρχικοποίηση του Text-To-Speech engine, τη σωστή διαχείριση του κύκλου ζωής του και τη ρύθμιση παραμέτρων όπως η γλώσσα και ο τόνος της φωνής. Παράλληλα, ελέγχει τα δικαιώματα πρόσβασης που απαιτούνται (π.χ. κάμερα, αποθήκευση), και υλοποιεί το βασικό UI container που φιλοξενεί τα fragments, διασφαλίζοντας ότι η εναλλαγή γίνεται ομαλά και χωρίς καθυστερήσεις. Μέσω του MainActivity επιτυγχάνεται η κεντρική ενοποίηση των διαφορετικών λειτουργικών μονάδων της εφαρμογής, διατηρώντας παράλληλα τη διακριτότητα, ακεραιότητα και την ανεξαρτησία τους.

CameraFragment:

Το CameraFragment αποτελεί την αρχική οθόνη της εφαρμογής, όπου ο χρήστης μπορεί να τραβήξει φωτογραφίες των ρούχων. Το fragment αξιοποιεί το Camera API του Android, και έχει σχεδιαστεί ώστε να επιτρέπει απλή και εύκολη χρήση μέσω αφής, με κύριο στόχο την ελαχιστοποίηση των δυσκολιών που μπορεί να αντιμετωπίσουν οι χρήστες με οπτική αναπηρία. Η διεπαφή περιλαμβάνει σχεδόν όλη την οθόνη ως περιοχή αφής για να διευκολύνει την αλληλεπίδραση και περιέχει και ένα βοηθητικό σχήμα στο κέντρο της οθόνης κάμερας (σε περίπτωση μη οπτικά ανάπηρου χρήστη). Σε αυτή τη φάση, το fragment αναλαμβάνει να αποθηκεύσει την εικόνα τοπικά σε προσωρινή μνήμη και να ενημερώσει το σύστημα για την ύπαρξη νέας φωτογραφίας προς αναγνώριση.

GalleryNavigator:

Το GalleryNavigator είναι ένα βοηθητικό fragment που εξυπηρετεί την αναζήτηση και περιήγηση των εικόνων που έχει ήδη ο χρήστης και περιέχονται στη συλλογή της κινητής συσκευής του. Προσφέρει τη δυνατότητα χειρονομιών συρσίματος (swipe gestures) για να μετακινείται ο χρήστης ανάμεσα στις αποθηκευμένες φωτογραφίες, ταξινομημένες χρονολογικά, ώστε ο χρήστης να μπορεί να επιλέξει την επιθυμητή εικόνα προς ανάλυση χωρίς οπτική βοήθεια. Το fragment χειρίζεται το φόρτωμα και αποθήκευση εικόνων σε προσωρινή μνήμη με ασύγχρονο τρόπο για να διατηρεί ομαλή την εμπειρία χρήσης και ενημερώνει το ConfirmFragment και άλλα τμήματα για την εικόνα που επιλέχθηκε.

ConfirmFragment:

Σε αυτό το fragment παρουσιάζεται η εικόνα που έχει επιλεγεί είτε από την κάμερα είτε από την συλλογή (gallery). Στόχος είναι ο χρήστης να επιβεβαιώσει την εικόνα πριν την αποστολή της προς ανάλυση. Το fragment δίνει τη δυνατότητα επιβεβαίωσης μέσω απλού πατήματος και επιστροφής πίσω στην κάμερα μέσω παρατεταμένου πατήματος, ενώ υποστηρίζει χειρονομίες συρσίματος (swipe gestures), μέσω της χρήσης του GalleryNavigator, για πλοήγηση σε άλλες εικόνες της συλλογής. Σε συνδυασμό με αναπαραγωγή ηχητικών εφέ, εξασφαλίζεται ότι ο χρήστης ενημερώνεται πλήρως για την τρέχουσα επιλογή και τις ενέργειες που μπορεί να πραγματοποιήσει.

LoadingFragment:

Αυτό το fragment λειτουργεί ως οθόνη αναμονής κατά τη διάρκεια που η εφαρμογή αποστέλλει την εικόνα στον πίσω διακομιστή (backend server) και περιμένει το αποτέλεσμα. Η συγκεκριμένη οθόνη παρουσιάζει μια σταθερή κατάσταση που αποτρέπει αλληλεπιδράσεις μέχρι να ληφθεί η απάντηση. Με τον τρόπο αυτό αποφεύγονται παρεμβολές, παρεξηγήσεις ή πολλαπλές αποστολές.

ResultsFragment:

Μετά την επιτυχή λήψη του αποτελέσματος από τον διακομιστή (server), το ResultsFragment αναλαμβάνει την μετατροπή του map με τα αποτελέσματα που δέχεται σε παρουσιάσιμη μορφή και εν τέλει την παρουσίαση των χαρακτηριστικών του ρεύχου που ταξινομήθηκαν από τα μοντέλα. Η πληροφόρηση δίνεται τόσο σε μορφή κειμένου, όσο και μέσω Text-To-Speech (TTS) τεχνολογίας, ώστε ο χρήστης να μπορεί να κατανοήσει πλήρως τα αποτελέσματα. Τα στοιχεία που εμφανίζονται περιλαμβάνουν τύπο ενδύματος, χρώμα, εποχή, φύλο χρήσης και άλλα σημαντικά χαρακτηριστικά. Επίσης, το fragment υποστηρίζει απλές χειρονομίες για επιστροφή σε προηγούμενα στάδια.

```
val formattedResult = formatPredictions(resultJson)
binding.textViewResults.text = formattedResult

(activity as? MainActivity)?.speak(formattedResult)
```

Εικόνα 8. Κομμάτι κώδικα ανακοίνωσης αποτελεσμάτων από το ResultsFragment.kt

PlaceholderFragment

Το PlaceholderFragment χρησιμοποιείται κυρίως ως ένα προσωρινό fragment που λειτουργεί ως σημείο ενδιάμεσης φόρτωσης ή ως προεπιλεγμένη οθόνη σε περιπτώσεις όπου δεν υπάρχει ενεργή επιλογή ή δεδομένο προς εμφάνιση. Ο ρόλος του είναι να παρέχει μια κενή, ουδέτερη διεπαφή ή ένα μήνυμα αναμονής προς τον χρήστη, ώστε να μην εμφανίζεται κενό ή ασταθές UI. Η χρήση του γίνεται κατά την πρώτη έναρξη της

εφαρμογής στη κινητή συσκευή και υπάρχει στην οθόνη μέχρι να γίνουν δεχτές οι άδειες για την πρόσβαση στην κάμερα και τη συλλογή της κινητής συσκευής. Μετά από αυτό φορτώνεται το CameraFragment και δεν ξαναχρησιμοποιείται ξανά το PlaceholderFragment και αρχική οθόνη είναι πάντα το CameraFragment.

5.3 Διασύνδεση με τον Διακομιστή (Server), Διαχείριση Εικόνων και Αποτελεσμάτων

NetworkUtils

Το NetworkUtils είναι ένα singleton αντικείμενο υπεύθυνο για την επικοινωνία της εφαρμογής με το backend Flask server. Χρησιμοποιεί το OkHttpClient, μια αξιόπιστη και ευρέως διαδεδομένη βιβλιοθήκη HTTP, με ρυθμίσεις που επιτρέπουν timeout για σύνδεση, ανάγνωση και εγγραφή, ώστε να αποφεύγονται κολλήματα ή υπερβολική αναμονή. Η βασική λειτουργία είναι η ασύγχρονη αποστολή μιας εικόνας σε μορφή multipart/form-data μέσω POST request προς το URL του server, με παράλληλη διαχείριση των αποκρίσεων μέσω callback. Όταν ο server απαντήσει, το NetworkUtils αναλύει το JSON αποτέλεσμα και το μετατρέπει σε δομή map, ώστε η εφαρμογή να το χρησιμοποιήσει εύκολα. Σε περίπτωση σφάλματος δικτύου, ο χρήστης ειδοποιείται μέσω Toast μηνύματος και η κλήση callback επιστρέφει κενό αποτέλεσμα.

```
object NetworkUtils {  
    private val client = OkHttpClient.Builder()  
        .connectTimeout( timeout: 30, java.util.concurrent.TimeUnit.SECONDS)  
        .writeTimeout( timeout: 30, java.util.concurrent.TimeUnit.SECONDS)  
        .readTimeout( timeout: 60, java.util.concurrent.TimeUnit.SECONDS)  
        .build()  
}
```

Εικόνα 9. Κομμάτι κώδικα έναρξης OkHttpClient από το NetworkUtils.kt

AndroidManifest

Το AndroidManifest.xml περιέχει όλες τις απαραίτητες δηλώσεις αδειών και χαρακτηριστικών της εφαρμογής. Εκτός από βασικές άδειες όπως η κάμερα και το διαδίκτυο, η εφαρμογή υποστηρίζει και τις νεότερες άδειες που απαιτούνται για πρόσβαση σε πολυμέσα σε Android 13 και άνω (όπως READ_MEDIA_IMAGES). Παράλληλα, διαχειρίζεται την backward compatibility με τις άδειες αποθήκευσης (READ_EXTERNAL_STORAGE και WRITE_EXTERNAL_STORAGE) για παλιότερες εκδόσεις Android. Το manifest επίσης περιλαμβάνει ρυθμίσεις για το network security config, ώστε να επιτρέπεται η επικοινωνία με τον τοπικό server μέσω HTTP. Τέλος, ορίζει ότι το MainActivity είναι το κεντρικό σημείο εισόδου της εφαρμογής με την κατάλληλη διαχείριση αλλαγών στην οθόνη (orientation, screen size, keyboard).

```
<uses-permission android:name="android.permission.CAMERA" />  
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />  
...  
<uses-feature android:name="android.hardware.camera" android:required="false" />  
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" android:maxSdkVersion="32" />  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" android:maxSdkVersion="32" />
```

Εικόνα 10. Κομμάτι κώδικα συμπλήρωσης αδειών από το AndroidManifest.xml

Οι εικόνες στέλνονται στον server και τα αποτελέσματα γίνονται δεκτά από αυτόν, μέσω του NetworkUtils και ύστερα τα αποτελέσματα δέχονται επεξεργασία μέχρι την τελική παρουσίαση, μέσα στο ResultsFragment, μέσα από μετατροπή του map με τα αποτελέσματα σε παρουσιάσιμη μορφή.

5.4 Χειρισμός από Άτομα με Οπτική Αναπηρία

Η αλληλεπίδραση χρήστη με την εφαρμογή είναι σχεδιασμένη ώστε να στηρίζεται αποκλειστικά σε απλές και κατανοητές χειρονομίες αφής (gestures), σε συνδυασμό με φωνητική υποστήριξη (TTS) και ηχητικά εφέ που βελτιώνουν την εμπειρία χρήσης, ιδιαίτερα για άτομα με οπτική αναπηρία.

Χειρονομίες αφής (gestures):

Οι χειρονομίες που υποστηρίζονται περιλαμβάνουν το απλό πάτημα (tap), το παρατεταμένο πάτημα (long tap/press), το σύρσιμο (swipe) προς αριστερά ή δεξιά και το διπλό πάτημα (double tap). Κάθε χειρονομία (gesture) έχει σαφή και συγκεκριμένη λειτουργία, καθώς και γίνεται ανίχνευση των χειρονομιών αυτών αφής μέσα στα προαναφερθέντα fragments. Με το απλό πάτημα γίνεται τράβηγμα φωτογραφίας και επιβεβαίωση, με το διπλό γίνεται εναλλαγή κάμερας, με τα swipes περιήγηση στις εικόνες συλλογής και με το παρατεταμένο πάτημα γίνεται καθαρισμός του fragment stack και επιστρέφει η οθόνη πάντα στο αρχικό fragment/camera fragment. Επίσης είναι σχεδιασμένα για να είναι όσο πιο απλά και κατανοητά γίνεται, έτσι ώστε να απομνημονευτούν εύκολα από τους οπτικά ανάπηρους χρήστες, χωρίς ιδιαίτερες δυσκολίες.

```
override fun onFling(e1: MotionEvent?, e2: MotionEvent, velocityX: Float, velocityY: Float): Boolean {
    (activity as? MainActivity)?.stopSpeaking()
    if (e1 == null) return false
    val deltaX = e2.x - e1.x
    if (abs(deltaX) > SWIPE_THRESHOLD && abs(velocityX) > SWIPE_VELOCITY_THRESHOLD) {
        if (deltaX < 0) {
            openMostRecentGalleryImage()
            return true
        }
    }
    return false
}
```

Εικόνα 11. Κομμάτι κώδικα διαχείρισης συρσίματος (swipe-fling) από το CameraFragment.kt

Ηχητικά εφέ: Η χρήση ηχητικών εφέ, όπως το ηχητικό εφέ «αλλαγής σελίδας» κατά την αλλαγή εικόνας με σύρσιμο (swipe), παρέχει επιπρόσθετη αισθητηριακή επιβεβαίωση της ενέργειας. Τα ηχητικά εφέ που περιέχει η εφαρμογή είναι τα εξής:

- Ήχος κλείστρου κάμερας: Κατά την λήψη φωτογραφίας
- Ήχος αλλαγής σελίδας: Κατά την μετάβαση στις εικόνες της συλλογής και κατά την πλοήγηση μεταξύ αυτών, μέσω συρσιμάτων (swipes)
- Ήχος ακύρωσης: Κατά το παρατεταμένο/μακρύ πάτημα για μετάβαση στην αρχική οθόνη/οθόνη κάμερας.
- Ήχος επιβεβαίωσης: Κατά την επιβεβαίωση-έγκριση εικόνας προς αναγνώριση.
- Ήχος εναλλαγής διακόπτη: Κατά την εναλλαγή μεταξύ μπροστινής και πίσω κάμερας


```
private fun playSwipeSound() {
    MediaPlayer.create(requireContext(), R.raw.pageturn).apply {
        setOnCompletionListener { it.release() }
        start()
    }
}
```

Εικόνα 12. Κομμάτι κώδικα διαχείρισης αναπαραγωγής ήχου αλλαγής εικόνας από το ConfirmFragment.kt

Τα εφέ αυτά είναι σύντομα, διακριτικά, εύκολα κατανοητά, αναγνωρίσιμα και σχεδιασμένα να μη διακόπτουν το ροή των φωνητικών ανακοινώσεων. Προσδίδουν μια αίσθηση φυσικότητας και ανατροφοδότησης, κάνοντας την πλοήγηση πιο ευχάριστη και διαδραστική. Επίσης, βοηθούν στην ελαχιστοποίηση λαθών κατά την επιλογή εικόνας, προσφέροντας άμεση επιβεβαίωση.

Συνδυασμός με Text-To-Speech (TTS):

Το Text-To-Speech λειτουργεί παράλληλα με τις χειρονομίες και τα ηχητικά εφέ, ώστε να παρέχει πλούσια φωνητική πληροφόρηση για την κατάσταση της εφαρμογής και τα αποτελέσματα. Το TTS ενημερώνει το χρήστη ότι η κάμερα είναι ήδη ενεργή, αν ο χρήστης βρίσκεται ήδη στην οθόνη κάμερα και κάνει απόπειρα παρατεταμένου πατήματος για να πάει σε αυτή. Επίσης ανακοινώνει τα αποτελέσματα των προβλέψεων στο χρήστη όταν είναι έτοιμα και βρίσκονται στην οθόνη αποτελεσμάτων. Η φωνητική καθοδήγηση υποστηρίζει τους χρήστες, διασφαλίζοντας προσβασιμότητα. Η σωστή συγχρονισμένη χρήση TTS και ηχητικών εφέ βελτιώνει σημαντικά την εμπειρία χρήσης, ελαχιστοποιώντας την ανάγκη για οπτική παρακολούθηση.

```
fun speak(text: String) {
    if (isTtsReady) {
        tts.speak(text, TextToSpeech.QUEUE_FLUSH, params: null, utterancelid: null)
    }
}
```

Εικόνα 13. Κομμάτι κώδικα διαχείρισης αναπαραγωγής φωνητικής ομιλίας (Text-To-Speech / TTS) από το MainActivity.kt

Συνολική εμπειρία χρήστη: Ο συνδυασμός αυτών των μηχανισμών καθιστά την εφαρμογή προσβάσιμη, φιλική προς το χρήστη και αποδοτική. Η πολυαισθητηριακή προσέγγιση καλύπτει τις ανάγκες ατόμων με οπτική αναπηρία, εξασφαλίζοντας ότι οι ενέργειες και τα αποτελέσματα είναι πάντα κατανοητά και προσβάσιμα με ελάχιστη προσπάθεια, ενώ παράλληλα η δομή αυτή επιτρέπει και σε χρήστες χωρίς ειδικές ανάγκες να έχουν γρήγορη και απλή χρήση.

5.5 Εγχειρίδιο Χρήστη και Περιγραφή Ροής Χρήστη

Εγχειρίδιο Χρήστη

Η εφαρμογή υποστηρίζει τις ακόλουθες λειτουργίες:

- Λήψη Φωτογραφίας:
 - Άνοιγμα εφαρμογής και επομένως της κάμερας, που αποτελεί την αρχική οθόνη της εφαρμογής.



Εικόνα 14. Οθόνη Κάμερας/Αρχική Οθόνη

- Διπλό πάτημα για εναλλαγή μεταξύ μπροστινής και πίσω κάμερας (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια)
- Απλό πάτημα για φωτογράφιση (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Μετάβαση στην οθόνη Επιβεβαίωσης, στην οποία παρουσιάζεται η τραβηγμένη φωτογραφία.



Εικόνα 15. Οθόνη Επιβεβαίωσης

- Επιβεβαίωση εικόνας στην οθόνη Επιβεβαίωσης, με απλό πάτημα (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Ακύρωση εικόνας και επιστροφή στην αρχική οθόνη-κάμερα, από την οθόνη Επιβεβαίωσης, με παρατεταμένο/μακρύ πάτημα (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Επιλογή εικόνας από Συλλογή κινητής συσκευής:
 - Σύρσιμο (Swipe) προς τα αριστερά, ενώ ο χρήστης βρίσκεται στην αρχική οθόνη/οθόνη κάμερας.
 - Εμφάνιση της πιο πρόσφατης εικόνας της συλλογής του κινητού, μέσα σε μια Οθόνη επιβεβαίωσης (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια, με το κάθε σύρσιμο-swipe).



Εικόνα 16. Οθόνη Επιβεβαίωσης για Εικόνες Συλλογής Συσκευής

- Πλοήγηση μεταξύ των πρόσφατων εικόνων της συλλογής με συρσίματα (swipe) δεξιά/αριστερά. Η καθεμία φορτώνεται σε μια οθόνη επιβεβαίωσης κάθε φορά. Σύρσιμο (Swipe) αριστερά για πλοήγηση σε λιγότερο πρόσφατη εικόνα της συλλογής και σύρσιμο (swipe) δεξιά για πλοήγηση σε πιο πρόσφατη εικόνα της συλλογής (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια, με το κάθε σύρσιμο-swipe).
- Επιβεβαίωση εικόνας στην οθόνη Επιβεβαίωσης (στην προκειμένη περίπτωση πάνω από κάθε παρούσα εικόνα συλλογής), με απλό πάτημα (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Ακύρωση εικόνας και επιστροφή στην αρχική οθόνη-κάμερα, από την οθόνη Επιβεβαίωσης (στην προκειμένη περίπτωση πάνω από κάθε παρούσα εικόνα συλλογής), με παρατεταμένο/μακρύ πάτημα (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Αποστολή και Αναγνώριση:
 - Μετά την επιβεβαίωση, η εικόνα αποστέλλεται στον διακομιστή (server).
 - Φορτώνεται στο εν τω μεταξύ η οθόνη Αναμονής, που ενημερώνει για την αναμονή.



Εικόνα 17. Οθόνη Αναμονής

- Όταν η κινητή εφαρμογή δεχθεί τα αποτελέσματα από τον διακομιστή (server), φορτώνεται η οθόνη αποτελεσμάτων
- Ανακοίνωση Αποτελεσμάτων:
 - Στην Οθόνη Αποτελεσμάτων παρουσιάζονται όλα τα αποτελέσματα μέσα σε ένα κεντρικό κουτί, με αχνή παρουσίαση της αναλυμένης εικόνας από πίσω.



Εικόνα 18. Οθόνη Αποτελεσμάτων σε Φωτογραφία



Εικόνα 19. Οθόνη Αποτελεσμάτων σε Εικόνα Συλλογής

- Όλα τα αποτελέσματα διαβάζονται φωνητικά.
- Διακοπή ανακοίνωσης και επιστροφή στην αρχική οθόνη-κάμερα, από την οθόνη Αποτελεσμάτων, με παρατεταμένο/μακρύ πάτημα (ακούγεται αντίστοιχος χαρακτηριστικός ήχος κατά την ενέργεια).
- Επιπλέον Διευκρινίσεις:
 - Κατά την πρώτη χρήση πρέπει να γίνουν αποδεκτές οι άδειες για πρόσβαση της εφαρμογής στην κάμερα και στη συλλογή της κινητής συσκευής. Μέχρι να γίνει επιβεβαίωση, φορτώνεται μια οθόνη αναμονής (Place Holder) και μόλις γίνει η επιβεβαίωση, φορτώνεται κανονικά η οθόνη κάμερας-αρχική οθόνη και δεν χρειάζεται να ξαναγίνει έγκριση αδειών από δω και πέρα, κατά το άνοιγμα της εφαρμογής.
 - Σε περίπτωση που ο χρήστης εκτελέσει παρατεταμένο/μακρύ πάτημα επιστροφής ενώ ήδη βρίσκεται στη αρχική οθόνη-οθόνη κάμερας, ανακοινώνεται φωνητικά ότι η κάμερα είναι ενεργή.

Περιγραφή Ροής Χρήστη

Σενάριο 1 – Επιλογή εικόνας από τη Συλλογή:

- Ο χρήστης ξεκινά από την Οθόνη Κάμερας/Αρχική Οθόνη, την αρχική οθόνη της εφαρμογής.
- Κάνει παρατεταμένο πάτημα και του ανακοινώνεται φωνητικά ότι η κάμερα είναι ήδη ενεργή.
- Κάνει σύρσιμο (swipe) προς τα αριστερά για να μεταβεί σε λειτουργία περιήγησης στη συλλογή και αναπαράγεται ήχος πλοήγησης.
- Φορτώνεται η Οθόνη Επιβεβαίωσης με την πιο πρόσφατη εικόνα από τη συλλογή.
- Ο χρήστης κάνει σύρσιμο (swipe) αριστερά για να δει την αμέσως παλαιότερη εικόνα και μετά ξανά σύρσιμο (swipe) δεξιά για να δει αμέσως νεότερη εικόνα (άρα την αρχική), όπου η κάθε εικόνα παρουσιάζεται μέσα σε μια Οθόνη Επιβεβαίωσης κάθε φορά. Σε κάθε αλλαγή εικόνας αναπαράγεται ήχος πλοήγησης.

- Με απλό πάτημα επιβεβαιώνει την εικόνα στη οποία βρίσκεται, στην προκειμένη περίπτωση μια εικόνα ενός τζιν, και αναπαράγεται ήχος επιβεβαίωσης.
- Φορτώνεται η Οθόνη Αναμονής και η εικόνα αποστέλλεται στον server.
- Μετά την επεξεργασία, η εφαρμογή μεταβαίνει στην Οθόνη Αποτελεσμάτων, όπου παρουσιάζονται τα αποτελέσματα με το τζιν να αχνοφαίνεται στο φόντο και γίνονται φωνητικά ανακοινώσεις των αποτελεσμάτων, αναφέροντας ότι το ρούχο είναι ένδυμα, τζιν, καθώς και τα υπόλοιπα αναγνωρισμένα χαρακτηριστικά.

Σενάριο 2 – Φωτογράφιση με την Κάμερα:

- Ο χρήστης ξεκινά από την Οθόνη Κάμερας/Αρχική Οθόνη, την αρχική οθόνη της εφαρμογής.
- Κάνει διπλό πάτημα αν θέλει να αλλάξει κάμερα (μπροστινή/πίσω) και αναπαράγεται ήχος αλλαγής κάμερας.
- Με απλό πάτημα τραβάει φωτογραφία. Αναπαράγεται ήχος φωτογράφισης.
- Φορτώνεται η Οθόνη Επιβεβαίωσης με την φωτογραφία που μόλις τραβήχτηκε
- Πιστεύει ότι δεν έβγαλε σωστή φωτογραφία και με παρατεταμένο/μακρύ πάτημα αναπαράγεται ήχος ακύρωση–επιστροφής και επιστρέφει στην Οθόνη Κάμερας/Αρχική Οθόνη.
- Με απλό πάτημα τραβάει καινούργια φωτογραφία. Αναπαράγεται ήχος φωτογράφισης.
- Φορτώνεται η Οθόνη Επιβεβαίωσης με την φωτογραφία που μόλις τραβήχτηκε.
- Με απλό πάτημα επιβεβαιώνει την εικόνα, στην προκειμένη περίπτωση μια φωτογραφία ενός κοντομάνικου, και αναπαράγεται ήχος επιβεβαίωσης.
- Φορτώνεται η Οθόνη Αναμονής και η εικόνα αποστέλλεται στον server.
- Μετά την επεξεργασία, η εφαρμογή μεταβαίνει στην Οθόνη Αποτελεσμάτων, όπου παρουσιάζονται τα αποτελέσματα με το κοντομάνικο να αχνοφαίνεται στο φόντο και γίνονται φωνητικά ανακοινώσεις των αποτελεσμάτων, αναφέροντας ότι το ρούχο είναι ένδυμα, κοντομάνικο, καθώς και τα υπόλοιπα αναγνωρισμένα χαρακτηριστικά.
- Με παρατεταμένο/μακρύ πάτημα, επιστρέφει στην Οθόνη Κάμερας/Αρχική Οθόνη και η εφαρμογή είναι έτοιμη για νέα χρήση.

6. ΠΕΙΡΑΜΑΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΣΥΣΤΗΜΑΤΟΣ

Στο παρόν κεφάλαιο παρουσιάζονται τα πειραματικά αποτελέσματα που προέκυψαν από την αξιολόγηση των μοντέλων βαθιάς μάθησης, καθώς και η διαδικασία της τελικής επιλογής των μοντέλων βαθιάς μάθησης που επιλέχθηκαν τελικά. Γίνεται ανάλυση των επιδόσεων σε μετρικές όπως ακρίβεια (accuracy), ανάκληση (recall), ακρίβεια (precision), F1-score και χρόνος πρόβλεψης. Αρχικά, αξιολογούνται συγκριτικά όλα τα μοντέλα που εκπαιδεύτηκαν ανά χαρακτηριστικό, με σκοπό την επιλογή των καταλληλότερων για χρήση στο τελικό σύστημα. Στη συνέχεια, εξετάζεται η λειτουργία της εφαρμογής σε πραγματικές συνθήκες χρήσης, με έμφαση στην αποτελεσματικότητα των προβλέψεων και τη γενική εμπειρία του χρήστη.

6.1 Αναλυτική Αξιολόγηση και Επιλογή Τελικών Αλγορίθμων Ταξινόμησης

Πολύ-Ετικετικά (Multi-Task) Μοντέλα:

EfficientNetB3 Fashion 1 και InceptionV3 Fashion 1 (Πρώτες Εκδόσεις):

Οι πρώτες εκδόσεις των μοντέλων με τα ονόματα EfficientNetB3_Fashion_1 και InceptionV3_Fashion_1, δεν προχώρησαν σε επίσημη αξιολόγηση, καθώς εμφάνιζαν πολύ χαμηλές επιδόσεις τόσο στη ακρίβεια εκπαίδευσης (training), όσο και στην ακρίβεια επικύρωσης (validation accuracy). Συγκεκριμένα, η ακρίβεια στο χαρακτηριστικό articleType δεν ξεπερνούσε το 70%, ενώ στα season και baseColour οι επιδόσεις ήταν ακόμα πιο χαμηλές (60% και 50% αντίστοιχα). Το InceptionV3 παρουσίαζε ακόμη χειρότερη συμπεριφορά σε σχέση με το EfficientNet, με βραδύτερη εκμάθηση και μεγαλύτερη αστάθεια. Οι επιδόσεις αυτές κρίθηκαν ανεπαρκείς για πρακτική χρήση, οδηγώντας άμεσα στη σχεδίαση και υλοποίηση των επόμενων εκδόσεων των μοντέλων, οι οποίες περιλάμβαναν ουσιαστικές βελτιώσεις στη διαδικασία εκπαίδευσης και στη διαχείριση των δεδομένων.

EfficientNetB3 Fashion 2/3 και InceptionV3 Fashion 2/3 (Δεύτερες/Τρίτες Εκδόσεις):

Μετά τις βελτιώσεις και αλλαγές που περιεγράφηκαν αναλυτικά στο υπο-κεφάλαιο 3.5, εφαρμόστηκε πλήρης αξιολόγηση σε όλα τα μοντέλα και των δύο εκδόσεων και των δύο τύπων, τόσο πάνω στα καλύτερα αποθηκευμένα βάρη τους (best weights), όσο και στα τελικά αποθηκευμένα βάρη τους (final weights). Για κάθε μοντέλο υπήρξε πλήρης ανάλυση με μετρικά και πίνακες σύγχυσης (confusion matrices) για κάθε χαρακτηριστικό-ετικέτα. Παρακάτω παραθέτονται τα σημαντικότερα εκ αυτών:

EfficientNetB3 Πολύ-Ετικετικά (Multi-Task) Μοντέλα-Αποτελέσματα Αξιολόγησης (Δεύτερες και Τρίτες Εκδόσεις):

Attribute	Metric	MODELS			
		EfficientNetB3_Fashion_2 (final weights)	EfficientNetB3_Fashion_2 (best weights)	EfficientNetB3_Fashion_3 (best weights)	EfficientNetB3_Fashion_3 (final weights)
Inference	Time/sample	0.005958 s	0.005956 s	0.010635 s	0.005961 s
masterCategory	Accuracy	0.9941	0.9941	0.9932	0.9932
	P / R / F1 (macro)	0.7456 / 0.7454 / 0.7455	0.7456 / 0.7454 / 0.7455	0.7448 / 0.7451 / 0.7449	0.7448 / 0.7451 / 0.7449
subCategory	Accuracy	0.9484	0.9484	0.9501	0.9501
	P / R / F1 (macro)	0.8328 / 0.8000 / 0.8136	0.8328 / 0.8000 / 0.8136	0.8111 / 0.7865 / 0.7969	0.8111 / 0.7865 / 0.7969
articleType	Accuracy	0.8451	0.8457	0.8530	0.8530
	P / R / F1 (macro)	0.7159 / 0.6755 / 0.6840	0.7165 / 0.6762 / 0.6846	0.6218 / 0.5914 / 0.5942	0.6218 / 0.5914 / 0.5942
baseColour	Accuracy	0.5657	0.5660	0.7458	0.7458
	P / R / F1	0.4163 / 0.3689 / 0.3805	0.4171 / 0.3691 / 0.3807	0.5928 / 0.5390 / 0.5517	0.5928 / 0.5390 / 0.5517
gender	Accuracy	0.8967	0.8964	0.9094	0.9094
	P / R / F1 (macro)	0.7932 / 0.7014 / 0.7405	0.7932 / 0.7014 / 0.7405	0.8130 / 0.7019 / 0.7459	0.8130 / 0.7019 / 0.7459
season	Accuracy	0.6919	0.6919	0.7063	0.7063
	P / R / F1 (macro)	0.5257 / 0.4672 / 0.4838	0.5273 / 0.4658 / 0.4827	0.5203 / 0.5017 / 0.5088	0.5203 / 0.5017 / 0.5088
usage	Accuracy	0.8973	0.8976	0.9089	0.9089

Attribute	Metric	MODELS			
		EfficientNetB3_Fashion_2 (final weights)	EfficientNetB3_Fashion_2 (best weights)	EfficientNetB3_Fashion_3 (best weights)	EfficientNetB3_Fashion_3 (final weights)
	P / R / F1 (macro)	0.4190 / 0.3950 / 0.4055	0.4191 / 0.3953 / 0.4058	0.4282 / 0.4063 / 0.4163	0.4282 / 0.4063 / 0.4163

Πίνακας 1. Αποδόσεις EfficientNetB3 Πολύ-ετικετικών (Multi-Task) μοντέλων

InceptionV3 Πολύ-Ετικετικά (Multi-Task) Μοντέλα–Αποτελέσματα Αξιολόγησης (Δεύτερες και Τρίτες Εκδόσεις):

Attribute	Metric	MODELS			
		InceptionV3_Fashion_2 (best weights)	InceptionV3_Fashion_2 (final weights)	InceptionV3_Fashion_3 (best weights)	InceptionV3_Fashion_3 (best weights)
Inference	Time/sample	0.012922 s	0.005772 s	0.008728 s	0.005793 s
masterCategory	Accuracy	0.9870	0.9870	0.9935	0.9935
	P / R / F1 (macro)	0.7416 / 0.7403 / 0.7409	0.7416 / 0.7403 / 0.7409	0.7453 / 0.7453 / 0.7453	0.7453 / 0.7453 / 0.7453
subCategory	Accuracy	0.9312	0.9312	0.9565	0.9565
	P / R / F1 (macro)	0.8341 / 0.7749 / 0.7971	0.8341 / 0.7749 / 0.7971	0.8063 / 0.7854 / 0.7951	0.8063 / 0.7854 / 0.7951
articleType	Accuracy	0.8172	0.8172	0.8493	0.8493
	P / R / F1 (macro)	0.6402 / 0.5874 / 0.5963	0.6402 / 0.5874 / 0.5963	0.7018 / 0.6724 / 0.6728	0.7018 / 0.6724 / 0.6728
baseColour	Accuracy	0.5440	0.5440	0.7161	0.7161
	P / R / F1 (macro)	0.3595 / 0.3182 / 0.3262	0.3595 / 0.3182 / 0.3262	0.5654 / 0.5287 / 0.5339	0.5654 / 0.5287 / 0.5339
gender	Accuracy	0.8719	0.8719	0.9117	0.9117
	P / R / F1 (macro)	0.7412 / 0.7097 / 0.7200	0.7412 / 0.7097 / 0.7200	0.7956 / 0.7638 / 0.7754	0.7956 / 0.7638 / 0.7754

Attribute	Metric	MODELS			
		InceptionV3_Fashion_2 (best weights)	InceptionV3_Fashion_2 (final weights)	InceptionV3_Fashion_3 (best weights)	InceptionV3_Fashion_3 (best weights)
season	Accuracy	0.6775	0.6775	0.7040	0.7040
	P / R / F1 (macro)	0.5244 / 0.4514 / 0.4692	0.5244 / 0.4514 / 0.4692	0.5187 / 0.5085 / 0.5132	0.5187 / 0.5085 / 0.5132
usage	Accuracy	0.8871	0.8871	0.9032	0.9032
	P / R / F1 (macro)	0.5544 / 0.4060 / 0.4419	0.5544 / 0.4060 / 0.4419	0.4429 / 0.3972 / 0.4168	0.4429 / 0.3972 / 0.4168

Πίνακας 2. Αποδόσεις InceptionV3 Πολύ-ετικετικών (Multi-Task) μοντέλων

Από τη συγκριτική ανάλυση των μετρικών αξιολόγησης προκύπτει ότι τα μοντέλα EfficientNetB3 υπερέχουν σαφώς έναντι των αντίστοιχων εκδόσεων του InceptionV3, σε σχεδόν όλα τα χαρακτηριστικά. Ειδικότερα, τα EfficientNetB3_Fashion_3 (final_weights) και EfficientNetB3_Fashion_3 (best_weights) παρουσιάζουν την υψηλότερη συνολική απόδοση μεταξύ όλων των εκδοχών, με εξαιρετικά αποτελέσματα στην ακρίβεια και στις μακρομετρικές τιμές (P/R/F1) για χαρακτηριστικά όπως baseColour (74.58% ακρίβεια, F1 $\approx$ 0.55), gender (90.94% ακρίβεια, F1 $\approx$ 0.75), season (70.63% ακρίβεια, F1 $\approx$ 0.51) και usage (90.89% ακρίβεια, F1 $\approx$ 0.42). Αντίθετα, τα καλύτερα InceptionV3 μοντέλα υστερούν ιδιαίτερα στο baseColour (71.61% ακρίβεια αλλά χαμηλότερο F1 $\approx$ 0.53), ενώ εμφανίζουν και ελαφρώς μειωμένες τιμές F1 σε season, usage και gender, παρά την καλή ακρίβεια.

Αξίζει να σημειωθεί ότι, επειδή το χαρακτηριστικό articleType κρίθηκε ως κρίσιμο και εκπαιδεύτηκε τελικά μέσω ξεχωριστού μοντέλου για βελτιστοποίηση, η σύγκριση επικεντρώνεται κυρίως στα υπόλοιπα χαρακτηριστικά. Σε αυτό το πλαίσιο, οι δύο τελικές εκδόσεις του EfficientNetB3 (final και best) εμφανίζουν πρακτικά ίδιες επιδόσεις σε όλες τις μετρικές. Παρόλα αυτά, το EfficientNetB3_Fashion_3 (final_weights) επιλέγεται ως τελικό μοντέλο προς χρήση, καθώς παρουσιάζει ελαφρώς μικρότερο inference time per sample (0.005961 δευτερόλεπτα έναντι 0.010635 του best), καθιστώντας το καταλληλότερο για εφαρμογές πραγματικού χρόνου.

Τελικό Μοντέλο:

EfficientNetB3 Multi-Task (Final Weights):

Average inference time per sample: 0.005961 seconds

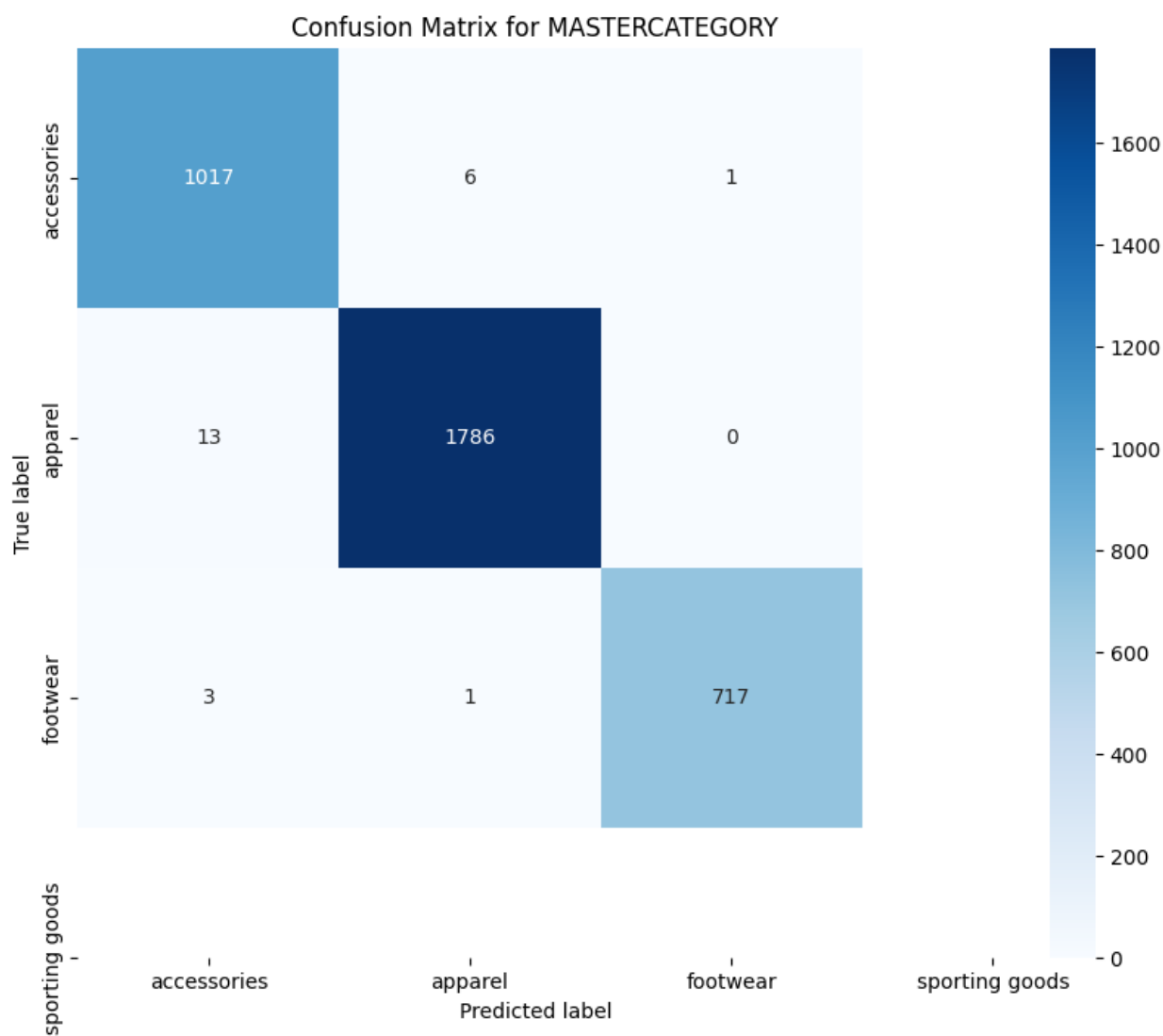
=== Attribute: MASTERCATEGORY ===

Accuracy: 0.9932

Precision / Recall / F1 (macro): 0.7448 / 0.7451 / 0.7449

Top-3 Accuracy: 0.9997

Πίνακας Σύγκρισης (Confusion Matrix):



Σχήμα 10. Πίνακας Σύγχυσης (Confusion Matrix) masterCategory

=== Attribute: SUBCATEGORY ===

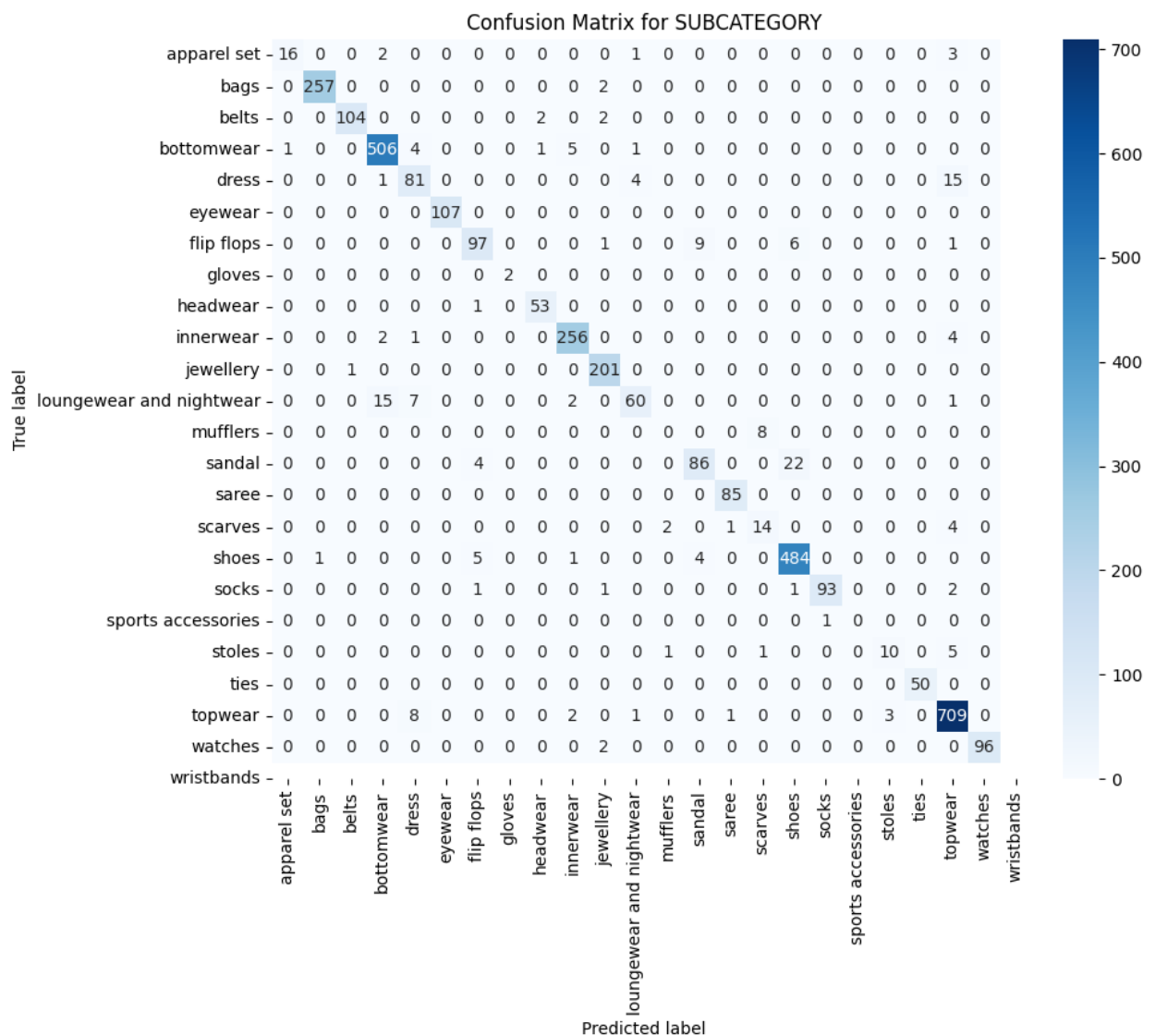
Accuracy: 0.9501

Precision / Recall / F1 (macro): 0.8111 / 0.7865 / 0.7969

Top-3 Accuracy: 0.9949

Top-5 Accuracy: 0.9975

Πίνακας Σύγχυσης (Confusion Matrix):



Σχήμα 11. Πίνακας Σύγχυσης (Confusion Matrix) subCategory

=== Attribute: BASECOLOUR ===

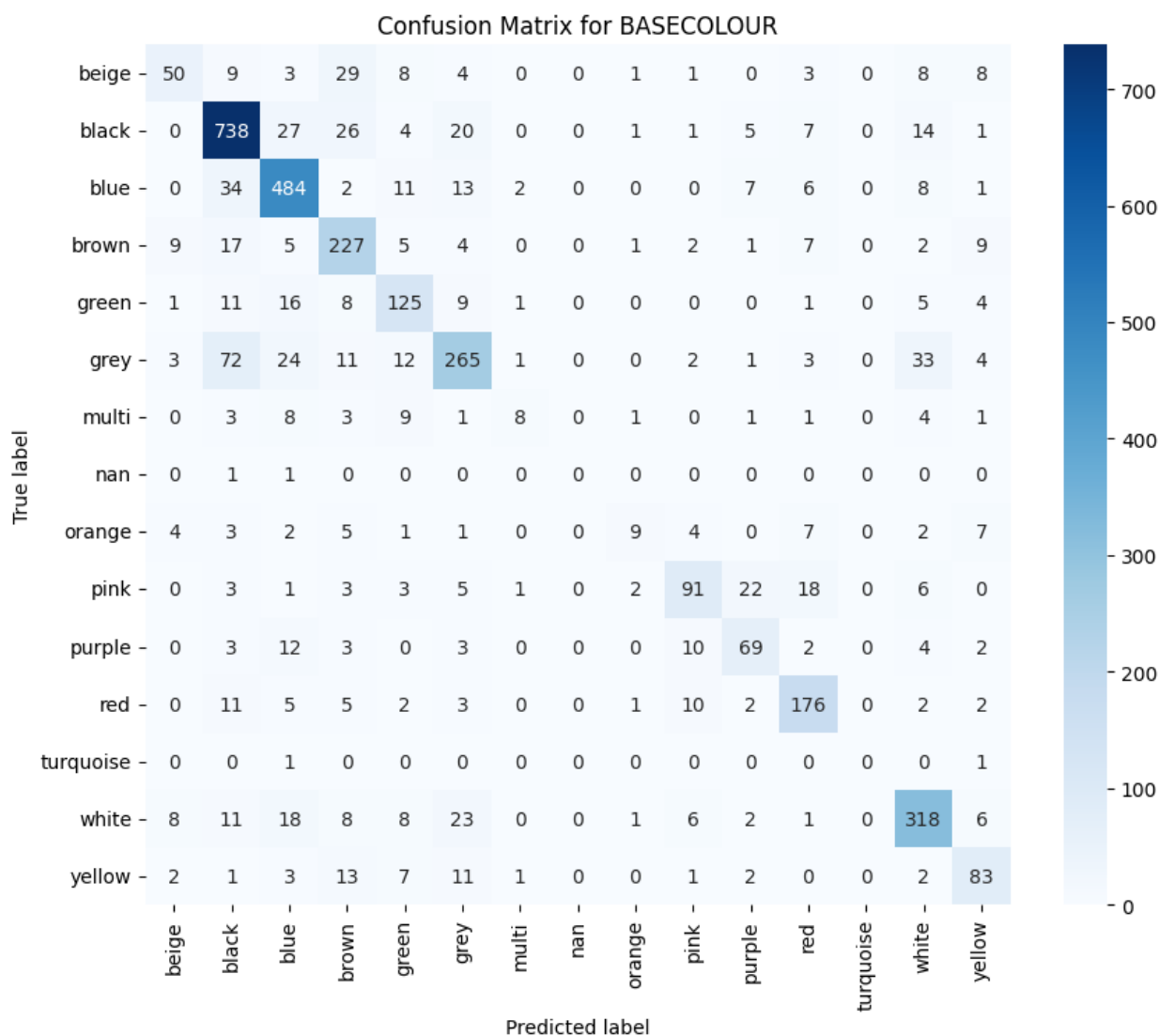
Accuracy: 0.7458

Precision / Recall / F1 (macro): 0.5928 / 0.5390 / 0.5517

Top-3 Accuracy: 0.9278

Top-5 Accuracy: 0.9687

Πίνακας Σύγχυσης (Confusion Matrix):



Σχήμα 12. Πίνακας Σύγχυσης (Confusion Matrix) baseColour

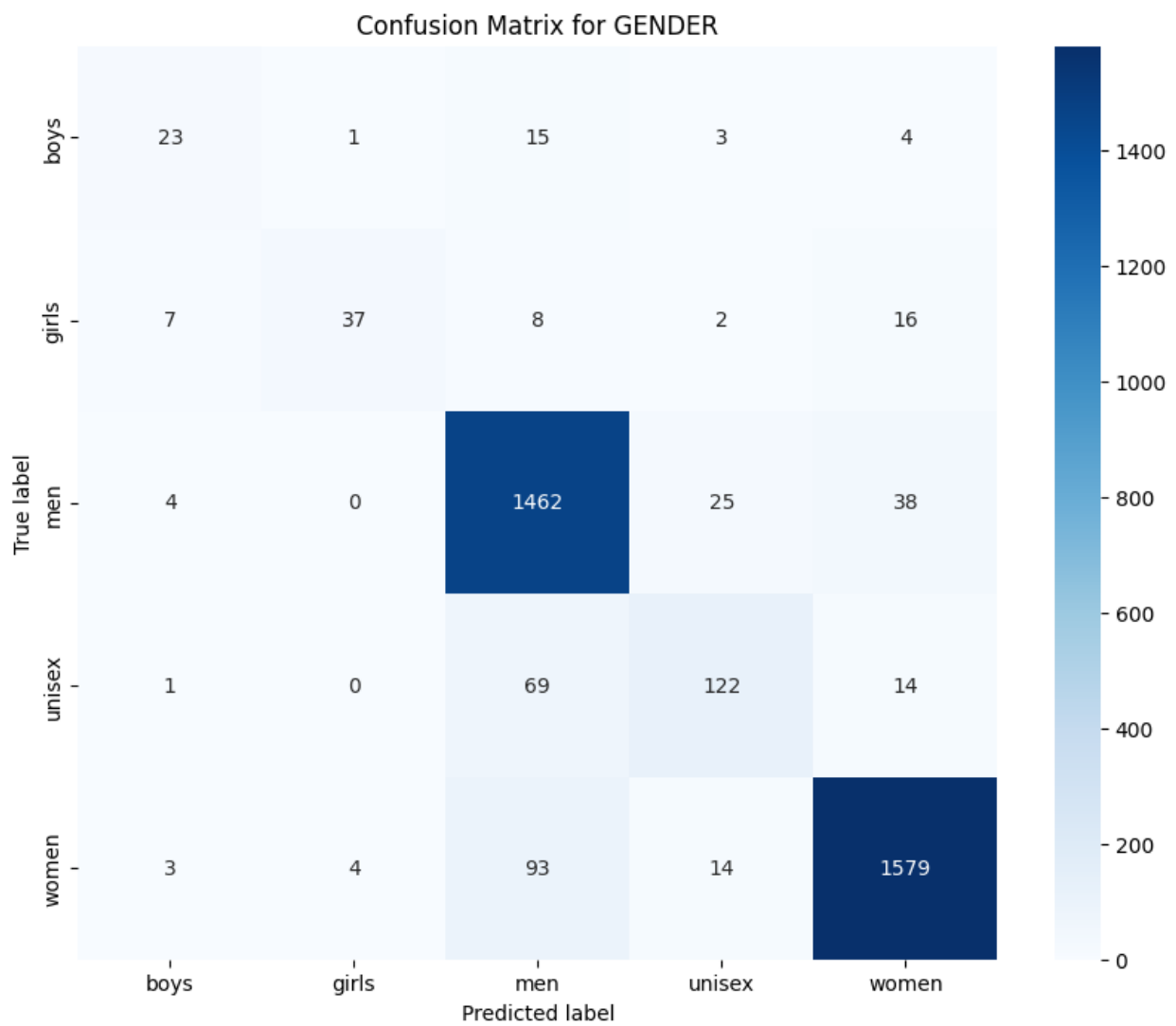
=== Attribute: GENDER ===

Accuracy: 0.9094

Precision / Recall / F1 (macro): 0.8130 / 0.7019 / 0.7459

Top-3 Accuracy: 0.9929

Πίνακας Σύγχυσης (Confusion Matrix):



Σχήμα 13. Πίνακας Σύγχυσης (Confusion Matrix) gender

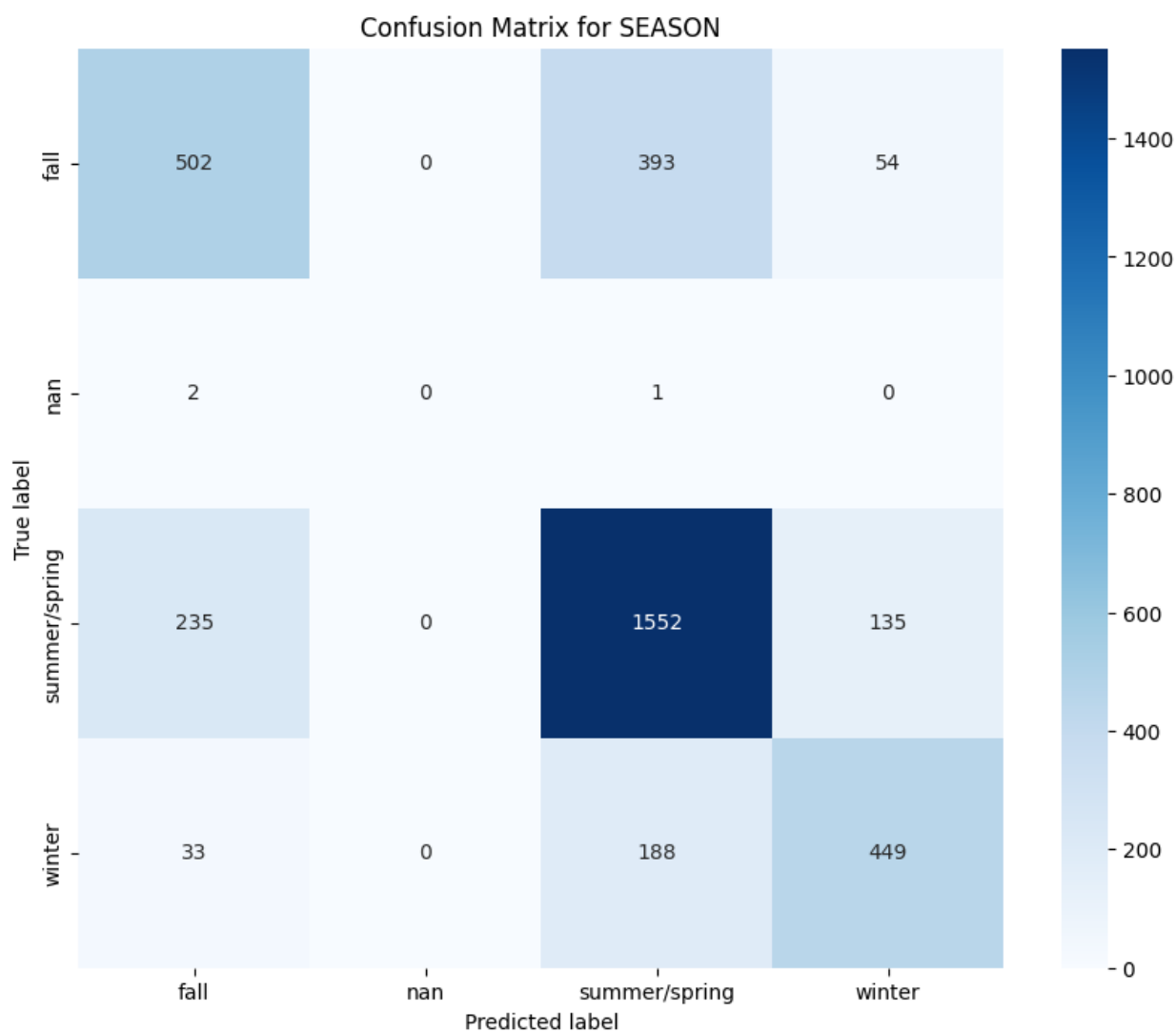
=== Attribute: SEASON ===

Accuracy: 0.7063

Precision / Recall / F1 (macro): 0.5203 / 0.5017 / 0.5088

Top-3 Accuracy: 0.9992

Πίνακας Σύγχυσης (Confusion Matrix):



Σχήμα 14. Πίνακας Σύγχυσης (Confusion Matrix) season

=== Attribute: USAGE ===

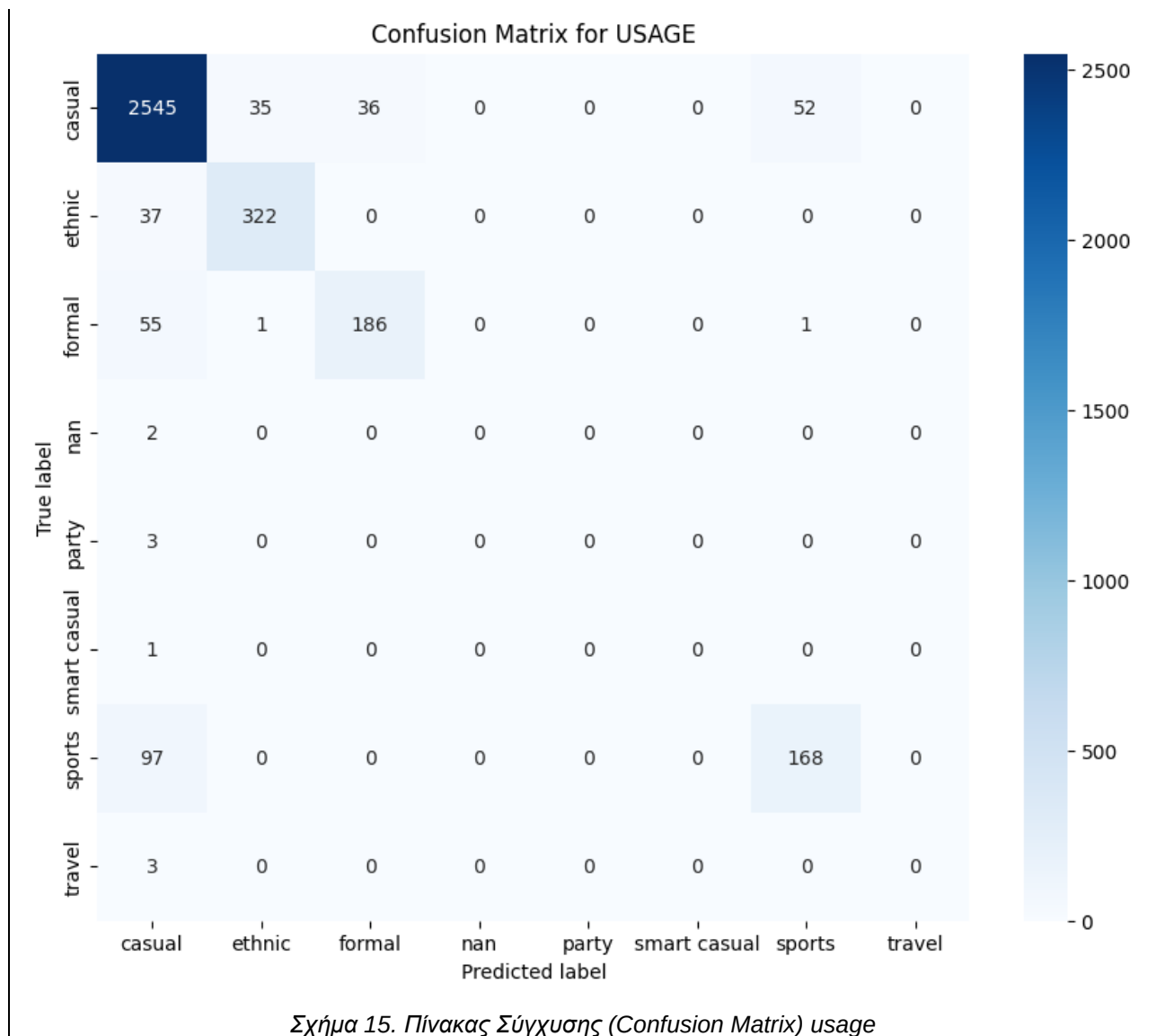
Accuracy: 0.9089

Precision / Recall / F1 (macro): 0.4282 / 0.4063 / 0.4163

Top-3 Accuracy: 0.9983

Top-5 Accuracy: 0.9994

Πίνακας Σύγχυσης (Confusion Matrix):



Πίνακας 3. Αποτελέσματα Τελικού Πολύ-ετικετικού (Multi-Task) μοντέλου EfficientNetB3_Fashion_3 (Final Weights)

Μέσα από παρατήρηση των πινάκων συγχύσεων (confusion matrices) πάνω στο σύνολο ελέγχου (test set), για κάθε χαρακτηριστικό, μπορούν να βγουν κάποια συγκεκριμένα συμπεράσματα. Είναι ξεκάθαρο ότι, αν και έγινε όσο περισσότερη βελτιστοποίηση είναι δυνατή στη δεδομένη κατάσταση, το μοντέλο είναι πολύ καλό και αποδοτικό, όμως δεν είναι σίγουρα τέλειο. Παρατηρείται συγκεκριμένα λανθασμένη κατηγοριοποίηση μερικές φορές στα εξής χαρακτηριστικά:

1. Στο season, όπου φθινοπωρινά ρούχα κατηγοριοποιούνται ως καλοκαιρινά και το αντίστροφο.
2. Λίγα λάθη στα baseColour, usage και gender.

Παρ' όλα αυτά, φαίνεται ότι οι λανθασμένες κατηγοριοποιήσεις βρίσκονται πάνω σε χαρακτηριστικά που δεν είναι εκ φύσεως απόλυτα και επικρατεί ένας βαθμός υποκειμενικότητας πάνω τους έτσι κι αλλιώς, σε σημείο που, όπως και ο ανθρώπινος εγκέφαλος θα μπορούσε να τα κατηγοριοποιήσει «λανθασμένα», έτσι συμβαίνει και με το παραπάνω μοντέλο. Για παράδειγμα η γραμμή διαχωρισμού μεταξύ φθινοπωρινών και ανοιξιάτικων ρούχων είναι λεπτή και διαφέρει από χωρά σε χωρά. Επίσης δύο άτομα μπορεί να διαφωνούσαν πάνω στο αν ένα ροζ σορτσάκι είναι ανδρικό, γυναικείο ή unisex.

Αν εξαιρεθούν λοιπόν τα αόριστα και αυθαίρετα στοιχεία κάποιων κλάσεων (που ισχύουν τόσο στη μηχανή, όσο και στον άνθρωπο), το μοντέλο λειτουργεί σε αρκετά ικανοποιητικό βαθμό και με εξαιρετικές επιδόσεις.

Μονό-Ετικετικά (Multi-Task) Μοντέλα:

EfficientNetB0_artType Single-Task:

EfficientNetB0_artType Single-Task (Final Weights)	EfficientNetB0_artType Single-Task (Best Weights)
Average inference time per sample: 0.002263 seconds	Average inference time per sample: 0.006654 seconds
=== Attribute: ARTICLETYPE === Accuracy: 0.8787 Precision / Recall / F1 (macro): 0.7517 / 0.7195 / 0.7179 Top-3 Accuracy: 0.9771 Top-5 Accuracy: 0.9913	=== Attribute: ARTICLETYPE === Accuracy: 0.8756 Precision / Recall / F1 (macro): 0.7492 / 0.7114 / 0.7169 Top-3 Accuracy: 0.9783 Top-5 Accuracy: 0.9901

Πίνακας 4. Αποδόσεις EfficientNetB0_artType Μονό-ετικετικού (Single-Task) μοντέλου

EfficientNetB1_artType Single-Task:

EfficientNetB1_artType Single-Task (Final Weights)	EfficientNetB1_artType Single-Task (Best Weights)
Average inference time per sample: 0.002941 seconds	Average inference time per sample: 0.008994 seconds
=== Attribute: ARTICLETYPE === Accuracy: 0.8846 Precision / Recall / F1 (macro): 0.7543 / 0.7405 / 0.7407 Top-3 Accuracy: 0.9802 Top-5 Accuracy: 0.9907	=== Attribute: ARTICLETYPE === Accuracy: 0.8826 Precision / Recall / F1 (macro): 0.7610 / 0.7235 / 0.7284 Top-3 Accuracy: 0.9805 Top-5 Accuracy: 0.9893

Πίνακας 5. Αποδόσεις EfficientNetB1_artType Μονό-ετικετικού (Single-Task) μοντέλου

EfficientNetB3_artType Single-Task:

EfficientNetB3_artType Single-Task (Final Weights)	EfficientNetB3_artType Single-Task (Best Weights)
Average inference time per sample: 0.005506 seconds	Average inference time per sample: 0.013010 seconds
=== Attribute: ARTICLETYPE === Accuracy: 0.8699 Precision / Recall / F1 (macro): 0.7537 / 0.7214 / 0.7221 Top-3 Accuracy: 0.9698 Top-5 Accuracy: 0.9822	=== Attribute: ARTICLETYPE === Accuracy: 0.8798 Precision / Recall / F1 (macro): 0.7561 / 0.7469 / 0.7403 Top-3 Accuracy: 0.9715 Top-5 Accuracy: 0.9822

Πίνακας 6. Αποδόσεις EfficientNetB3_artType Μονό-ετικτικού (Single-Task) μοντέλου

Μεταξύ όλων των μοντέλων που εκπαιδεύτηκαν αποκλειστικά για την κατηγοριοποίηση του χαρακτηριστικού articleType, τα καλύτερα αποτελέσματα επιτεύχθηκαν από τα μοντέλα EfficientNetB1_artType Single-Task (Final Weights) και EfficientNetB1_artType Single-Task (Best Weights). Τα δύο αυτά μοντέλα παρουσίασαν τις υψηλότερες τιμές σε ακρίβεια (accuracy), F1-score και top-k accuracy, υπερέχοντας τόσο έναντι των αντίστοιχων εκδόσεων των EfficientNetB0_artType και EfficientNetB3_artType, όσο και στο σύνολο της απόδοσης. Μετά από σειρά δοκιμών που πραγματοποιήθηκαν και σε πραγματικά δεδομένα από την εφαρμογή, το μοντέλο EfficientNetB1_artType Single-Task (Best Weights) απέδωσε λίγο καλύτερα και με μεγαλύτερη σταθερότητα, παρουσιάζοντας ακρίβεια 88.26% και top-3 accuracy 98.05%, παρόλο που το EfficientNetB1_artType Single-Task (Final Weights) υπερτερούσε πολύ ελαφρώς στα μετρικά (88.46% accuracy, ενώ το πολύ-εξοδικό είχε 85.30%) ενώ διατήρησε εξαιρετική ταχύτητα πρόβλεψης ανά δείγμα. Για τον λόγο αυτό, επιλέχθηκε το EfficientNetB1_artType Single-Task (Best Weights) τελικά ως το βασικό μοντέλο ταξινόμησης για την κατηγορία articleType στο τελικό σύστημα. Η διαδικασία δοκιμής σε πραγματικά δεδομένα, παρουσιάζεται στο υποκεφάλαιο 6.2.

Τελικό Μοντέλο:

EfficientNetB1_artType Single-Task (Best Weights):

Average inference time per sample: 0.008994 seconds

=== Attribute: ARTICLETYPE ===

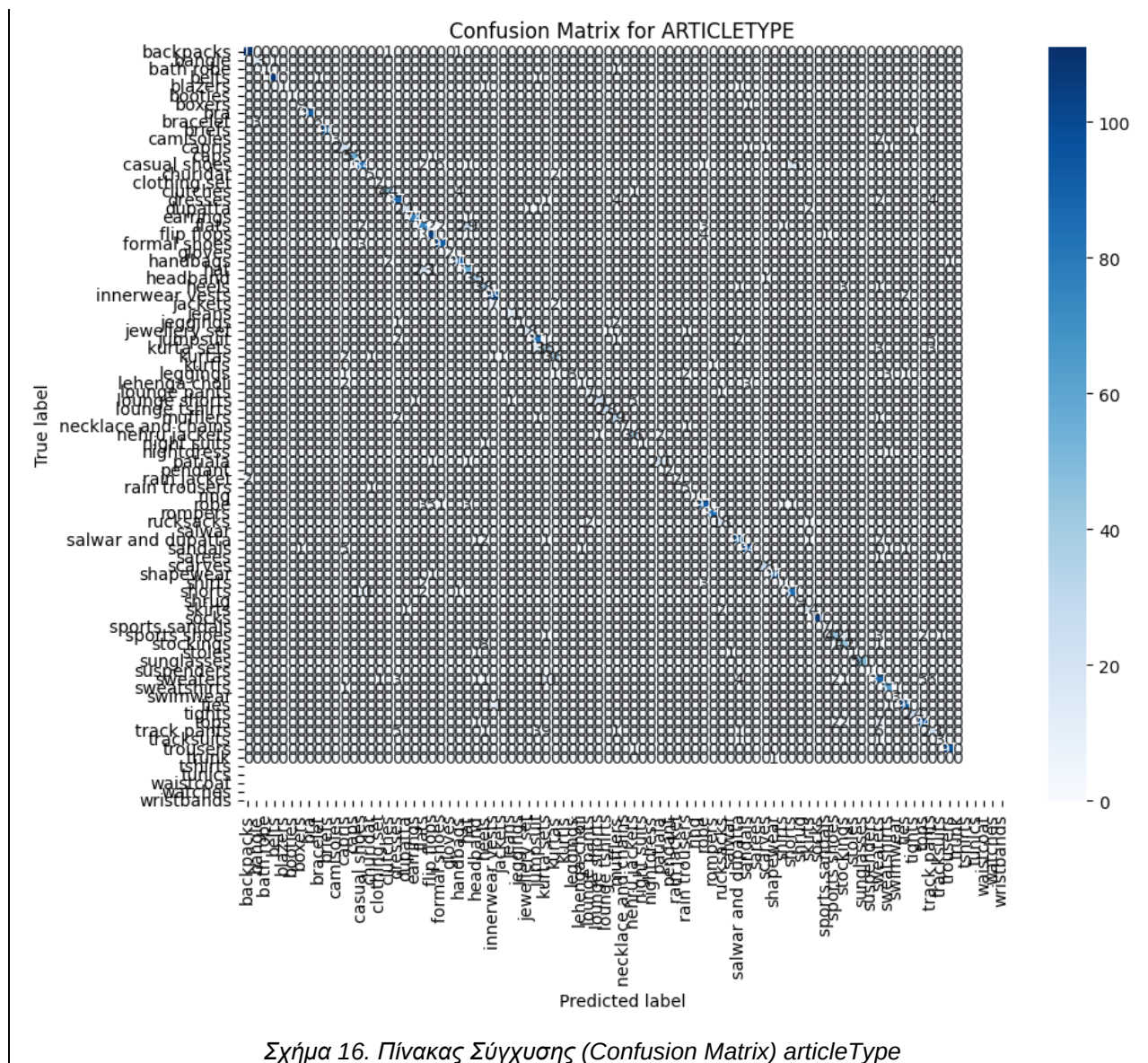
Accuracy: 0.8826

Precision / Recall / F1 (macro): 0.7610 / 0.7235 / 0.7284

Top-3 Accuracy: 0.9805

Top-5 Accuracy: 0.9893

Πίνακας Σύγχυσης (Confusion Matrix):

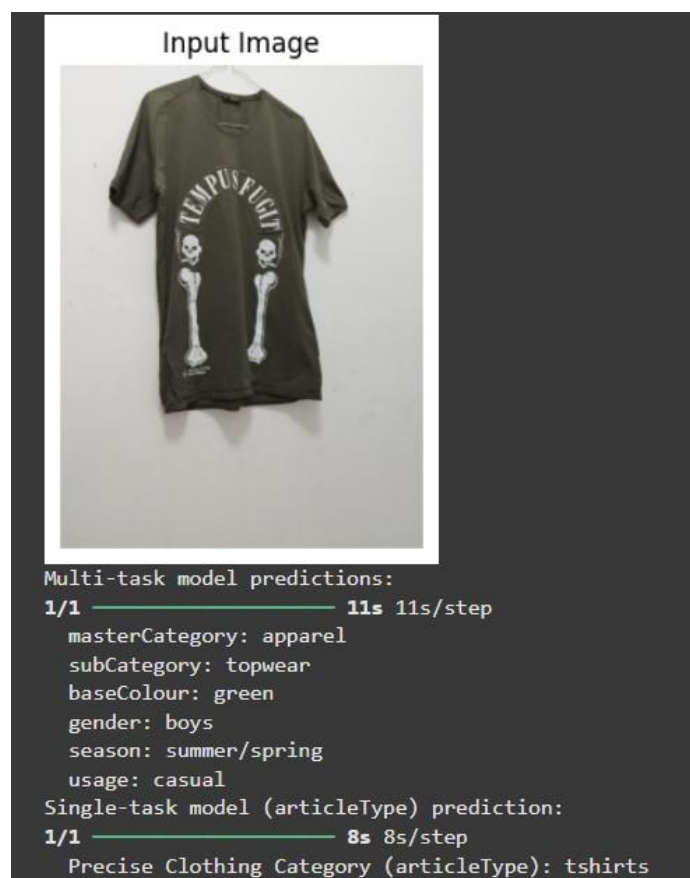


Πίνακας 7. Αποτελέσματα Τελικού Μονό-ετικτικού (Single-Task) μοντέλου EfficientNetB1_artType (Best Weights)

6.2 Δοκιμές Εφαρμογής σε Πραγματικές Συνθήκες

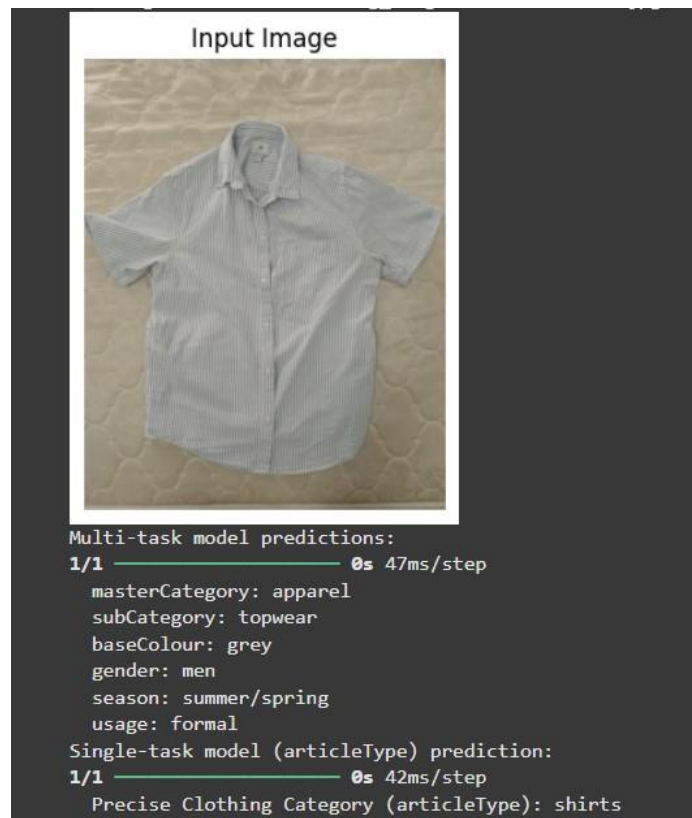
Αφού επιλέχθηκαν τα τελικά μοντέλα ταξινόμησης προς δοκιμή, ακολούθησαν δοκιμές της εφαρμογής σε ρεαλιστικές συνθήκες χρήσης. Δοκιμάστηκαν 1 πολύ-ετικετικό μοντέλο, το EfficientNetB3_Fashion_3 (Final Weights) και 2 μονο-ετικετικά μοντέλα για το articleType, τα EfficientNetB1_artType (Final Weights) και EfficientNetB1_artType (Best Weights). Σε αυτή την ενότητα παρουσιάζεται πώς τα μοντέλα ανταποκρίθηκαν κατά την επεξεργασία φωτογραφιών από κινητό τηλέφωνο και μετά παρουσιάζονται παραδείγματα μέσω της χρήσης της Android εφαρμογής που αναπτύχθηκε.

Έγιναν, εξ αρχής, δοκιμές πάνω σε 18 παραδείγματα φωτογραφιών ρουχισμού. Το EfficientNetB1_artType (Final Weights) έβγαλε σωστά αποτελέσματα σε 15/18 δείγματα (λανθασμένη πρόβλεψη jackets αντί για sweatshirts στο δείγμα 15, λανθασμένη πρόβλεψη jeans αντί για trousers/capris στο δείγμα 16 και λανθασμένη πρόβλεψη leggings αντί για capris στο δείγμα 18), ενώ το EfficientNetB1_artType (Best Weights) έβγαλε σωστά αποτελέσματα σε 17/18 αποτελέσματα (λανθασμένη πρόβλεψη jeans αντί για trousers/capris στο δείγμα 16). Η παραπάνω παρατήρηση σε συνδυασμό με την φύση των λαθών του EfficientNetB1_artType (Final Weights) (οι φύσεις των λαθών του είχαν μεγάλη απόκλιση από την ορθή πρόβλεψη και παρατηρήθηκε συγκεκριμένο μπέρδεμα μπλουζών με τζάκετ), καθώς και με παρατήρηση των αποδόσεων μετά από εκτεταμένη χρήση και των δύο μέσω της κινητής εφαρμογής, έγινε καταληκτική και τελική επιλογή του EfficientNetB1_artType (Best Weights) ως το τελικό μοντέλο προς χρήση. Παρακάτω παραθέτονται οι 18 προβλέψεις του EfficientNetB3_Fashion_3 (Final Weights) για τα υπόλοιπα χαρακτηριστικά και του EfficientNetB1_artType (Best Weights) για την ακριβή κατηγορία ρούχου (articleType), συνοδευόμενα γραπτώς και με τα αντίστοιχα αποτελέσματα του EfficientNetB1_artType (Final Weights) πάνω στην ίδια εικόνα.



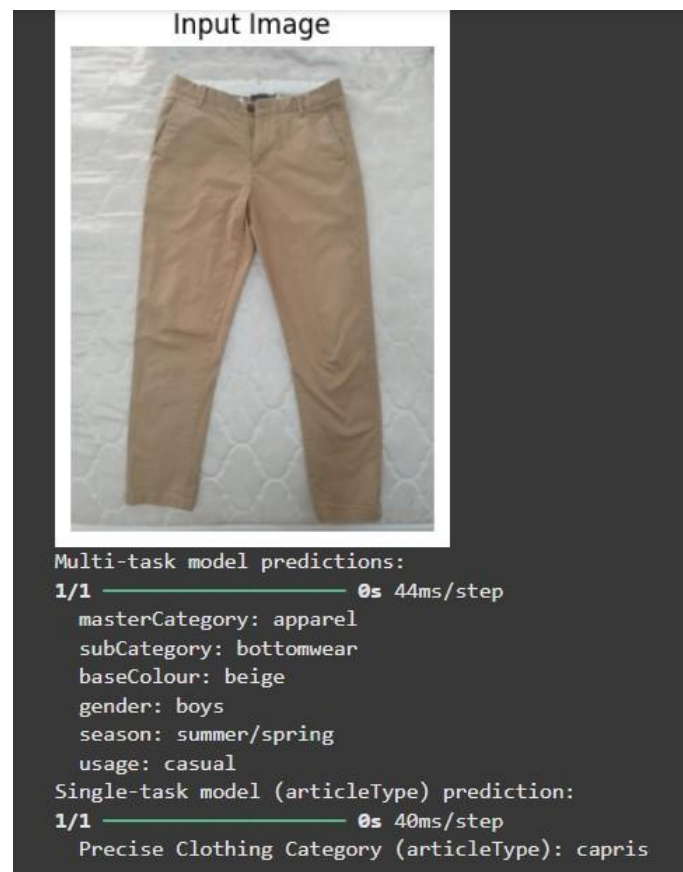
Εικόνα 20. Δοκιμή Μοντέλων 1

Αποτέλεσμα EfficientNetB1_artType (Final Weights): tshirts



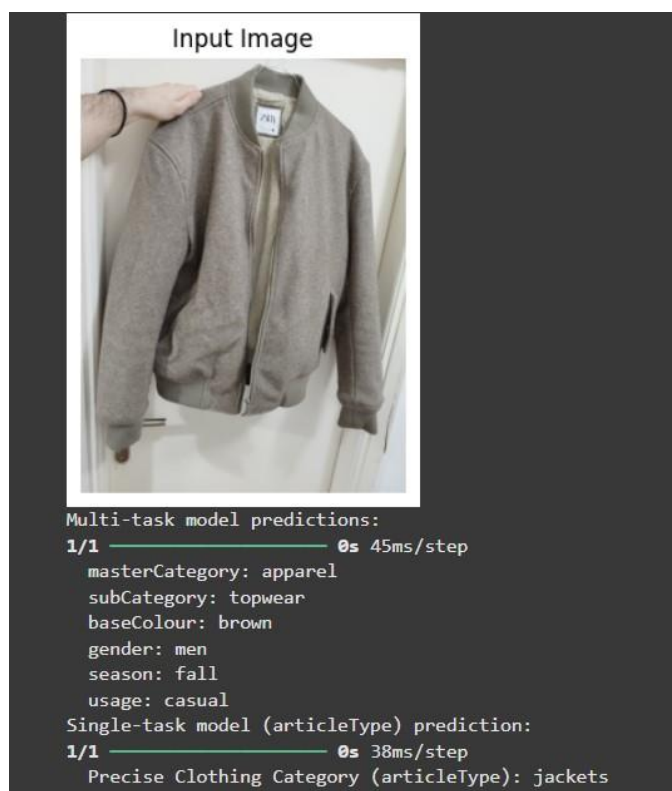
Εικόνα 21. Δοκιμή Μοντέλων 2

Αποτέλεσμα EfficientNetB1_artType (Final Weights): shirts



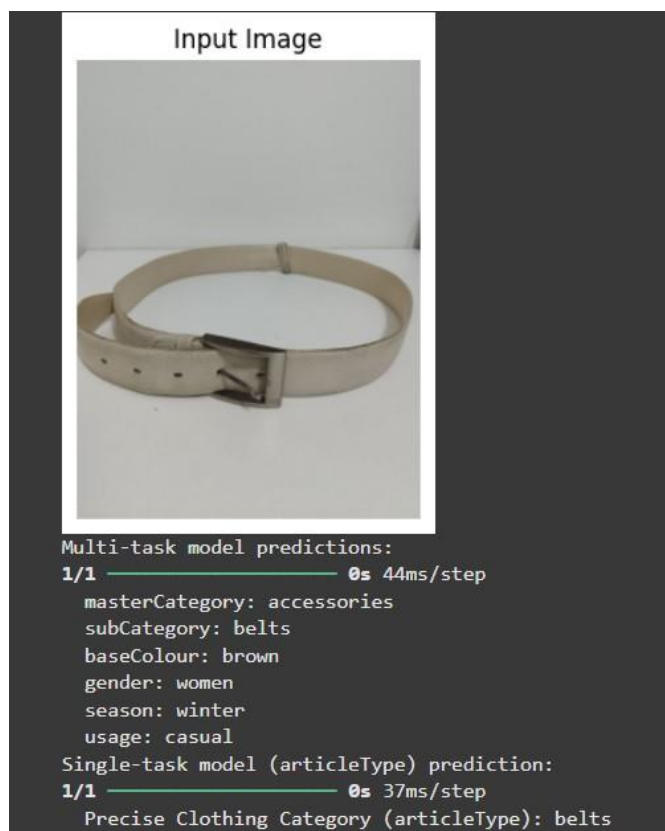
Εικόνα 22. Δοκιμή Μοντέλων 3

Αποτέλεσμα EfficientNetB1_artType (Final Weights): capris



Εικόνα 23. Δοκιμή Μοντέλων 4

Αποτέλεσμα EfficientNetB1_artType (Final Weights): jackets



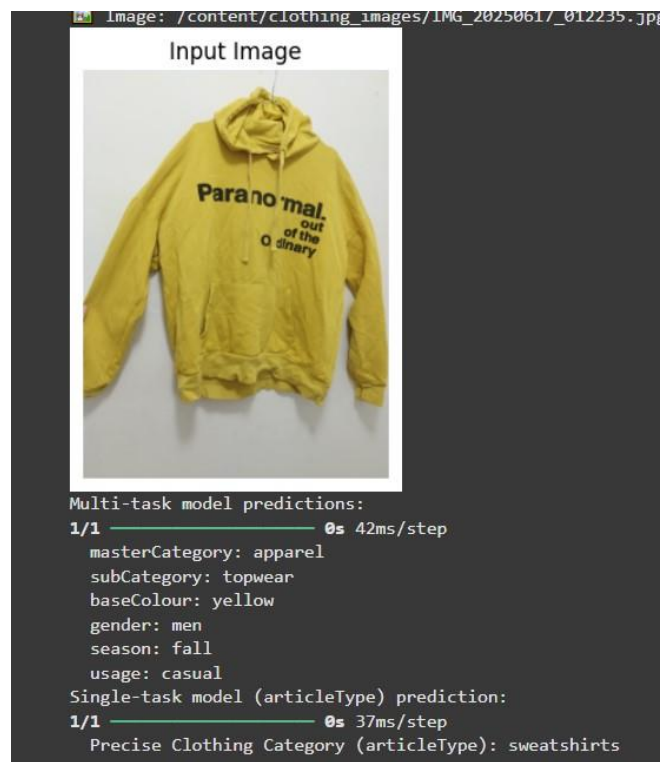
Εικόνα 24. Δοκιμή Μοντέλων 5

Αποτέλεσμα EfficientNetB1_artType (Final Weights): belts



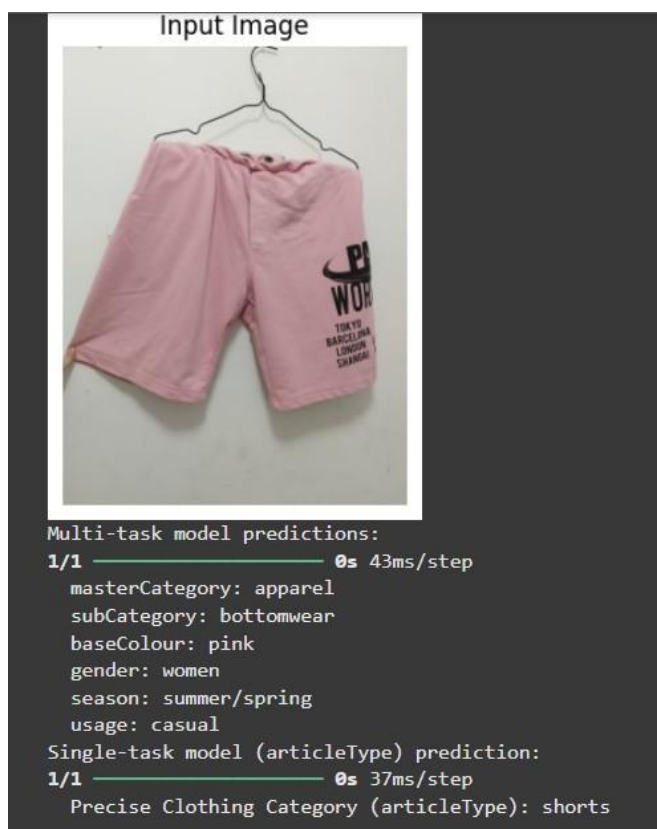
Εικόνα 25. Δοκιμή Μοντέλων 6

Αποτέλεσμα EfficientNetB1_artType (Final Weights): sweatshirts



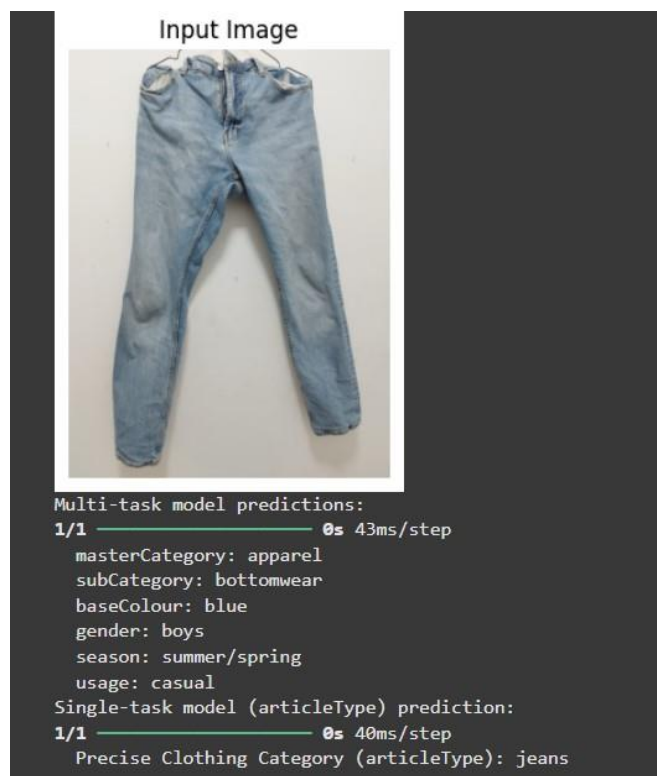
Εικόνα 26. Δοκιμή Μοντέλων 7

Αποτέλεσμα EfficientNetB1_artType (Final Weights): sweatshirts



Εικόνα 27. Δοκιμή Μοντέλων 8

Αποτέλεσμα EfficientNetB1_artType (Final Weights): shorts



Εικόνα 28. Δοκιμή Μοντέλων 9

Αποτέλεσμα EfficientNetB1_artType (Final Weights): jeans



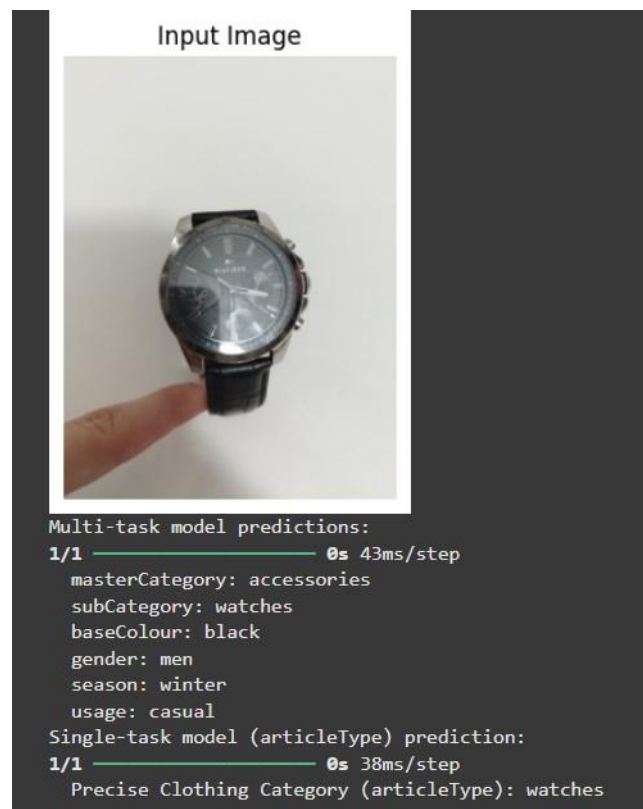
Εικόνα 29. Δοκιμή Μοντέλων 10

Αποτέλεσμα EfficientNetB1_artType (Final Weights): socks



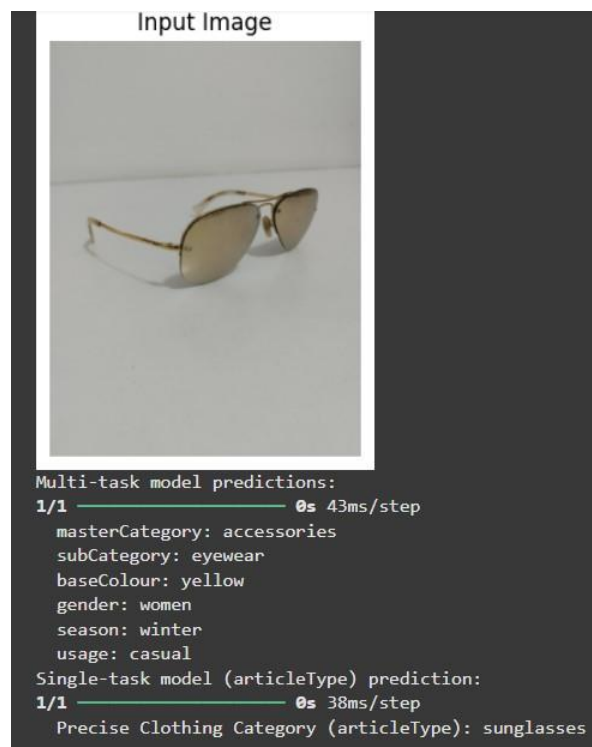
Εικόνα 30. Δοκιμή Μοντέλων 11

Αποτέλεσμα EfficientNetB1_artType (Final Weights): tshirts



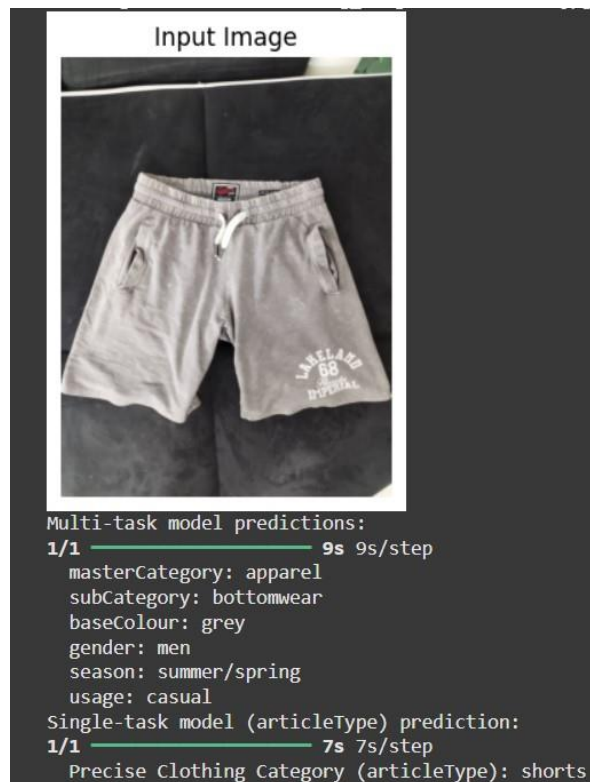
Εικόνα 31. Δοκιμή Μοντέλων 12

Αποτέλεσμα EfficientNetB1_artType (Final Weights): watches



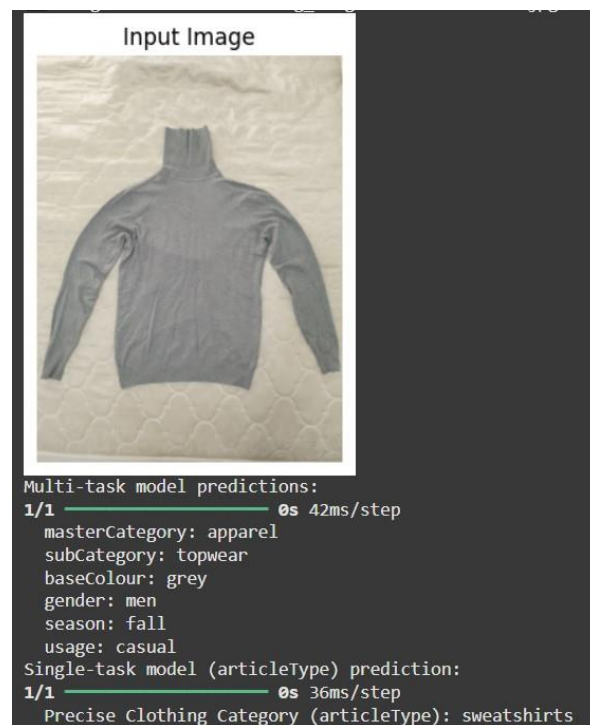
Εικόνα 32. Δοκιμή Μοντέλων 13

Αποτέλεσμα EfficientNetB1_artType (Final Weights): sunglasses



Εικόνα 33. Δοκιμή Μοντέλων 14

Αποτέλεσμα EfficientNetB1_artType (Final Weights): shorts



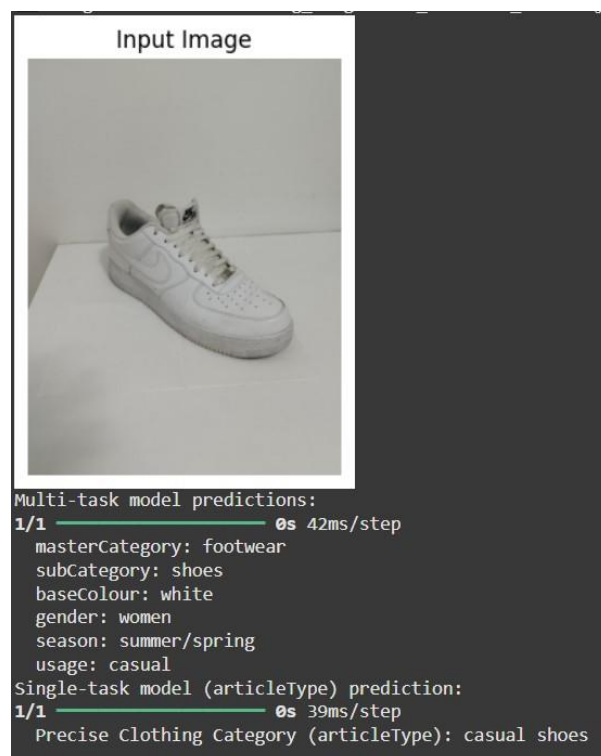
Εικόνα 34. Δοκιμή Μοντέλων 15

Αποτέλεσμα EfficientNetB1_artType (Final Weights): jackets



Εικόνα 35. Δοκιμή Μοντέλων 16

Αποτέλεσμα EfficientNetB1_artType (Final Weights): jeans



Εικόνα 36. Δοκιμή Μοντέλων 17

Αποτέλεσμα EfficientNetB1_artType (Final Weights): casual shoes



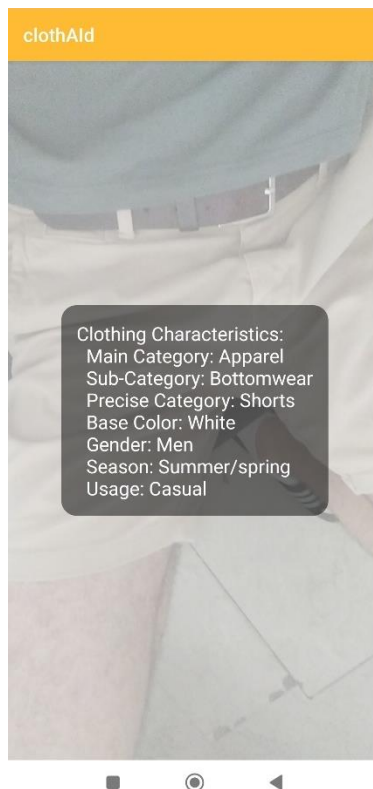
Εικόνα 37. Δοκιμή Μοντέλων 18

Αποτέλεσμα EfficientNetB1_artType (Final Weights): leggings

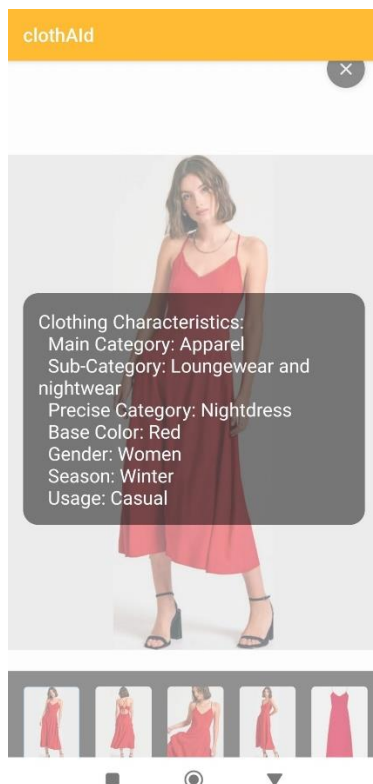
Το τελικό σύνολο μοντέλων προς χρήση, λοιπόν, αναδείχθηκε αποτελεσματικό και με υψηλές επιδόσεις. Παρακάτω δίνονται κάποια επιπλέον παραδείγματα χρήσης της εφαρμογής στον πραγματικό κόσμο, πάνω σε εικόνες από τυχαία e-shops/διαδικτυακά μαγαζιά ρούχων, καθώς και επιπλέον φωτογραφίες:



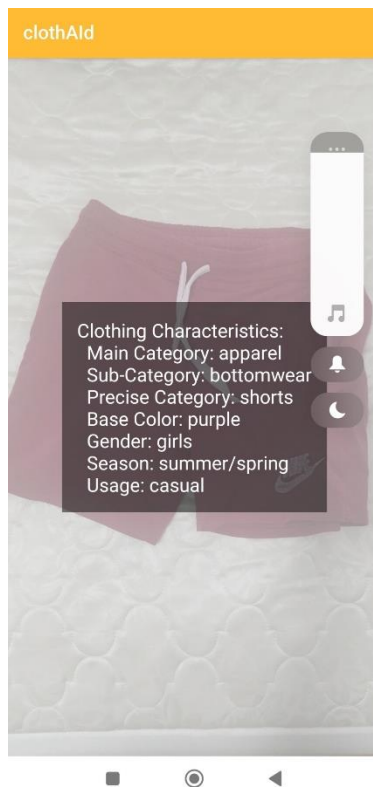
Εικόνα 38. Παράδειγμα Χρήσης Εφαρμογής 1



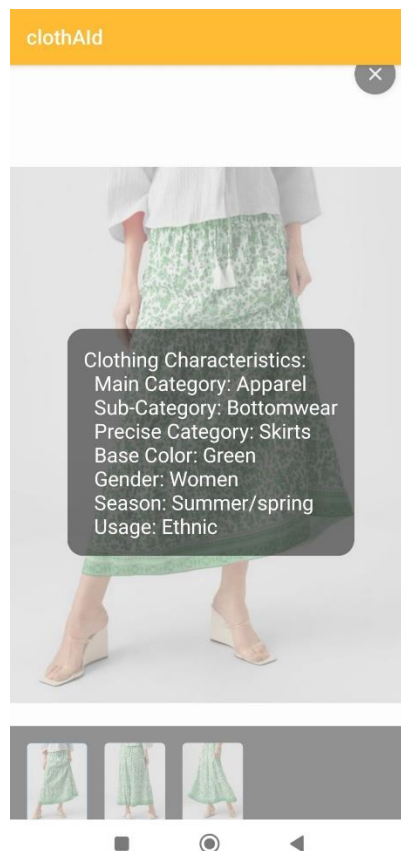
Εικόνα 39. Παράδειγμα Χρήσης Εφαρμογής 2



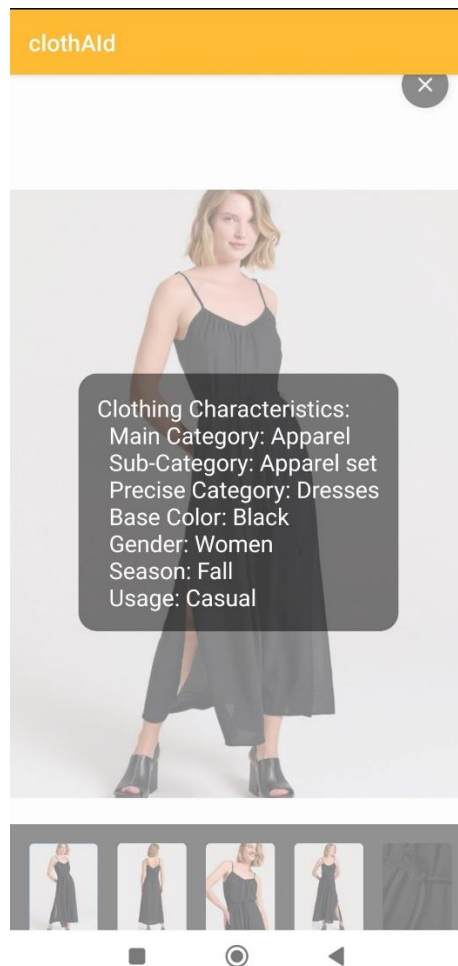
Εικόνα 40. Παράδειγμα Χρήσης Εφαρμογής 3



Εικόνα 41. Παράδειγμα Χρήσης Εφαρμογής 4



Εικόνα 42. Παράδειγμα Χρήσης Εφαρμογής 5



Εικόνα 43. Παράδειγμα Χρήσης Εφαρμογής 6

Με αυτό το κεφάλαιο ολοκληρώνεται η υλοποίηση του έργου και επομένως της παρούσας πτυχιακής εργασίας.

7. ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ ΚΑΙ ΒΕΛΤΙΩΣΕΙΣ

Η παρούσα εργασία αποτελεί μια ισχυρή βάση για την υποστήριξη τυφλών χρηστών μέσω της τεχνητής νοημοσύνης και της φορητής τεχνολογίας. Ωστόσο, υπάρχουν πολλαπλές δυνατότητες επέκτασης και βελτίωσης της εφαρμογής και των υποστηρικτικών της συστημάτων. Οι μελλοντικές κατευθύνσεις περιλαμβάνουν την υποστήριξη επιπλέον γλωσσών, την ενσωμάτωση περισσότερων χαρακτηριστικών ενδυμάτων, την αναγνώριση μη έγκυρων φωτογραφιών, τη βελτίωση της ακρίβειας μέσω καλύτερων δεδομένων, την προσωποποιημένη μάθηση προτιμήσεων, αλλά και τον εκσυγχρονισμό της εμφάνισης της εφαρμογής με δυνατότητες διαδικτυακής πρόσβασης.

7.1 Υποστήριξη Ελληνικών και Άλλων Γλωσσών

Η εφαρμογή προς το παρόν υποστηρίζει αποκλειστικά την αγγλική γλώσσα στα μηνύματα του TTS (Text-to-Speech). Μια φυσική εξέλιξη είναι η ενσωμάτωση πλήρους υποστήριξης ελληνικών, τόσο για την είσοδο/έξοδο του TTS όσο και για το περιβάλλον χρήστη (UI). Η

ελληνική υποστήριξη είναι ιδιαίτερα σημαντική καθώς η εφαρμογή αναπτύχθηκε με γνώμονα την τοπική κοινωνία. Επιπλέον, μπορεί να προστεθεί δυναμική υποστήριξη πολλαπλών γλωσσών, έτσι ώστε η εφαρμογή να προσαρμόζεται στην κατάλληλη γλώσσα προτίμησής, διευρύνοντας το δυναμικό κοινό της εφαρμογής σε παγκόσμιο επίπεδο.

7.2 Ενσωμάτωση Αναγνώρισης Επιπλέον Χαρακτηριστικών

Στο υπάρχον σύστημα γίνεται αναγνώριση επτά βασικών χαρακτηριστικών ενδυμάτων. Στο μέλλον, μπορούν να ενσωματωθούν επιπλέον χαρακτηριστικά, όπως το υλικό (π.χ. βαμβακερό, συνθετικό), τα μοτίβα (ριγέ, φλοράλ), το σχήμα του κολάρου (neckline) και το μήκος των μανικιών (sleeve length π.χ. κοντομάνικο). Η ενσωμάτωση αυτών απαιτεί εμπλουτισμό του συνόλου δεδομένων (ή διαφορετικό σύνολο δεδομένων), καθώς και επανεκπαίδευση των μοντέλων για να διαχειρίζονται με ακρίβεια τις νέες κλάσεις. Ένα πιο πλούσιο σε πληροφορία σύστημα θα παρέχει μεγαλύτερη χρηστική αξία στον τελικό χρήστη.

7.3 Ενσωμάτωση Ανιχνευτή Αναγνώρισης Πλαισίου Έγκυρης Φωτογραφίας

Μια πρακτική δυσκολία για τυφλούς χρήστες είναι η λήψη φωτογραφιών που να απεικονίζουν καθαρά το ένδυμα. Για το λόγο αυτό, μια χρήσιμη επέκταση είναι η ανάπτυξη και ενσωμάτωση ενός βοηθητικού ανιχνευτή που θα αξιολογεί αν η φωτογραφία είναι έγκυρη για ανίχνευση (π.χ. αν περιέχει το ρούχο, αν είναι θολή, αν είναι πολύ κοντινή ή μακρινή στο ρούχο). Μπορεί να χρησιμοποιηθεί ένα ελαφρύ CNN ή ένα σύστημα ταξινόμησης εικόνας που να λειτουργεί προκαταρκτικά και να παρέχει άμεση ηχητική ειδοποίηση στον χρήστη για την ποιότητα της φωτογραφίας, πριν προχωρήσει σε αναγνώριση.

7.4 Βελτίωση Ακρίβειας Μοντέλων (Επένδυση σε Δεδομένα και Υποδομές)

Αν και τα τρέχοντα μοντέλα παρουσιάζουν ικανοποιητική ακρίβεια, η απόδοσή τους μπορεί να ενισχυθεί μέσω δύο βασικών δρόμων: εμπλουτισμού του συνόλου δεδομένων και αναβάθμισης των υπολογιστικών υποδομών. Περισσότερες εικόνες ανά κατηγορία, καλύτερη ποιότητα δεδομένων, καθώς και ισορροπημένη εκπροσώπηση σπάνιων κατηγοριών μπορούν να οδηγήσουν σε αισθητή βελτίωση. Επιπλέον, με πρόσβαση σε ισχυρότερες GPU, μπορούν να εκπαιδευτούν πιο πολύπλοκα μοντέλα ή να χρησιμοποιηθούν πιο υπολογιστικά βαριές τεχνικές για περαιτέρω αύξηση της ακρίβειας.

7.5 Ενσωμάτωση Μηχανικής Μάθησης για την Εκμάθηση Γούστου

Ένα ακόμα πιο εξελιγμένο σενάριο χρήσης αφορά τη δημιουργία ενός συστήματος προσωποποιημένης μάθησης των προτιμήσεων του χρήστη. Εφόσον έτσι κι αλλιώς καταγράφονται τα αποτελέσματα και τα χαρακτηριστικά του ρούχου κατά την αναγνώριση, με τη βοήθεια τεχνικών μηχανικής μάθησης, το σύστημα θα μπορούσε να χρησιμοποιήσει αυτά τα δεδομένα και να «μαθαίνει» τις προτιμήσεις του χρήστη σε χρώματα, είδη ρούχων ή εποχές και να προτείνει ρούχα ή συνδυασμούς ανάλογα, καθώς και να αναφέρει το ποσοστό σύγκλισης ενός αναγνωρισμένου ρούχου, σε σχέση με το γούστο του (π.χ. 93% σύμφωνο με το γούστο του). Αυτό μετατρέπει την εφαρμογή από έναν παθητικό περιγραφέα σε έναν ενεργό ψηφιακό βοηθό μόδας για τους οπτικά ανάπηρους, με προσαρμοστική συμπεριφορά.

7.6 Βελτίωση Εξωτερικής Εμφάνισης Εφαρμογής και Διαδικτυακή Διαθεσιμότητα

Η βελτίωση του οπτικού σχεδιασμού της εφαρμογής (UI/UX) μπορεί να κάνει την εμπειρία πιο ευχάριστη και λειτουργική για τους βοηθούς ή συγγενείς των χρηστών. Η χρήση μοντέρνων σχεδιαστικών στοιχείων και πιο προσιτών επιλογών πλοήγησης μπορεί να ενισχύσει την χρηστικότητα της εφαρμογής. Παράλληλα, μια διαδικτυακή έκδοση της υπηρεσίας — κάνοντας την υπηρεσία προσβάσιμη από παντού —, μέσω ανάπτυξη κατάλληλων υποδομών και αδειών, μπορεί να επιτρέψει τη χρήση της πλατφόρμας από όλες τις συσκευές, καθιστώντας το σύστημα πλήρως ευέλικτο και προσβάσιμο.

8. ΤΕΧΝΟΛΟΓΙΕΣ ΚΑΙ ΕΡΓΑΛΕΙΑ

8.1 Ανάπτυξη Μοντέλων Βαθιάς Μάθησης

- Python
- TensorFlow
- Keras
- scikit-learn
- Pandas
- NumPy
- Matplotlib
- Seaborn
- OpenCV
- PIL / Pillow
- EfficientNetB0
- EfficientNetB1
- EfficientNetB3
- InceptionV3
- Google Colab
- Jupyter Notebooks
- Paperspace
- CSV

8.2 Ανάπτυξη Διακομιστή και Κινητής Εφαρμογής

- Kotlin
- Java
- XML
- Android Studio
- Flask
- Android Jetpack
- LiveData
- GestureDetector
- TextToSpeech API
- Coroutines

- Retrofit
- Intent
- MediaPlayer
- Glide
- Gradle
- Git
- GitHub

9. ΣΥΜΠΕΡΑΣΜΑΤΑ

Το έργο αυτό της παρούσας πτυχιακής εργασίας επιβεβαιώνει τη σημαντική συμβολή της τεχνητής νοημοσύνης στην αναγνώριση και περιγραφή ενδυμάτων, προσφέροντας ουσιαστική υποστήριξη σε άτομα με οπτική αναπηρία.

Μέσω της ανάπτυξης εξειδικευμένων μοντέλων βαθιάς μάθησης, υλοποιήθηκε με επιτυχία υψηλή ακρίβεια στην ταξινόμηση τόσο βασικών κατηγοριών ενδυμάτων όσο και λεπτομερών χαρακτηριστικών, όπως το χρώμα, η εποχή και η χρήση του ρούχου. Η πολύ-ετικετική προσέγγιση με πολλαπλές εξόδους για τα χαρακτηριστικά, σε συνδυασμό με την εστιασμένη ανάπτυξη ειδικού μοντέλου για την αναγνώριση της ακριβούς κατηγορίας ρούχου, επέτρεψε μια ολοκληρωμένη περιγραφή του κάθε ρούχου, αναβαθμίζοντας την ποιότητα των προβλέψεων. Το έργο χαρακτηρίστηκε από συνεχή έρευνα, μελέτη και απόπειρα προς βελτιστοποίηση, ειδικά όσο αναφορά τον πυρήνα της πτυχιακής εργασίας, που αφορά την ανάπτυξη των κατάλληλων συνελκτικών νευρωνικών δικτύων.

Τα πειράματα και οι δοκιμές που πραγματοποιήθηκαν απέδειξαν την αποτελεσματικότητα της προσέγγισης, καθώς και την αναγκαιότητα συνεχούς βελτίωσης σε σημεία όπως η ισορροπία του συνόλου δεδομένων, η επιλογή των υπερπαραμέτρων, και η βελτιστοποίηση των τεχνικών αύξησης δεδομένων. Η επιλογή δύο φάσεων εκπαίδευσης, καθώς και η χρήση ξεχωριστών μοντέλων για επιμέρους χαρακτηριστικά, βελτίωσαν σημαντικά την ακρίβεια και την γενίκευση, ειδικά σε σπάνιες κατηγορίες. Παράλληλα, τα μοντέλα απέδειξαν ανθεκτικότητα σε ποικίλες συνθήκες φωτισμού και ποιότητας εικόνας, στοιχείο κρίσιμο για πραγματικές εφαρμογές.

Η υλοποίηση του συστήματος σε συνδυασμό με την ανάπτυξη μιας φιλικής προς οπτικά ανάπηρους χρήστη εφαρμογής Android, με δυνατότητες πλοήγησης μέσω χειρονομιών και φωνητικής ανατροφοδότησης, προσέφερε μια προσιτή και πρακτική λύση για τυφλούς χρήστες. Η χρήση απλών, κατανοητών χειρονομιών, λειτουργιών και η έμφαση στην προσβασιμότητα ανέδειξαν το ανθρωποκεντρικό χαρακτήρα του έργου.

Συμπληρωματικά, η ενσωμάτωση του πίσω μέρους (backend) με την εφαρμογή, μέσω ενός Flask API, διασφάλισε την ομαλή επικοινωνία και την ταχεία απόκριση, ενώ παράλληλα διατήρησε την ευελιξία για μελλοντική φόρτωση των μοντέλων.

Η συνολική εμπειρία ανάπτυξης και ενσωμάτωσης του συστήματος ανέδειξε τις προκλήσεις που συναντώνται όταν επιχειρείται η εφαρμογή τεχνολογιών αιχμής σε κοινωνικά ευαίσθητους τομείς, όπως η προσβασιμότητα. Ταυτόχρονα, κατέδειξε τις δυνατότητες που προσφέρει η τεχνητή νοημοσύνη για την ενίσχυση της αυτονομίας και της ποιότητας ζωής των ατόμων με αναπηρίες, ανοίγοντας δρόμους για μελλοντική έρευνα και καινοτομία στον χώρο της ανθρωπίνης υποστήριξης μέσω τεχνολογίας.

Τέλος, η παρούσα εργασία θέτει τις βάσεις για περαιτέρω εξέλιξη, όντας και η πρώτη έκδοση του παραπάνω έργου, με βλέψεις προς μελλοντικές, ακόμα πιο βελτιωμένες εκδόσεις. Η συνέχιση της προσπάθειας αυτής μπορεί να οδηγήσει στην ανάπτυξη ολοκληρωμένων συστημάτων που θα υποστηρίζουν όχι μόνο την αναγνώριση ρούχων, αλλά και την ευρύτερη καθημερινή αλληλεπίδραση των οπτικά ανάπηρων ατόμων, με αντικείμενα ρουχισμού στο περιβάλλον τους.

ΠΙΝΑΚΑΣ ΟΡΟΛΟΓΙΑΣ

Ξενόγλωσσος όρος	Ελληνικός Όρος
Artificial Intelligence (AI)	Τεχνητή Νοημοσύνη
Deep Learning	Βαθιά Μάθηση
Machine Learning	Μηχανική Μάθηση
Transfer Learning	Μεταφορά Μάθησης
Convolutional Neural Network	Συνελικτικό Νευρωνικό Δίκτυο
Training	Εκπαίδευση
Fine-tuning	Λεπτομερής Επαναρύθμιση
Epoch	Εποχή (κύκλος εκπαίδευσης)
Batch Normalization	Κανονικοποίηση Παρτίδας
Augmentation	Επέκταση / Ενίσχυση Δεδομένων
Loss	Απώλεια
Sparse Categorical Crossentropy	Αραιή Κατηγορική Διασταύρωση Εντροπίας
Optimizer	Βελτιστοποιητής
Learning Rate	Ρυθμός Μάθησης
Early Stopping	Πρόωρος Τερματισμός
Callback	Αναδρομική Κλήση
Checkpoint	Σημείο Ελέγχου
Multi-task	Πολύ-εργασιακό / Πολύ-ετικετικό
Multi-output	Πολυ-έξοδο
Single-task	Μονό-εργασιακό / Μονό-ετικετικό
Single-output	Μονό-έξοδο
Data Generator	Γεννήτρια Δεδομένων
Dataset	Σύνολο Δεδομένων
Validation	Επικύρωση
Test Dataset	Δοκιμή
Classification	Ταξινόμηση
Attribute	Χαρακτηριστικό
Preprocessing	Προεπεξεργασία
Resize	Αλλαγή Μεγέθους
Normalization	Κανονικοποίηση
Application	Εφαρμογή
Gesture	Χειρονομία
Text-to-Speech	Μετατροπή Κειμένου σε Ομιλία
Hyperparameters	Υπερπαράμετροι
Overfitting	Υπερεκπαίδευση/Υπερπροσαρμογή
Generalization	Γενίκευση
Metadata	Μεταδεδομένα
Server	Διακομιστής
Evaluation	Αξιολόγηση
Prediction	Πρόβλεψη
Accessibility	Προσβασιμότητα
Split	Διαχωρισμός
Weights	Βάρη
Regularization	Κανονικοποίηση
Mobile	Κινητό

Application Programming Interface	Διεπαφή Προγραμματισμού Εφαρμογών
Stochastic Gradient Descent	Στοχαστική Κατιούσα Κλίση
Graphics Processing Unit	Μονάδα Επεξεργασίας Γραφικών
Comma-Separated Values	Τιμές Διαχωρισμένες με Κόμματα
HyperText Transfer Protocol	Πρωτόκολλο Μεταφοράς Υπερκειμένου
Neural Network	Νευρωνικό Δίκτυο
Supervised Learning	Εποπτευόμενη Μάθηση
Unsupervised Learning	Μη-Εποπτευόμενη Μάθηση
Gallery	Συλλογή
Fragment	Τμήμα
Swipe	Σύρσιμο

ΣΥΝΤΜΗΣΕΙΣ – ΑΡΚΤΙΚΟΛΕΞΑ – ΑΚΡΩΝΥΜΙΑ

AI	Artificial Intelligence
DL	Deep Learning
ML	Machine Learning
CNN	CNN
API	Application Programming Interface
TTS	Text-to-Speech
SGD	Stochastic Gradient Descent
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
LR	Learning Rate
HTTP	HyperText Transfer Protocol

ΠΑΡΑΡΤΗΜΑ Ι

data_loading.py:

```
import pandas as pd
import numpy as np
from PIL import Image
import os
import json
from sklearn.model_selection import train_test_split

class FashionDatasetLoader:
    generalized_color_map = {
        'blue': ['blue', 'navy blue', 'teal'],
        'black': ['black'],
        'grey': ['grey', 'charcoal', 'steel', 'grey melange', 'silver', 'metallic'],
        'white': ['white', 'off white', 'cream'],
        'beige': ['beige', 'khaki', 'tan', 'skin', 'nude'],
        'brown': ['brown', 'bronze', 'copper', 'rust', 'coffee brown', 'mushroom
brown', 'taupe'],
        'yellow': ['yellow', 'gold', 'mustard'],
        'red': ['red', 'maroon', 'burgundy'],
        'orange': ['orange', 'peach'],
        'green': ['green', 'olive', 'sea green', 'lime green', 'fluorescent green'],
        'purple': ['purple', 'lavender', 'mauve'],
        'pink': ['pink', 'magenta', 'rose'],
        'multi': ['multi'],
        'nan': ['nan'],
        'turquoise': ['turquoise blue'],
    }

    def __init__(self, dataset_path, metadata_file, gen=True):
        self.dataset_path = dataset_path
        if os.path.exists(metadata_file) and os.path.getsize(metadata_file) > 0:
            try:
                self.metadata = pd.read_csv(metadata_file, on_bad_lines='skip') #
Handle malformed rows
            except pd.errors.ParserError as e:
                raise ValueError(f"Error parsing metadata file: {e}")
            # List of subcategories to remove
            for i in ['masterCategory', 'subCategory', 'articleType', 'baseColour',
'gender', 'season', 'usage']:
                self.metadata[i] = self.metadata[i].astype(str).str.strip().str.lower()
            unwanted_subcategories = ['lips', 'nails', 'wallets',
'skin care', 'makeup', 'free gifts', 'skin',
'beauty accessories', 'water bottle', 'eyes', 'bath and body', 'shoe
accessories', 'cufflinks', 'sports equipment', 'hair',
'perfumes', 'home furnishing', 'umbrellas', 'fragrance', 'vouchers']
            unwanted_maincategories = ['personal care', 'home', 'free items']
```

```

        unwanted_articleTypes = ['baby dolls', 'deodorant', 'perfume and body
mist', 'laptop bag', 'trolley bag', 'duffel bag', 'travel accessory',
        'mobile pouch', 'messenger bag', 'accessory gift set', 'tablet sleeve',
'footballs', 'hair colour',
        'cushion covers', 'key chain', 'umbrellas', 'water bottle', 'ipad',
'wallets', 'waist pouch', 'hair accessory', 'cufflinks']
        self.metadata =
self.metadata[~self.metadata['subCategory'].isin(unwanted_subcategories)]
        self.metadata =
self.metadata[~self.metadata['masterCategory'].isin(unwanted_maincategories
)]
        self.metadata =
self.metadata[~self.metadata['articleType'].isin(unwanted_articleTypes)]

```

```

        if gen:
            # Apply generalizations here:
            self.metadata['season'] =
self.metadata['season'].apply(self._generalize_season)
            self.metadata['baseColour'] =
self.metadata['baseColour'].apply(self._generalize_base_color)
        else:
            raise FileNotFoundError(f"Metadata file {metadata_file} not found or
empty")

```

```

def _generalize_season(self, season_value):
    # Group summer and spring as "summer/spring"
    if pd.isna(season_value) or season_value == 'nan':
        return 'nan'
    season_value = season_value.lower().strip()
    if season_value in ['summer', 'spring']:
        return 'summer/spring'
    # keep fall and winter as is
    if season_value in ['fall', 'winter']:
        return season_value
    # fallback - keep original
    return season_value

```

```

def _generalize_base_color(self, base_color_value):
    if pd.isna(base_color_value) or base_color_value == 'nan':
        return 'nan'
    base_color_value = base_color_value.lower().strip()
    # find the generalized key for this base color
    for gen_color, variants in self.generalized_color_map.items():
        if base_color_value in variants:
            return gen_color
    # fallback - if no match, return original
    return base_color_value

```

```

def load_image(self, image_id):
    image_path = os.path.join(self.dataset_path, f"{image_id}.jpg")

```



```

try:
    with Image.open(image_path) as img:
        return img.convert("RGB").copy() # forces full load into memory
except FileNotFoundError:
    print(f"[SKIP] Image not found: {image_path}")
    return None
except Exception as e:
    print(f"[ERROR] Failed to load image {image_path}: {e}")
    return None

def get_metadata(self, image_id):
    return self.metadata[self.metadata['id'] == image_id].iloc[0].to_dict()

def get_all_metadata(self):
    return self.metadata

def filter_by_article_type(self, article_type, max_samples=500):
    filtered = self.metadata[self.metadata['articleType'] == article_type]
    filtered = filtered.sample(min(len(filtered), max_samples))
    return filtered

```

ΠΑΡΑΡΤΗΜΑ II

preprocessing.py:

```
import tensorflow as tf
from sklearn.utils import shuffle
import numpy as np

class DataPreprocessor:
    def __init__(self, target_size_ef=(300, 300), target_size_iv3=(299, 299)):
        self.target_size_ef = target_size_ef
        self.target_size_iv3 = target_size_iv3

    def preprocess_image_ef(self, image):
        # Resize and normalize
        image = image.resize(self.target_size_ef)
        image_array = tf.keras.preprocessing.image.img_to_array(image)
        return image_array

    def preprocess_image_iv3(self, image):
        # Resize and normalize
        image = image.resize(self.target_size_iv3)
        image_array = tf.keras.preprocessing.image.img_to_array(image)
        return image_array

    def create_balanced_dataset(self, metadata, target_column,
samples_per_class=500):
        balanced_df = (
            metadata.groupby(target_column, group_keys=False)
                .apply(lambda x: x.sample(n=min(len(x), samples_per_class),
random_state=42))
                .reset_index(drop=True)
        )
        return balanced_df

    def prepare_ef_dataset(self, image_ids, metadata, loader):
        images = []
        labels = {col: [] for col in ['masterCategory', 'subCategory', 'articleType',
'baseColour', 'gender', 'season', 'usage']}

        for img_id in image_ids:
            img = loader.load_image(img_id)
            if img is None:
                continue

            img_data = self.preprocess_image_ef(img)
            images.append(img_data)
```

```

        row = metadata[metadata['id'] == img_id].iloc[0]
        for col in labels.keys():
            labels[col].append(row[col])

    return np.array(images), labels

def prepare_iv3_dataset(self, image_ids, metadata, loader):
    images = []
    labels = {col: [] for col in ['masterCategory', 'subCategory', 'articleType',
                                   'baseColour', 'gender', 'season', 'usage']}

    for img_id in image_ids:
        img = loader.load_image(img_id)
        if img is None:
            continue

        img_data = self.preprocess_image_iv3(img)
        images.append(img_data)

        row = metadata[metadata['id'] == img_id].iloc[0]
        for col in labels.keys():
            labels[col].append(row[col])

    return np.array(images), labels

```

ΠΑΡΑΡΤΗΜΑ ΙΙΙ

efficientnet_model.py:

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.applications.efficientnet import preprocess_input as
ef_preprocess

class EfficientNetClothingClassifier:
    def __init__(self, num_classes_dict, input_shape=(300, 300, 3)):
        self.num_classes_dict = num_classes_dict
        self.input_shape = input_shape
        self.base_model = EfficientNetB3(
            weights='imagenet',
            include_top=False,
            input_shape=input_shape
        )
        self.models = {}

    def build_model(self):
        # Freeze base model layers
        self.base_model.trainable = False

        # Create input
        inputs = tf.keras.Input(shape=self.input_shape)
        x = ef_preprocess(inputs)
        x = self.base_model(x, training=False)
        x = GlobalAveragePooling2D()(x)
        x = Dropout(0.4)(x)

        # Create output branches for each attribute
        outputs = {}
        for attr, num_classes in self.num_classes_dict.items():
            outputs[attr] = Dense(num_classes, activation='softmax', name=attr,
kernel_regularizer=tf.keras.regularizers.L2(0.01))(x)

        # Create model
        model = Model(inputs=inputs, outputs=outputs)

        # Compile model
        optimizer = SGD(learning_rate=0.001, momentum=0.9)
        losses = {attr: 'sparse_categorical_crossentropy' for attr in
self.num_classes_dict}
        metrics = {attr: 'accuracy' for attr in self.num_classes_dict.keys()}
```

```

print("Model outputs:", model.output_names)
print("Loss keys:", list(losses.keys()))

model.compile(optimizer=optimizer, loss=losses, metrics=metrics)

return model

def fine_tune(self, model,
unfreeze_from_layer_name='block5e_expand_conv'):
    # Unfreeze from specified layer, keep BatchNorm frozen
    unfreeze = False
    for layer in self.base_model.layers:
        if layer.name == unfreeze_from_layer_name:
            unfreeze = True
        if unfreeze:
            if not isinstance(layer, tf.keras.layers.BatchNormalization):
                layer.trainable = True
            else:
                layer.trainable = False
        else:
            layer.trainable = False

    # Ensure all layers outside the base model (i.e., heads) are trainable
    for layer in model.layers:
        if layer not in self.base_model.layers:
            layer.trainable = True

    model.trainable = True # Important safety net

    # Recompile model
    optimizer = SGD(learning_rate=0.0001, momentum=0.9) # Lower
learning rate for fine-tuning
    losses = {attr: 'sparse_categorical_crossentropy' for attr in
self.num_classes_dict}
    loss_weights = {
        'articleType': 1.2,
        'subCategory': 0.8,
        'baseColour': 1.5,
        'season': 1.5,
        'usage': 1.0,
        'masterCategory': 0.7,
        'gender': 1.0
    }
    metrics = {attr: 'accuracy' for attr in self.num_classes_dict.keys()}

    model.compile(optimizer=optimizer, loss=losses,
loss_weights=loss_weights, metrics=metrics)

    return model

```

ΠΑΡΑΡΤΗΜΑ IV

inceptionv3_model.py:

```
import tensorflow as tf
from tensorflow.keras.applications.inception_v3 import InceptionV3,
preprocess_input as iv3_preprocess
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.optimizers import SGD

class IV3ClothingClassifier:
    def __init__(self, num_classes_dict, input_shape=(299, 299, 3)):
        self.num_classes_dict = num_classes_dict
        self.input_shape = input_shape
        self.base_model = InceptionV3( # Load pre-trained InceptionV3
            weights='imagenet',        # Pre-trained on ImageNet
            include_top=False,         # Remove original classification head
            input_shape=self.input_shape
        )

    def build_model(self):
        # Freeze base model layers
        self.base_model.trainable = False

        # Explicitly create input layer
        inputs = tf.keras.Input(shape=self.input_shape)
        x = iv3_preprocess(inputs)
        x = self.base_model(x, training=False)
        x = GlobalAveragePooling2D()(x)
        x = Dropout(0.4)(x)

        # Create multi-output branches
        outputs = {}
        for attr, num_classes in self.num_classes_dict.items():
            outputs[attr] = Dense(num_classes, activation='softmax', name=attr,
kernel_regularizer=tf.keras.regularizers.l2(0.01))(x)

        model = Model(inputs=inputs, outputs=outputs)

        # Compile
        optimizer = SGD(learning_rate=0.001, momentum=0.9)
        losses = {attr: 'sparse_categorical_crossentropy' for attr in
self.num_classes_dict}
        metrics = {attr: 'accuracy' for attr in self.num_classes_dict.keys()}

        model.compile(optimizer=optimizer, loss=losses, metrics=metrics)
        return model
```

```

def fine_tune(self, model, unfreeze_from_layer_name='mixed7'):
    # Unfreeze from specific layer onward in base model
    unfreeze = False
    for layer in self.base_model.layers:
        if layer.name == unfreeze_from_layer_name:
            unfreeze = True
        if unfreeze:
            if not isinstance(layer, tf.keras.layers.BatchNormalization):
                layer.trainable = True
            else:
                layer.trainable = False
        else:
            layer.trainable = False

    # Ensure all layers outside the base model (i.e., heads) are trainable
    for layer in model.layers:
        if layer not in self.base_model.layers:
            layer.trainable = True

    model.trainable = True # Important safety net

    # Lower learning rate for fine-tuning
    optimizer = SGD(learning_rate=0.0001, momentum=0.9)
    losses = {attr: 'sparse_categorical_crossentropy' for attr in
self.num_classes_dict}
    loss_weights = {
        'articleType': 1.2,
        'subCategory': 0.8,
        'baseColour': 1.5,
        'season': 1.5,
        'usage': 1.0,
        'masterCategory': 0.7,
        'gender': 1.0
    }
    metrics = {attr: 'accuracy' for attr in self.num_classes_dict.keys()}

    model.compile(optimizer=optimizer, loss=losses,
loss_weights=loss_weights, metrics=metrics)
    return model

```

ΠΑΡΑΡΤΗΜΑ V

articleType_efnet_model.py:

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3, EfficientNetB0,
EfficientNetB1
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.applications.efficientnet import preprocess_input as
ef_preprocess
from tensorflow.keras.layers import Resizing

class EfficientNetArtTypeClothingClassifier:
    def __init__(self, num_classes, model_name, input_shape=(300, 300, 3)):
        self.num_classes = num_classes
        self.model_name = model_name
        self.input_shape = input_shape
        if model_name == 'efficientnetB0_artType':
            self.base_model = EfficientNetB0(
                weights='imagenet',
                include_top=False,
                input_shape=input_shape
            )
        elif model_name == 'efficientnetB1_artType':
            self.base_model = EfficientNetB1(
                weights='imagenet',
                include_top=False,
                input_shape=input_shape
            )
        else:
            self.base_model = EfficientNetB3(
                weights='imagenet',
                include_top=False,
                input_shape=input_shape
            )

    def build_fine_tuned_model(self, resume=False):
        # Unfreeze all layers except BatchNorm
        for layer in self.base_model.layers:
            if isinstance(layer, tf.keras.layers.BatchNormalization):
                layer.trainable = False
            else:
                layer.trainable = True

        target_size = (224, 224) if self.model_name == 'efficientnetB0_artType'
    else \
        (240, 240) if self.model_name == 'efficientnetB1_artType' else \
```


None

```
# Create input
inputs = tf.keras.Input(shape=self.input_shape)
x = inputs
if target_size:
    x = Resizing(*target_size)(inputs)
x = ef_preprocess(x)
x = self.base_model(x, training=True)
x = GlobalAveragePooling2D()(x)
if self.model_name == 'efficientnetB0_artType':
    x = Dropout(0.2)(x)
elif self.model_name == 'efficientnetB1_artType':
    x = Dropout(0.3)(x)
else:
    x = Dropout(0.4)(x)
if self.model_name == 'efficientnetB0_artType':
    output = Dense(self.num_classes, activation='softmax',
name='articleType')(x)
elif self.model_name == 'efficientnetB1_artType':
    output = Dense(self.num_classes, activation='softmax',
name='articleType')(x)
else:
    output = Dense(self.num_classes, activation='softmax',
name='articleType', kernel_regularizer=tf.keras.regularizers.l2(0.01))(x)

# Create model
model = Model(inputs=inputs, outputs=output)
optimizer = SGD(learning_rate=0.001, momentum=0.9)

model.compile(optimizer=optimizer,
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
print("Model output:", model.output_names)

return model
```

ΠΑΡΑΡΤΗΜΑ VI

training.py:

```
import tensorflow as tf
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import json
import os
import numpy as np
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
import tensorflow as tf
import os
import json
import numpy as np
```

```
class SaveTrainingStateCallback(tf.keras.callbacks.Callback):
```

```
    def __init__(self, trainer, save_dir_resume, model_name):
        super().__init__()
        self.trainer = trainer
        self.save_dir_resume = save_dir_resume
        self.model_name = model_name
        self.history = {}
```

```
    def on_epoch_end(self, epoch, logs=None):
        # Update history
        for k, v in (logs or {}).items():
            self.history.setdefault(k, []).append(float(v))
```

```
        # Save history
        history_path = os.path.join(self.save_dir_resume,
f'{self.model_name}_training_history.json')
        with open(history_path, 'w') as f:
            json.dump(self.history, f)
```

```
        # Load existing state if available
        state_path = os.path.join(self.save_dir_resume,
f'{self.model_name}_training_state.json')
        if os.path.exists(state_path):
            with open(state_path, 'r') as f:
                existing_state = json.load(f)
                previous_epoch = existing_state.get('last_epoch', -1)
        else:
            previous_epoch = -1
```

```
        # Update only if current epoch is newer
        if epoch > previous_epoch:
```

```

        state = {'last_epoch': epoch}
        with open(state_path, 'w') as f:
            json.dump(state, f)

    # Save label encoders
    for attr, encoder in self.trainer.label_encoders.items():
        np.save(os.path.join(self.save_dir_resume, f'{attr}_classes.npy'),
encoder.classes_)
        print(" Saving label encoders:", list(self.trainer.label_encoders.keys()))

    # Save model weights
    self.trainer.model.save_weights(os.path.join(self.save_dir_resume,
f'{self.model_name}_final_weights.weights.h5'))

class MultiOutputDataGenerator(tf.keras.utils.Sequence):
    def __init__(self, X, y_dict, batch_size=32, shuffle=True):
        self.X = X
        self.y_dict = y_dict
        self.batch_size = batch_size
        self.shuffle = shuffle
        self.indices = np.arange(len(X))
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.X) / self.batch_size))

    def __getitem__(self, index):
        batch_indices =
self.indices[index*self.batch_size:(index+1)*self.batch_size]
        batch_X = self.X[batch_indices]
        batch_y = {key: val[batch_indices] for key, val in self.y_dict.items()}
        print(f"Batch {index}: X shape {batch_X.shape}, y keys:
{list(batch_y.keys())}, y[0] shape: {batch_y[next(iter(batch_y))].shape}")
        return batch_X, batch_y

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)

class ClothingClassifierTrainer:
    def __init__(self, model, model_name, save_dir='.', save_dir_resume='.'):
        self.model = model
        self.model_name = model_name
        self.save_dir = save_dir
        self.save_dir_resume = save_dir_resume
        self.label_encoders = {}
        if model_name == 'efficientnetB0_artType':
            self.augmentor = ImageDataGenerator(
                horizontal_flip=True
            )

```

```

elif model_name == 'efficientnetB1_artType':
    self.augmentor = ImageDataGenerator(
        horizontal_flip=True,
        rotation_range=10,
        zoom_range=0.1,
        width_shift_range=0.1,
        height_shift_range=0.1,
    )
else:
    self.augmentor = ImageDataGenerator(
        horizontal_flip=True,
        brightness_range=[0.8, 1.2],
        zoom_range=0.1,
        rotation_range=15,
        width_shift_range=0.05,
        height_shift_range=0.05,
        shear_range=0.05
    )

def fit_label_encoders(self, *label_dicts):
    all_attrs = label_dicts[0].keys()
    for attr in all_attrs:
        combined = []
        for d in label_dicts:
            if d and attr in d:
                combined += list(d[attr])
        self.label_encoders[attr] = LabelEncoder()
        self.label_encoders[attr].fit(combined)

def encode_labels(self, labels_dict):
    """Transform labels using already fitted encoders."""
    encoded_labels = {}
    for attr, labels in labels_dict.items():
        if attr not in self.label_encoders:
            raise ValueError(f"Label encoder for '{attr}' not fitted yet.")
        encoded_labels[attr] =
np.array(self.label_encoders[attr].transform(labels))
    return encoded_labels

def save_training_state(self, last_epoch):
    state = {'last_epoch': last_epoch}
    with open(os.path.join(self.save_dir,
f'{self.model_name}_training_state.json'), 'w') as f:
        json.dump(state, f)

def load_training_state(self):
    state_path = os.path.join(self.save_dir_resume,
f'{self.model_name}_training_state.json')
    if os.path.exists(state_path):

```

```

        with open(state_path, 'r') as f:
            state = json.load(f)
            return state.get('last_epoch', -1)
    else:
        return -1

    def train(self, X_train, y_train, X_val, y_val, batch_size=16, epochs=50,
              X_test=None, y_test=None, resume_training=False, fit_label_encoders=True,
              all_labels=None):

        if resume_training:
            print("Resuming training...")
            # Load weights if they exist
            weights_path = os.path.join(self.save_dir_resume,
                                         f'{self.model_name}_final_weights.weights.h5')
            if os.path.exists(weights_path):
                self.model.load_weights(weights_path)
                print(f"Loaded weights from {weights_path}")
            else:
                print("No saved weights found, starting fresh.")

            # Load label encoders
            for attr in y_train.keys():
                classes_path = os.path.join(self.save_dir_resume,
                                             f'{attr}_classes.npy')
                if os.path.exists(classes_path):
                    classes = np.load(classes_path, allow_pickle=True)
                    encoder = LabelEncoder()
                    encoder.classes_ = classes
                    self.label_encoders[attr] = encoder
                else:
                    raise FileNotFoundError(f"Label encoder classes for {attr} not
                    found at {classes_path}")

            last_epoch = self.load_training_state()
            initial_epoch = last_epoch + 1
            print(f"Resuming from epoch {initial_epoch}")

            # Load previous training history
            history_path = os.path.join(self.save_dir_resume,
                                         f'{self.model_name}_training_history.json')
            if os.path.exists(history_path):
                with open(history_path, 'r') as f:
                    previous_history = json.load(f)
            else:
                previous_history = {}

        else:
            print("Starting fresh training...")
            # Fit label encoders anew

```

```

    if fit_label_encoders:
        if all_labels:
            self.fit_label_encoders(all_labels)
        else:
            self.fit_label_encoders(y_train)
    initial_epoch = 0
    previous_history = {}

# Encode labels
y_train_encoded = self.encode_labels(y_train)
y_val_encoded = self.encode_labels(y_val)

print("Encoded y_train keys and shapes:")
for k, v in y_train_encoded.items():
    print(f'{k}: {v.shape}')

print("Encoded y_val keys and shapes:")
for k, v in y_val_encoded.items():
    print(f'{k}: {v.shape}')

# Convert encoded labels dict into a NumPy array for multi-output (if
needed)
# For training generator
y_train_array = {attr: y_train_encoded[attr] for attr in y_train_encoded}

# For validation generator
y_val_array = {attr: y_val_encoded[attr] for attr in y_val_encoded}

# Create data generators
train_gen = MultiOutputDataGenerator(X_train, y_train_array,
batch_size=batch_size)
val_gen = MultiOutputDataGenerator(X_val, y_val_array,
batch_size=batch_size, shuffle=False)

# Callbacks
callbacks = [
    ModelCheckpoint(
        os.path.join(self.save_dir,
f'{self.model_name}_best_weights.weights.h5'),
        monitor='val_loss',
        save_best_only=True,
        save_weights_only=True
    ),
    EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(

```

```

        monitor='val_loss',
        factor=0.5,
        patience=3,
        verbose=1,
        min_lr=1e-6
    ),
    SaveTrainingStateCallback(self, self.save_dir_resume,
self.model_name)
]

x0, y0 = train_gen[0]
print("Sanity Check Shapes:")
print("X:", x0.shape, x0.dtype)
for k, v in y0.items():
    print(f'{k}: {v.shape}, {v.dtype}')

# Train model
history = self.model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=epochs,
    callbacks=callbacks,
    initial_epoch=initial_epoch,
    verbose=1
)

print("Training History Keys:")
print(history.history.keys())

# Merge current history with previous history
for key in previous_history:
    if key in history.history:
        history.history[key] = previous_history[key] + history.history[key]

# Save updated history to disk
with open(os.path.join(self.save_dir,
f'{self.model_name}_training_history.json'), 'w') as f:
    json.dump(history.history, f)

# Find a valid key to estimate how many epochs were completed
key = next((k for k in history.history if k.endswith('_loss')), None)

# Fallback to generic 'loss' if no detailed loss keys found
if key is None and 'loss' in history.history:
    key = 'loss'

# Raise error only if no loss information is present
if key is None:

```

```

        raise ValueError(f"No loss keys found in training history. Got keys:
{list(history.history.keys())}")

    self.save_training_state(initial_epoch + len(history.history[key]) - 1)

    return history

def save_model(self, save_dir=None):
    if save_dir is None:
        save_dir = self.save_dir
    # Save model weights
    self.model.save_weights(os.path.join(save_dir,
f'{self.model_name}_final_weights.weights.h5'))

    # Ensure best weights have already been saved by ModelCheckpoint
    during training
    best_weights_path = os.path.join(save_dir,
f'{self.model_name}_best_weights.weights.h5')
    if not os.path.exists(best_weights_path):
        print("Best weights not found. Only final weights were saved.")
    else:
        print("Best weights already saved during training.")

    # Save label encoders
    for attr, encoder in self.label_encoders.items():
        np.save(os.path.join(save_dir, f'{attr}_classes.npy'), encoder.classes_)

    # Save model architecture if needed
    with open(os.path.join(save_dir, f'{self.model_name}_architecture.json'),
'w') as f:
        f.write(self.model.to_json())

    @staticmethod
    def load_model(model_name, model_arch, save_dir, best_weights=False,
training=False, only_artType=False):
        # Load model architecture
        model = model_arch

        # Load weights
        # Decide which weights to load
        weights_filename = f'{model_name}_best_weights.weights.h5' if
best_weights else f'{model_name}_final_weights.weights.h5'
        weights_path = os.path.join(save_dir, weights_filename)

        if os.path.exists(weights_path):
            if training:
                model.load_weights(weights_path, by_name=True,
skip_mismatch=True)
            else:
                model.load_weights(weights_path)

```



```

        print(f" Loaded {'best' if best_weights else 'final'} weights from
{weights_path}")
    else:
        raise FileNotFoundError(f" ❌ Weights file not found: {weights_path}")

# Load label encoders
label_encoders = {}
if only_artType:
    # Only load articleType encoder
    attr = 'articleType'
    classes = np.load(os.path.join(save_dir, f'{attr}_classes.npy'))
    encoder = LabelEncoder()
    encoder.classes_ = classes
    label_encoders[attr] = encoder
    return model, label_encoders
else:
    for attr in ['masterCategory', 'subCategory', 'articleType',
                'baseColour', 'gender', 'season', 'usage']:
        classes = np.load(os.path.join(save_dir, f'{attr}_classes.npy'))
        encoder = LabelEncoder()
        encoder.classes_ = classes
        label_encoders[attr] = encoder

return model, label_encoders

```

ΠΑΡΑΡΤΗΜΑ VII

evaluation.py:

```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import time
from sklearn.metrics import top_k_accuracy_score

class ModelEvaluator:
    def __init__(self, model, label_encoders):
        self.model = model
        self.label_encoders = label_encoders

    def evaluate(self, X_test, y_test, history=None, training_time=None,
num_epochs=None):

        # Handle single-task y_test (e.g., a 1D array) by wrapping it
        if not isinstance(y_test, dict):
            y_test = {'articleType': y_test}
            is_single_task = True
        else:
            is_single_task = len(y_test) == 1

        # Encode test labels
        y_test_encoded = {}
        for attr, labels in y_test.items():
            y_test_encoded[attr] = self.label_encoders[attr].transform(labels)

        # Measure inference time
        start_time = time.time()
        y_pred_raw = self.model.predict(X_test, batch_size=32)
        inference_duration = time.time() - start_time
        avg_inference_time = inference_duration / len(X_test)

        # Handle dict or list prediction outputs
        if isinstance(y_pred_raw, dict):
            y_pred = y_pred_raw
        elif isinstance(y_pred_raw, list):
            if len(y_pred_raw) != len(y_test):
                raise ValueError("Number of model outputs does not match number
of attributes.")
            y_pred = {attr: preds for attr, preds in zip(y_test.keys(), y_pred_raw)}
        elif isinstance(y_pred_raw, np.ndarray):
            if is_single_task:
                y_pred = {'articleType': y_pred_raw}
            else:
```

```

        raise TypeError("Model output is ndarray but multiple outputs
expected.")
    else:
        raise TypeError("Model prediction output must be dict or list.")

    results = {}

    # Evaluate each attribute
    for attr in y_test.keys():
        if attr not in y_pred:
            raise ValueError(f"Missing predictions for attribute: {attr}")

        true_labels = y_test_encoded[attr]
        pred_probs = y_pred[attr]
        pred_labels = np.argmax(pred_probs, axis=1)

        unique_in_test = np.unique(true_labels)
        total_classes = len(self.label_encoders[attr].classes_)
        if len(unique_in_test) < total_classes:
            print(f" Attribute '{attr}': Only {len(unique_in_test)}/{total_classes}
classes present in test set.")

    # Classification report
    report = classification_report(
        true_labels,
        pred_labels,
        labels=range(total_classes), # Ensures full label set is used
        target_names=self.label_encoders[attr].classes_,
        output_dict=True,
        zero_division=0 # Prevents divide-by-zero warnings
    )

    # Confusion matrix
    cm = confusion_matrix(true_labels, pred_labels)

    # Top-k accuracy
    if total_classes > 3:
        top3_acc = top_k_accuracy_score(true_labels, pred_probs, k=3,
labels=range(total_classes))
    else:
        top3_acc = None # Too few classes for top-3 to be meaningful

    if total_classes > 5:
        top5_acc = top_k_accuracy_score(true_labels, pred_probs, k=5,
labels=range(total_classes))
    else:
        top5_acc = None # Too few classes for top-5 to be meaningful

    results[attr] = {
        'report': report,

```

```

        'confusion_matrix': cm,
        'top3_accuracy': top3_acc,
        'top5_accuracy': top5_acc
    }

# Add training history if available
if history:
    results['training'] = {
        'train_accuracy': history.history.get('accuracy', [0,0])[-1],
        'val_accuracy': history.history.get('val_accuracy', [0,0])[-1],
        'train_loss': history.history.get('loss', [0,0])[-1],
        'val_loss': history.history.get('val_loss', [0,0])[-1]
    }
if training_time:
    results['training_time'] = training_time
if num_epochs:
    results['num_epochs'] = num_epochs
results['avg_inference_time'] = avg_inference_time

return results

def print_summary(self, results):
    # Print training summary
    if 'training' in results:
        print("\n=== Training Summary ===")
        for k, v in results['training'].items():
            print(f"{k.replace('_', ' ').title():<20}<10>: {v:.4f}" if isinstance(v, float) else
f"{k.replace('_', ' ').title():<20}<10>: {v}")

    if 'avg_inference_time' in results:
        print(f"\nAverage inference time per sample:
{results['avg_inference_time']:.6f} seconds")

    # Print evaluation per attribute
    for attr, metrics in results.items():
        if attr in ['training', 'training_time', 'num_epochs', 'avg_inference_time']:
            continue

        report = metrics['report']
        print(f"\n=== Attribute: {attr.upper()} ===")
        print(f"Accuracy: {report['accuracy']:.4f}")
        print("Precision / Recall / F1 (macro):",
            f"{report['macro avg']['precision']:.4f} /",
            f"{report['macro avg']['recall']:.4f} /",
            f"{report['macro avg']['f1-score']:.4f}")
        if metrics.get('top3_accuracy') is not None:
            print(f"Top-3 Accuracy: {metrics['top3_accuracy']:.4f}")
        if metrics.get('top5_accuracy') is not None:
            print(f"Top-5 Accuracy: {metrics['top5_accuracy']:.4f}")

```

```

def plot_confusion_matrix(self, cm, classes, title='Confusion Matrix'):
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=classes, yticklabels=classes)
    plt.title(title)
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.show()

def plot_all_confusion_matrices(self, results):
    for attr, metrics in results.items():
        if attr in ['training', 'training_time', 'num_epochs', 'avg_inference_time']:
            continue
        cm = metrics.get('confusion_matrix')
        classes = self.label_encoders[attr].classes_
        title = f"Confusion Matrix for {attr.upper()}"
        self.plot_confusion_matrix(cm, classes, title)

```

ΠΑΡΑΡΤΗΜΑ VIII

prediction.py:

```
import numpy as np
from PIL import Image
import tensorflow as tf
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.applications.efficientnet import preprocess_input as
ef_preprocess

class ClothingAttributePredictor:

    def __init__(self, model, label_encoders, model_name):
        self.model = model
        self.label_encoders = label_encoders
        self.model_name = model_name

    def preprocess_image_from_pil(self, img, target_size=(300, 300)):
        if self.model_name == 'EffNet_artTypeB1':
            target_size = (240, 240)
        img = img.resize(target_size)
        img_array = tf.keras.preprocessing.image.img_to_array(img)
        img_array = ef_preprocess(img_array)
        img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

        return img_array

    def predict_attributes_from_pil(self, img):
        img_array = self.preprocess_image_from_pil(img)
        predictions = self.model.predict(img_array)
        result = {}
        if isinstance(predictions, dict): # Multi-task model
            for attr, pred_array in predictions.items():
                pred_class = np.argmax(pred_array[0])
                class_name = self.label_encoders[attr].classes_[pred_class]
                result[attr] = class_name
        else: # Single-task model
            attr = 'articleType'
            pred_class = np.argmax(predictions[0])
            class_name = self.label_encoders[attr].classes_[pred_class]
            result[attr] = class_name

        return result
```

ΠΑΡΑΡΤΗΜΑ IX

api_server.py:

```
from flask import Flask, request, jsonify
import io
from PIL import Image
import logging
import traceback
import pandas as pd

from prediction import ClothingAttributePredictor

# Your model and training imports
from training import ClothingClassifierTrainer
from efficientnet_model import EfficientNetClothingClassifier
from articleType_efnet_model import EfficientNetArtTypeClothingClassifier

app = Flask(__name__)
logging.basicConfig(level=logging.INFO)

# Load balanced metadata (adjust path as needed)
balanced_metadata = pd.read_parquet('insert parquet directory
here/filtered_balanced_dataset_ef_2.parquet')

num_classes_dict = {
    attr: len(balanced_metadata[attr].unique())
    for attr in ['masterCategory', 'subCategory', 'articleType',
                'baseColour', 'gender', 'season', 'usage']
}

def load_multi_task_model():
    logging.info("Loading multi-task model (EffNet_Fashion_3)...")
    multi_model_arch =
EfficientNetClothingClassifier(num_classes_dict).build_model()
    multi_model_dir = '../Saved_Models/EffNet_Fashion_3/fine_tuned/final'
    multi_model, multi_label_encoders = ClothingClassifierTrainer.load_model(
        model_name='efficientnet',
        model_arch=multi_model_arch,
        save_dir=multi_model_dir,
        best_weights=False
    )
    return multi_model, multi_label_encoders

def load_single_task_model():
    logging.info("Loading single-task articleType model (EffNet_artTypeB1)...")
```

```

    single_model_arch = EfficientNetArtTypeClothingClassifier(87,
'efficientnetB1_artType', (240, 240, 3)).build_fine_tuned_model()
    single_model_dir = '../Saved_Models/EffNet_artTypeB1/final'
    single_model, single_label_encoders =
ClothingClassifierTrainer.load_model(
    model_name='efficientnetB1_artType',
    model_arch=single_model_arch,
    save_dir=single_model_dir,
    best_weights=True,
    only_artType=True
)
    return single_model, single_label_encoders

# Load models once on startup
multi_model, multi_label_encoders = load_multi_task_model()
single_model, single_label_encoders = load_single_task_model()

multi_predictor = ClothingAttributePredictor(multi_model,
multi_label_encoders, 'EffNet_Fashion_3')
single_predictor = ClothingAttributePredictor(single_model,
single_label_encoders, 'EffNet_artTypeB1')

@app.route('/', methods=['GET'])
def home():
    return "Flask server is running and reachable!"

@app.route('/predict', methods=['POST'])
def predict():
    print("=== /predict POST received ===")
    try:
        if 'image' not in request.files:
            return jsonify({'status': 'error', 'message': 'No image provided'}), 400

        image_file = request.files['image']
        print(f"Received image filename: {image_file.filename}")
        print(f"Content-Type: {image_file.content_type}")
        print(f"Content-Length: {request.content_length}")
        image_bytes = image_file.read()

        try:
            image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
        except Exception as e:
            logging.error(f"Failed to open image: {e}")
            return jsonify({'status': 'error', 'message': 'Invalid image format'}), 400

        # Predict articleType with single-task model
        article_type_pred = single_predictor.predict_attributes_from_pil(image)
        article_type_value = article_type_pred.get('articleType')

        # Predict other attributes with multi-task model

```



```

fashion_preds = multi_predictor.predict_attributes_from_pil(image)

# Remove articleType from multi-task predictions to avoid duplication
fashion_preds.pop('articleType', None)

# Combine results: articleType from single-task + other attrs from multi-
task
combined_preds = {**fashion_preds, 'articleType': article_type_value}

print(f"Prediction results sent: {combined_preds}")

return jsonify({'status': 'success', 'predictions': combined_preds})

except Exception as e:
    logging.error(f"Unexpected error: {e}\n{traceback.format_exc()}")
    return jsonify({'status': 'error', 'message': 'Internal server error'}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, threaded=True)

```

ΠΑΡΑΡΤΗΜΑ Χ

Αρχιτεκτονική Επιπέδων (Layers) Βάσης EfficientNetB3, χωρίς κεφαλή:

Total layers in EfficientNetB3: 385

0 input_layer_1	31 block2a_expand_bn	62 block2c_dwconv
1 rescaling_2	32 block2a_expand_activation	63 block2c_bn
2 normalization_1	33 block2a_dwconv_pad	64 block2c_activation
3 rescaling_3	34 block2a_dwconv	65 block2c_se_squeeze
4 stem_conv_pad	35 block2a_bn	66 block2c_se_reshape
5 stem_conv	36 block2a_activation	67 block2c_se_reduce
6 stem_bn	37 block2a_se_squeeze	68 block2c_se_expand
7 stem_activation	38 block2a_se_reshape	69 block2c_se_excite
8 block1a_dwconv	39 block2a_se_reduce	70 block2c_project_conv
9 block1a_bn	40 block2a_se_expand	71 block2c_project_bn
10 block1a_activation	41 block2a_se_excite	72 block2c_drop
11 block1a_se_squeeze	42 block2a_project_conv	73 block2c_add
12 block1a_se_reshape	43 block2a_project_bn	74 block3a_expand_conv
13 block1a_se_reduce	44 block2b_expand_conv	75 block3a_expand_bn
14 block1a_se_expand	45 block2b_expand_bn	76 block3a_expand_activation
15 block1a_se_excite	46 block2b_expand_activation	77 block3a_dwconv_pad
16 block1a_project_conv	47 block2b_dwconv	78 block3a_dwconv
17 block1a_project_bn	48 block2b_bn	79 block3a_bn
18 block1b_dwconv	49 block2b_activation	80 block3a_activation
19 block1b_bn	50 block2b_se_squeeze	81 block3a_se_squeeze
20 block1b_activation	51 block2b_se_reshape	82 block3a_se_reshape
21 block1b_se_squeeze	52 block2b_se_reduce	83 block3a_se_reduce
22 block1b_se_reshape	53 block2b_se_expand	84 block3a_se_expand
23 block1b_se_reduce	54 block2b_se_excite	85 block3a_se_excite
24 block1b_se_expand	55 block2b_project_conv	86 block3a_project_conv
25 block1b_se_excite	56 block2b_project_bn	87 block3a_project_bn
26 block1b_project_conv	57 block2b_drop	88 block3b_expand_conv
27 block1b_project_bn	58 block2b_add	89 block3b_expand_bn
28 block1b_drop	59 block2c_expand_conv	90 block3b_expand_activation
29 block1b_add	60 block2c_expand_bn	91 block3b_dwconv
30 block2a_expand_conv	61 block2c_expand_activation	92 block3b_bn

93 block3b_activation	124 block4a_activation	155 block4c_se_reduce
94 block3b_se_squeeze	125 block4a_se_squeeze	156 block4c_se_expand
95 block3b_se_reshape	126 block4a_se_reshape	157 block4c_se_excite
96 block3b_se_reduce	127 block4a_se_reduce	158 block4c_project_conv
97 block3b_se_expand	128 block4a_se_expand	159 block4c_project_bn
98 block3b_se_excite	129 block4a_se_excite	160 block4c_drop
99 block3b_project_conv	130 block4a_project_conv	161 block4c_add
100 block3b_project_bn	131 block4a_project_bn	162 block4d_expand_conv
101 block3b_drop	132 block4b_expand_conv	163 block4d_expand_bn
102 block3b_add	133 block4b_expand_bn	164 block4d_expand_activation
103 block3c_expand_conv	134 block4b_expand_activation	165 block4d_dwconv
104 block3c_expand_bn	135 block4b_dwconv	166 block4d_bn
105 block3c_expand_activation	136 block4b_bn	167 block4d_activation
106 block3c_dwconv	137 block4b_activation	168 block4d_se_squeeze
107 block3c_bn	138 block4b_se_squeeze	169 block4d_se_reshape
108 block3c_activation	139 block4b_se_reshape	170 block4d_se_reduce
109 block3c_se_squeeze	140 block4b_se_reduce	171 block4d_se_expand
110 block3c_se_reshape	141 block4b_se_expand	172 block4d_se_excite
111 block3c_se_reduce	142 block4b_se_excite	173 block4d_project_conv
112 block3c_se_expand	143 block4b_project_conv	174 block4d_project_bn
113 block3c_se_excite	144 block4b_project_bn	175 block4d_drop
114 block3c_project_conv	145 block4b_drop	176 block4d_add
115 block3c_project_bn	146 block4b_add	177 block4e_expand_conv
116 block3c_drop	147 block4c_expand_conv	178 block4e_expand_bn
117 block3c_add	148 block4c_expand_bn	179 block4e_expand_activation
118 block4a_expand_conv	149 block4c_expand_activation	180 block4e_dwconv
119 block4a_expand_bn	150 block4c_dwconv	181 block4e_bn
120 block4a_expand_activation	151 block4c_bn	182 block4e_activation
121 block4a_dwconv_pad	152 block4c_activation	183 block4e_se_squeeze
122 block4a_dwconv	153 block4c_se_squeeze	184 block4e_se_reshape
123 block4a_bn	154 block4c_se_reshape	185 block4e_se_reduce

186 block4e_se_expand	222 block5c_expand_activation	258 block5e_se_reduce
187 block4e_se_excite	223 block5c_dwconv	259 block5e_se_expand
188 block4e_project_conv	224 block5c_bn	260 block5e_se_excite
189 block4e_project_bn	225 block5c_activation	261 block5e_project_conv
190 block4e_drop	226 block5c_se_squeeze	262 block5e_project_bn
191 block4e_add	227 block5c_se_reshape	263 block5e_drop
192 block5a_expand_conv	228 block5c_se_reduce	264 block5e_add
193 block5a_expand_bn	229 block5c_se_expand	265 block6a_expand_conv
194 block5a_expand_activation	230 block5c_se_excite	266 block6a_expand_bn
195 block5a_dwconv	231 block5c_project_conv	267 block6a_expand_activation
196 block5a_bn	232 block5c_project_bn	268 block6a_dwconv_pad
197 block5a_activation	233 block5c_drop	269 block6a_dwconv
198 block5a_se_squeeze	234 block5c_add	270 block6a_bn
199 block5a_se_reshape	235 block5d_expand_conv	271 block6a_activation
200 block5a_se_reduce	236 block5d_expand_bn	272 block6a_se_squeeze
201 block5a_se_expand	237 block5d_expand_activation	273 block6a_se_reshape
202 block5a_se_excite	238 block5d_dwconv	274 block6a_se_reduce
203 block5a_project_conv	239 block5d_bn	275 block6a_se_expand
204 block5a_project_bn	240 block5d_activation	276 block6a_se_excite
205 block5b_expand_conv	241 block5d_se_squeeze	277 block6a_project_conv
206 block5b_expand_bn	242 block5d_se_reshape	278 block6a_project_bn
207 block5b_expand_activation	243 block5d_se_reduce	279 block6b_expand_conv
208 block5b_dwconv	244 block5d_se_expand	280 block6b_expand_bn
209 block5b_bn	245 block5d_se_excite	281 block6b_expand_activation
210 block5b_activation	246 block5d_project_conv	282 block6b_dwconv
211 block5b_se_squeeze	247 block5d_project_bn	283 block6b_bn
212 block5b_se_reshape	248 block5d_drop	284 block6b_activation
213 block5b_se_reduce	249 block5d_add	285 block6b_se_squeeze
214 block5b_se_expand	250 block5e_expand_conv	286 block6b_se_reshape
215 block5b_se_excite	251 block5e_expand_bn	287 block6b_se_reduce
216 block5b_project_conv	252 block5e_expand_activation	288 block6b_se_expand
217 block5b_project_bn	253 block5e_dwconv	289 block6b_se_excite
218 block5b_drop	254 block5e_bn	290 block6b_project_conv
219 block5b_add	255 block5e_activation	291 block6b_project_bn
220 block5c_expand_conv	256 block5e_se_squeeze	292 block6b_drop
221 block5c_expand_bn	257 block5e_se_reshape	293 block6b_add

294 block6c_expand_conv	330 block6e_se_squeeze	366 block7a_project_bn
295 block6c_expand_bn	331 block6e_se_reshape	367 block7b_expand_conv
296 block6c_expand_activation	332 block6e_se_reduce	368 block7b_expand_bn
297 block6c_dwconv	333 block6e_se_expand	369 block7b_expand_activation
298 block6c_bn	334 block6e_se_excite	370 block7b_dwconv
299 block6c_activation	335 block6e_project_conv	371 block7b_bn
300 block6c_se_squeeze	336 block6e_project_bn	372 block7b_activation
301 block6c_se_reshape	337 block6e_drop	373 block7b_se_squeeze
302 block6c_se_reduce	338 block6e_add	374 block7b_se_reshape
303 block6c_se_expand	339 block6f_expand_conv	375 block7b_se_reduce
304 block6c_se_excite	340 block6f_expand_bn	376 block7b_se_expand
305 block6c_project_conv	341 block6f_expand_activation	377 block7b_se_excite
306 block6c_project_bn	342 block6f_dwconv	378 block7b_project_conv
307 block6c_drop	343 block6f_bn	379 block7b_project_bn
308 block6c_add	344 block6f_activation	380 block7b_drop
309 block6d_expand_conv	345 block6f_se_squeeze	381 block7b_add
310 block6d_expand_bn	346 block6f_se_reshape	382 top_conv
311 block6d_expand_activation	347 block6f_se_reduce	383 top_bn
312 block6d_dwconv	348 block6f_se_expand	384 top_activation
313 block6d_bn	349 block6f_se_excite	
314 block6d_activation	350 block6f_project_conv	
315 block6d_se_squeeze	351 block6f_project_bn	
316 block6d_se_reshape	352 block6f_drop	
317 block6d_se_reduce	353 block6f_add	
318 block6d_se_expand	354 block7a_expand_conv	
319 block6d_se_excite	355 block7a_expand_bn	
320 block6d_project_conv	356 block7a_expand_activation	
321 block6d_project_bn	357 block7a_dwconv	
322 block6d_drop	358 block7a_bn	
323 block6d_add	359 block7a_activation	
324 block6e_expand_conv	360 block7a_se_squeeze	
325 block6e_expand_bn	361 block7a_se_reshape	
326 block6e_expand_activation	362 block7a_se_reduce	
327 block6e_dwconv	363 block7a_se_expand	
328 block6e_bn	364 block7a_se_excite	
329 block6e_activation	365 block7a_project_conv	

ΠΑΡΑΡΤΗΜΑ ΧΙ

Αρχιτεκτονική Επιπέδων (Layers) Βάσης InceptionV3, χωρίς κεφαλή:

Total layers in InceptionV3: 311

0 input_layer_1	34 batch_normalization_104	68 conv2d_117
1 conv2d_94	35 batch_normalization_105	69 batch_normalization_114
2 batch_normalization_94	36 activation_99	70 batch_normalization_117
3 activation_94	37 activation_101	71 activation_114
4 conv2d_95	38 activation_104	72 activation_117
5 batch_normalization_95	39 activation_105	73 average_pooling2d_11
6 activation_95	40 mixed0	74 conv2d_113
7 conv2d_96	41 conv2d_109	75 conv2d_115
8 batch_normalization_96	42 batch_normalization_109	76 conv2d_118
9 activation_96	43 activation_109	77 conv2d_119
10 max_pooling2d_4	44 conv2d_107	78 batch_normalization_113
11 conv2d_97	45 conv2d_110	79 batch_normalization_115
12 batch_normalization_97	46 batch_normalization_107	80 batch_normalization_118
13 activation_97	47 batch_normalization_110	81 batch_normalization_119
14 conv2d_98	48 activation_107	82 activation_113
15 batch_normalization_98	49 activation_110	83 activation_115
16 activation_98	50 average_pooling2d_10	84 activation_118
17 max_pooling2d_5	51 conv2d_106	85 activation_119
18 conv2d_102	52 conv2d_108	86 mixed2
19 batch_normalization_102	53 conv2d_111	87 conv2d_121
20 activation_102	54 conv2d_112	88 batch_normalization_121
21 conv2d_100	55 batch_normalization_106	89 activation_121
22 conv2d_103	56 batch_normalization_108	90 conv2d_122
23 batch_normalization_100	57 batch_normalization_111	91 batch_normalization_122
24 batch_normalization_103	58 batch_normalization_112	92 activation_122
25 activation_100	59 activation_106	93 conv2d_120
26 activation_103	60 activation_108	94 conv2d_123
27 average_pooling2d_9	61 activation_111	95 batch_normalization_120
28 conv2d_99	62 activation_112	96 batch_normalization_123
29 conv2d_101	63 mixed1	97 activation_120
30 conv2d_104	64 conv2d_116	98 activation_123
31 conv2d_105	65 batch_normalization_116	99 max_pooling2d_6
32 batch_normalization_99	66 activation_116	100 mixed3
33 batch_normalization_101	67 conv2d_114	101 conv2d_128

102 batch_normalization_128	139 conv2d_135	176 activation_150
103 activation_128	140 conv2d_140	177 conv2d_146
104 conv2d_129	141 batch_normalization_135	178 conv2d_151
105 batch_normalization_129	142 batch_normalization_140	179 batch_normalization_146
106 activation_129	143 activation_135	180 batch_normalization_151
107 conv2d_125	144 activation_140	181 activation_146
108 conv2d_130	145 conv2d_136	182 activation_151
109 batch_normalization_125	146 conv2d_141	183 average_pooling2d_14
110 batch_normalization_130	147 batch_normalization_136	184 conv2d_144
111 activation_125	148 batch_normalization_141	185 conv2d_147
112 activation_130	149 activation_136	186 conv2d_152
113 conv2d_126	150 activation_141	187 conv2d_153
114 conv2d_131	151 average_pooling2d_13	188 batch_normalization_144
115 batch_normalization_126	152 conv2d_134	189 batch_normalization_147
116 batch_normalization_131	153 conv2d_137	190 batch_normalization_152
117 activation_126	154 conv2d_142	191 batch_normalization_153
118 activation_131	155 conv2d_143	192 activation_144
119 average_pooling2d_12	156 batch_normalization_134	193 activation_147
120 conv2d_124	157 batch_normalization_137	194 activation_152
121 conv2d_127	158 batch_normalization_142	195 activation_153
122 conv2d_132	159 batch_normalization_143	196 mixed6
123 conv2d_133	160 activation_134	197 conv2d_158
124 batch_normalization_124	161 activation_137	198 batch_normalization_158
125 batch_normalization_127	162 activation_142	199 activation_158
126 batch_normalization_132	163 activation_143	200 conv2d_159
127 batch_normalization_133	164 mixed5	201 batch_normalization_159
128 activation_124	165 conv2d_148	202 activation_159
129 activation_127	166 batch_normalization_148	203 conv2d_155
130 activation_132	167 activation_148	204 conv2d_160
131 activation_133	168 conv2d_149	205 batch_normalization_155
132 mixed4	169 batch_normalization_149	206 batch_normalization_160
133 conv2d_138	170 activation_149	207 activation_155
134 batch_normalization_138	171 conv2d_145	208 activation_160
135 activation_138	172 conv2d_150	209 conv2d_156
136 conv2d_139	173 batch_normalization_145	210 conv2d_161
137 batch_normalization_139	174 batch_normalization_150	211 batch_normalization_156
138 activation_139	175 activation_145	212 batch_normalization_161

213 activation_156	250 batch_normalization_174	287 activation_180
214 activation_161	251 activation_174	288 activation_184
215 average_pooling2d_15	252 conv2d_171	289 conv2d_181
216 conv2d_154	253 conv2d_175	290 conv2d_182
217 conv2d_157	254 batch_normalization_171	291 conv2d_185
218 conv2d_162	255 batch_normalization_175	292 conv2d_186
219 conv2d_163	256 activation_171	293 average_pooling2d_17
220 batch_normalization_154	257 activation_175	294 conv2d_179
221 batch_normalization_157	258 conv2d_172	295 batch_normalization_181
222 batch_normalization_162	259 conv2d_173	296 batch_normalization_182
223 batch_normalization_163	260 conv2d_176	297 batch_normalization_185
224 activation_154	261 conv2d_177	298 batch_normalization_186
225 activation_157	262 average_pooling2d_16	299 conv2d_187
226 activation_162	263 conv2d_170	300 batch_normalization_179
227 activation_163	264 batch_normalization_172	301 activation_181
228 mixed7	265 batch_normalization_173	302 activation_182
229 conv2d_166	266 batch_normalization_176	303 activation_185
230 batch_normalization_166	267 batch_normalization_177	304 activation_186
231 activation_166	268 conv2d_178	305 batch_normalization_187
232 conv2d_167	269 batch_normalization_170	306 activation_179
233 batch_normalization_167	270 activation_172	307 mixed9_1
234 activation_167	271 activation_173	308 concatenate_3
235 conv2d_164	272 activation_176	309 activation_187
236 conv2d_168	273 activation_177	310 mixed10
237 batch_normalization_164	274 batch_normalization_178	
238 batch_normalization_168	275 activation_170	
239 activation_164	276 mixed9_0	
240 activation_168	277 concatenate_2	
241 conv2d_165	278 activation_178	
242 conv2d_169	279 mixed9	
243 batch_normalization_165	280 conv2d_183	
244 batch_normalization_169	281 batch_normalization_183	
245 activation_165	282 activation_183	
246 activation_169	283 conv2d_180	
247 max_pooling2d_7	284 conv2d_184	
248 mixed8	285 batch_normalization_180	
249 conv2d_174	286 batch_normalization_184	

ΒΙΒΛΙΟΓΡΑΦΙΚΕΣ ΑΝΑΦΟΡΕΣ

- [1] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, Academic Press, 2006.
- [2] P.-N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, *Introduction to Data Mining*, 2nd ed., Pearson, 2018.
- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Prentice Hall, 2010.
- [4] J. Grus, *Data Science from Scratch: First Principles with Python*, 2nd ed., O'Reilly Media, 2019.
- [5] D. E. Comer, *Internetworking with TCP/IP: Principles, Protocols, and Architectures*, 6th ed., Pearson, 2013.
- [6] Δ. Γαβαλάς, Β. Κασσπάκης, και Θ. Χατζηδημητρίου, *Κινητές Τεχνολογίες*, Εκδόσεις Νέων Τεχνολογιών, 2021.
- [7] H. M. Deitel and P. J. Deitel, *Internet & World Wide Web: How to Program*, 4th ed., Prentice Hall, 2008.
- [8] B. Chen et al., "Blind People: Clothing Category Classification and Stain Detection Using Transfer Learning," *Applied Sciences*, vol. 13, no. 3, p. 1925, 2023.
- [9] Wikipedia contributors, "Artificial intelligence," *Wikipedia*, The Free Encyclopedia, 18 June 2025. [Online]. Available: https://en.wikipedia.org/wiki/Artificial_intelligence. [Accessed: 04-Jul-2024].
- [10] Wikipedia contributors, "Machine learning," *Wikipedia*, The Free Encyclopedia, 20 June 2025. [Online]. Available: https://en.wikipedia.org/wiki/Machine_learning. [Accessed: 04-Jul-2024].
- [11] Wikipedia contributors, "Convolutional neural network," *Wikipedia*, The Free Encyclopedia, 22 June 2025. [Online]. Available: https://en.wikipedia.org/wiki/Convolutional_neural_network. [Accessed: 04-Jul-2024].
- [12] Stanford University, "Convolutional Neural Networks for Visual Recognition," CS231n: Convolutional Neural Networks for Visual Recognition, 2025. [Online]. Available: <https://cs231n.github.io/convolutional-networks/>. [Accessed: 04-Jul-2024].
- [13] J. Brownlee, "How to Use Transfer Learning When Developing Convolutional Neural Network Models," *Machine Learning Mastery*, 2023. [Online]. Available: <https://machinelearningmastery.com/how-to-use-transfer-learning-when-developing-convolutional-neural-network-models/>. [Accessed: 04-Jul-2024].
- [14] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, p. 9, May 2016. [Online]. Available: <https://doi.org/10.1186/s40537-016-0043-6>. [Accessed: 04-Jul-2024].
- [15] S. Ruder, "An Overview of Multi-Task Learning in Deep Neural Networks," arXiv preprint arXiv:1706.05098, 15 June 2017. [Online]. Available: <https://arxiv.org/pdf/1706.05098>. [Accessed: 04-Jul-2024].
- [16] T. Standley, A. R. Zamir, D. Chen, L. Guibas, J. Malik, and S. Savarese, "Which Tasks Should Be Learned Together in Multi-task Learning?" arXiv preprint arXiv:1905.07553, 3 September 2020. [Online]. Available: <https://arxiv.org/pdf/1905.07553>. [Accessed: 04-Jul-2024].

- [17] Z. Chen, S. Badrinarayanan, A. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," arXiv preprint arXiv:1711.02257, 2017. [Online]. Available: <https://arxiv.org/abs/1711.02257>. [Accessed: 04-Jul-2024].
- [18] TensorFlow, "Transfer learning and fine-tuning," TensorFlow Tutorials, 2025. [Online]. Available: https://www.tensorflow.org/tutorials/images/transfer_learning#important_note_about_batchnormalization_layers. [Accessed: 04-Jul-2024].
- [19] S. V. Senthilnath, P. S. Kumar, and P. K. Mallick, "Transfer learning-based approach for image classification using convolutional neural networks," *Applied Sciences*, vol. 13, no. 3, p. 1925, 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/3/1925>. [Accessed: 04-Jul-2024].
- [20] S. V. Kumar and A. S. Reddy, "Deep transfer learning for image classification: A review," *Information*, vol. 15, no. 4, p. 196, 2024. [Online]. Available: <https://www.mdpi.com/2078-2489/15/4/196>. [Accessed: 04-Jul-2024].
- [21] Keras Team, "Transfer learning and fine-tuning," Keras Guides, 2025. [Online]. Available: https://keras.io/guides/transfer_learning/. [Accessed: 04-Jul-2024].
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv preprint arXiv:1801.06146, 2018. [Online]. Available: <https://arxiv.org/pdf/1801.06146>. [Accessed: 04-Jul-2024].
- [23] Wikipedia contributors, "Multi-task learning," *Wikipedia*, The Free Encyclopedia, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Multi-task_learning. [Accessed: 04-Jul-2024].
- [24] M. R. Caruana, "Multitask Learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997. [Online]. Available: https://www.researchgate.net/publication/323599831_An_overview_of_multi-task_learning. [Accessed: 04-Jul-2024].
- [25] Z. Wen, "Multi-task Learning in Deep Neural Networks," Iowa State University, Master's Thesis, 2020. [Online]. Available: <https://dr.lib.iastate.edu/server/api/core/bitstreams/e72a8f4f-adb3-48d3-9831-0bc95e7e1edb/content>. [Accessed: 04-Jul-2024].
- [26] World Health Organization, "Up to 45 million blind people globally — and growing," WHO, 09-Oct-2003. [Online]. Available: <https://www.who.int/news/item/09-10-2003-up-to-45-million-blind-people-globally---and-growing#:~:text=Globally%2C%20an%20estimated%2040%20to,135%20million%20have%20low%20vision>. [Accessed: 04-Jul-2024].
- [27] Y. Tay, M. Dehghani, J. Gupta, D. Bahri, V. Aribandi, Z. Qin, D. Metzler, "Are Pre-trained Convolutions Better than Pre-trained Transformers?," arXiv preprint arXiv:2110.07143, 2021. [Accessed: 04-Jul-2024]. Available: <https://arxiv.org/abs/2105.03322>
- [28] Z. Lu, H. Xie, C. Liu, Y. Zhang, *Bridging the Gap Between Vision Transformers and Convolutional Neural Networks on Small Datasets*, arXiv preprint arXiv:2104.13615, 2021. [Accessed: 04-Jul-2024]. Available: <https://arxiv.org/abs/2210.05958>
- [29] Keras Applications - EfficientNet, Keras Documentation. [Accessed: 04-Jul-2024]. Available: <https://keras.io/api/applications/efficientnet/>

- [30] *TensorFlow EfficientNetB3 API Documentation*, TensorFlow. [Accessed: 04-Jul-2024]. Available:
https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetB3
- [31] *TensorFlow InceptionV3 API Documentation*, TensorFlow. [Accessed: 04-Jul-2024]. Available:
https://www.tensorflow.org/api_docs/python/tf/keras/applications/InceptionV3
- [32] *Transfer Learning Guide*, Keras. [Accessed: 04-Jul-2024]. Available:
https://keras.io/guides/transfer_learning/
- [33] *Softmax function*, Wikipedia. [Accessed: 04-Jul-2024]. Available:
https://en.wikipedia.org/wiki/Softmax_function/